

MODUL PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

S1 INFORMATIKA



Published by school of computing

Our official instagram



@informaticslab_telu

Lembar Pengesahan

Saya yang bertanda tangan di bawah ini:

Nama : MONTERICO ADRIAN, S.T., M.T.
NIK : 20870024
Koordinator Mata Kuliah : Pemrograman Berorientasi Objek
Prodi : S1 Informatika

Menerangkan dengan sesungguhnya bahwa modul ini digunakan untuk pelaksanaan praktikum di Semester Genap Tahun Ajaran 2024/2025 di Laboratorium Informatika Fakultas Informatika Universitas Telkom.

Bandung, Februari 2025



Mengesahkan,
Koordinator Mata Kuliah
Pemrograman Berorientasi Objek

MONTERICO ADRIAN, S.T., M.T.



Fakultas Informatika
School of Computing
Telkom Unive

Mengetahui,
Kaprodi S1 Informatika

Dr. ERWIN BUDI SETIAWAN, S.Si., M.T.

Peraturan Praktikum Laboratorium Informatika 2024/2025

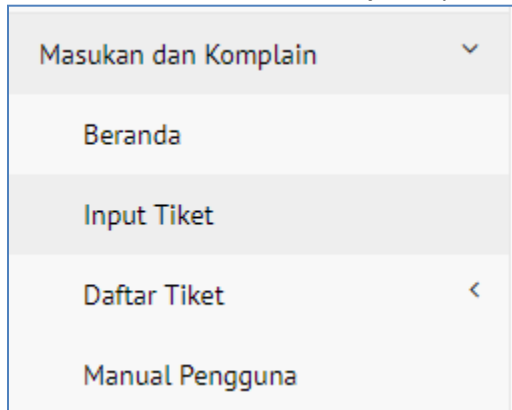
1. Praktikum diampu oleh dosen kelas dan dibantu oleh asisten laboratorium dan asisten praktikum.
2. Praktikum dilaksanakan di Gedung TULT (Telkom University Landmark Tower) lantai 6 dan 7 sesuai jadwal yang ditentukan.
3. Praktikan wajib membawa modul praktikum, kartu praktikum, dan alat tulis.
4. Praktikan wajib mengecek kehadiran di igracias dan sheet yang dibagikan asisten.
5. Durasi kegiatan praktikum S-1 = 2 jam (100 menit).
6. Jumlah pertemuan praktikum:
 - 16 kali pertemuan
7. Praktikan wajib hadir minimal 75% dari seluruh pertemuan praktikum di lab.
8. Praktikan yang datang terlambat :
 - ≤ 5 menit : diperbolehkan mengikuti praktikum tanpa tambahan praktikum
 - ≥ 30 menit : tidak diperbolehkan mengikuti praktikum
9. Saat praktikum berlangsung, asisten praktikum dan praktikan:
 - Wajib menggunakan seragam sesuai aturan institusi.
 - Wajib mematikan/ mengkondisikan semua alat komunikasi.
 - Dilarang membuka aplikasi yang tidak berhubungan dengan praktikum yang berlangsung.
 - Dilarang mengubah pengaturan *software* maupun *hardware* komputer tanpa ijin.
 - Dilarang membawa makanan maupun minuman di ruang praktikum.
 - Dilarang memberikan jawaban ke praktikan lain.
 - Dilarang menyebarkan soal praktikum.
 - Dilarang membuang sampah di ruangan praktikum.
 - Wajib meletakkan alas kaki dengan rapi pada tempat yang telah disediakan.
10. Setiap praktikan dapat mengikuti praktikum susulan maksimal dua modul untuk satu mata kuliah praktikum.
 - Praktikan yang dapat mengikuti praktikum susulan hanyalah praktikan yang memenuhi syarat sesuai ketentuan institusi, yaitu: sakit (dibuktikan dengan surat keterangan medis), tugas dari institusi (dibuktikan dengan surat dinas atau dispensasi dari institusi), atau mendapat musibah atau keduakaan (menunjukkan surat keterangan dari orangtua/wali mahasiswa.)
 - Persyaratan untuk praktikum susulan diserahkan sesegera mungkin kepada asisten laboratorium untuk keperluan administrasi.
 - Praktikan yang diijinkan menjadi peserta praktikum susulan ditetapkan oleh Lab Informatika dan tidak dapat diganggu gugat.
11. a. Ketidakhadiran pada kelas praktikum
 - **Nilai modul = 0**
- b. Meminta, mendapatkan, dan menyebarluaskan soal dan atau kunci jawaban praktikum:
 - **Penyebar soal dan kunci jawaban : Pengajuan sanksi kepada Komisi Disiplin Fakultas**
 - **Penerima soal dan kunci jawaban: Nilai '0' pada (seluruh assessment) praktikum**
- c. Lupa menghapus file praktikum
 - **Pengurangan nilai modul 20%**
- d. Memicu kegaduhan, sehingga membuat situasi tidak kondusif (jalan-jalan, mengganggu teman, mengobrol, dll), asisten praktikum diwajibkan menegur sebanyak 3x.

- Pengurangan nilai modul 50%
- e. MENYALAHGUNAKAN FITUR LMS
- Siap menerima sanksi dari lab

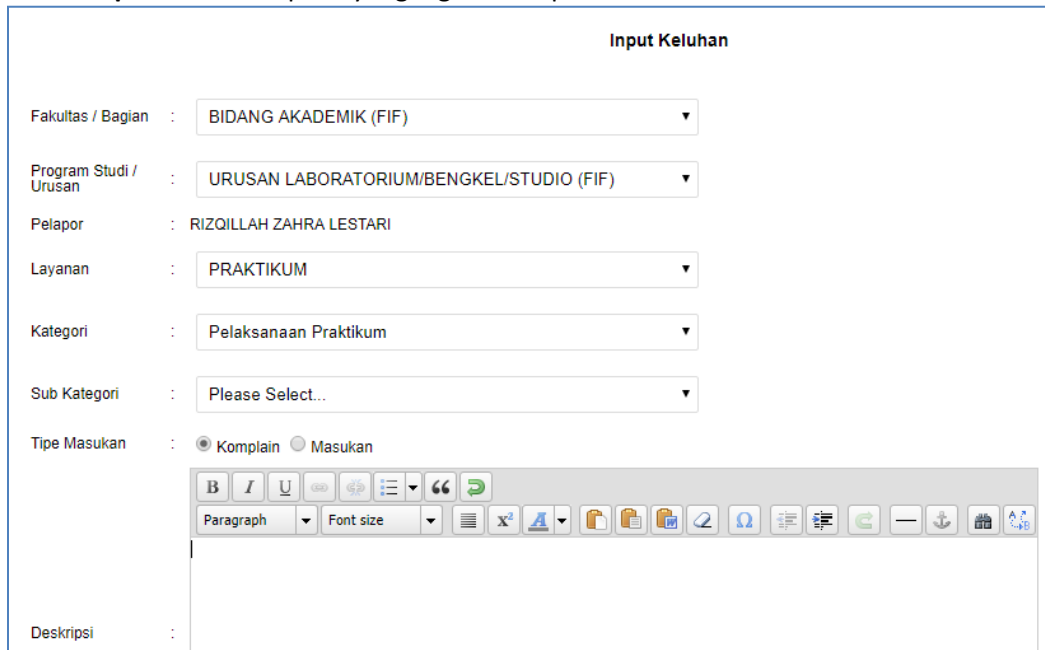


Tata Cara Komplain Praktikum IFLAB Melalui IGRACIAS

1. Login Igracias.
2. Pilih Menu **Masukan dan Komplain**, pilih **Input Tiket**.



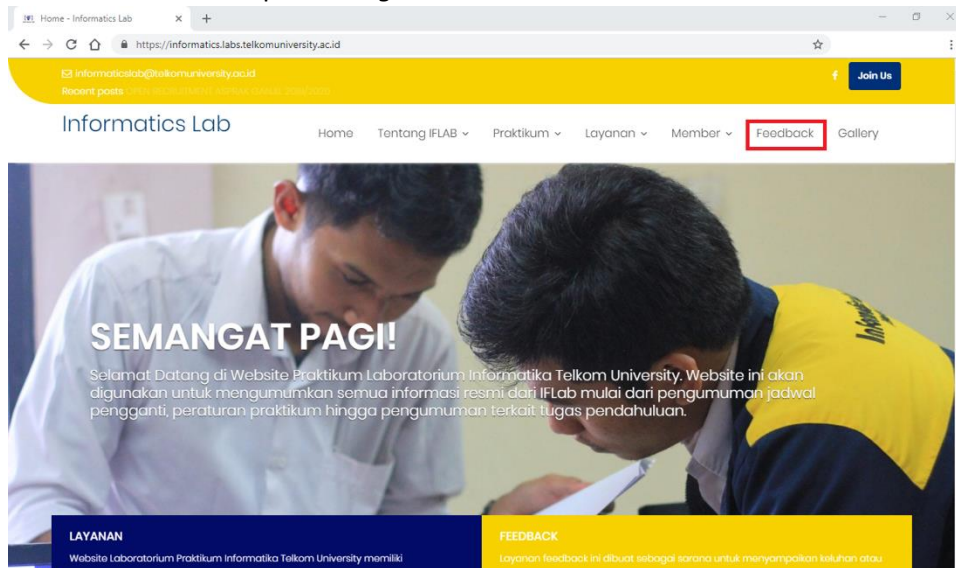
3. Pilih Fakultas/Bagian: **Bidang Akademik (FIF)**.
4. Pilih Program Studi/Urusan: **Urusan Laboratorium/Bengkel/Studio (FIF)**.
5. Pilih Layanan: **Praktikum**.
6. Pilih Kategori: **Pelaksanaan Praktikum**, lalu pilih **Sub Kategori**.
7. Isi **Deskripsi** sesuai komplain yang ingin disampaikan.

A screenshot of the 'Input Keluhan' form. The form contains several dropdown menus and a text input field. The fields are: 'Fakultas / Bagian' (set to 'BIDANG AKADEMIK (FIF)'), 'Program Studi / Urusan' (set to 'URUSAN LABORATORIUM/BENGKEL/STUDIO (FIF)'), 'Pelapor' (set to 'RIZQILLAH ZAHRA LESTARI'), 'Layanan' (set to 'PRAKTIKUM'), 'Kategori' (set to 'Pelaksanaan Praktikum'), 'Sub Kategori' (set to 'Please Select...'), and 'Tipe Masukan' (with radio buttons for 'Komplain' and 'Masukan', where 'Komplain' is selected). Below these fields is a rich text editor for 'Deskripsi' with a toolbar containing various formatting options like bold, italic, underline, link, unlink, and text color.

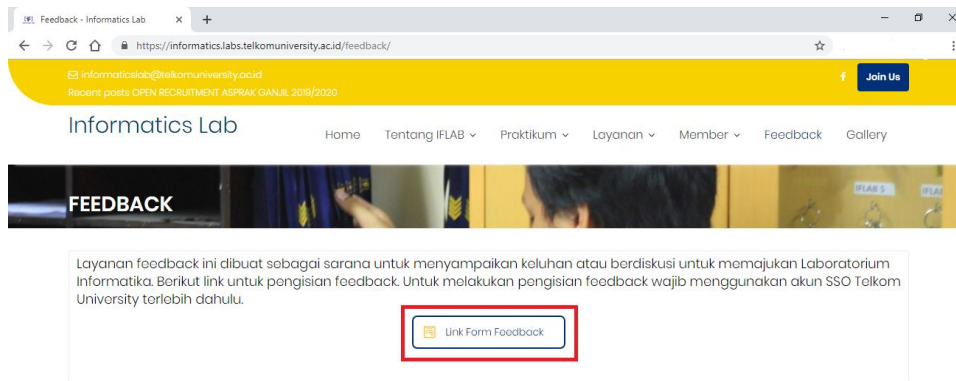
8. Lampirkan *file* jika perlu. Lalu klik Kirim.

Tata Cara Komplain Praktikum IFLAB Melalui Website

1. Buka website <https://informatics.labs.telkomuniversity.ac.id/> melalui browser.
2. Pilih menu **Feedback** pada *navigation bar website*.



3. Pilih tombol **Link Form Feedback**.



4. Lakukan *login* menggunakan akun **SSO Telkom University** untuk mengakses *form feedback*.
5. Isi *form* sesuai dengan *feedback* yang ingin diberikan.

Daftar Isi

Lembar Pengesahan.....	i
Peraturan Praktikum Laboratorium Informatika 2024/2025	ii
Tata Cara Komplain Praktikum IFLAB Melalui IGRACIAS	iv
Tata Cara Komplain Praktikum IFLAB Melalui <i>Website</i>	v
Daftar Isi	vi
Daftar Gambar	x
Modul 1 Running Modul	1
1.1 Pengenalan Java.....	1
1.1.1 Sejarah Singkat Java.....	1
1.1.2 Karakteristik Java	1
1.1.3 Garbage Collection.....	2
1.2 Instalasi Netbeans.....	2
1.2.1 Instalasi JDK (Java Development Kit)	2
1.2.2 Netbeans.....	3
Modul 2 Intro Pemrograman Berorientasi Objek	5
2.1 Tipe Data dan Variabel.....	5
2.2 Konstanta.....	6
2.3 Variabel Array	6
2.3.1 Deklarasi Variabel Array	6
2.3.2 Mengakses Variabel Array	7
2.3.3 Array 2 Dimensi.....	7
2.3.4 Array Bertipe Objek	8
2.4 Operator, Pernyataan Kondisional, dan Perulangan	9
2.4.1 Operator	9
2.4.2 Pernyataan Kondisional	11
2.4.3 Perulangan	13
2.5 Compile, Run, dan Jar File.....	14
2.5.1 Compile	14
2.5.2 Run	14
2.5.3 Jar File	14
Modul 3 Kelas dan Objek	16
3.1 Pendahuluan	16
3.1.1 Abstraksi	16
3.1.2 Enkapsulasi	16
3.1.3 Pewarisan (Inheritance).....	17
3.1.4 Polymorphisms	17

3.1.5	Message (Komunikasi Antar Objek).....	18
3.2	Kelas.....	18
3.3	Object.....	18
3.4	Field.....	19
3.5	Method	19
3.6	Keyword “this”	21
Modul 4	Encapsulation.....	24
4.1	Java Modifier	24
4.2	Constructor	26
4.3	Package	28
Modul 5	Relasi Antar Kelas.....	30
5.1	Diagram Kelas	30
5.2	Hubungan Antar Kelas	31
5.2.1	Asosiasi	31
5.2.2	Agregasi.....	33
5.2.3	Komposisi.....	34
Modul 6	Inheritance.....	35
6.1	Pendahuluan.....	35
6.2	Inheritance.....	35
6.3	Keyword Super.....	36
6.4	Overloading Method.....	37
6.5	Overriding Method	38
Modul 7	Abstract dan Interface	40
7.1	Kelas Abstrak.....	40
7.2	Interface.....	42
Modul 8	Presentasi Tugas Besar (Diagram Kelas)	47
8.1	Presentasi Tugas Besar	47
Modul 9	Polymorphism, Static Class, & Collection	48
9.1	Pendahuluan	48
9.2	Referensi Variabel Casting	48
9.3	Static	51
9.3.1	Static Variabel	51
9.3.2	Static <i>Method</i>	51
9.4	Collection	52
9.4.1	Melakukan Sorting pada Collection	53
9.4.2	Melakukan Filtering pada Collection	55
Modul 10	Exception	57

10.1	Exception	57
10.2	Eksepsi yang tidak dicek	57
10.3	Eksepsi yang dicek	58
10.4	Penggunaan Blok Try Catch	58
10.5	Penggunaan Throws	59
10.6	Pemakaian Finally	59
Modul 11	Web Frontend	61
11.1	HTML.....	61
11.1.1	Pengenalan HTML.....	61
11.1.2	Dasar Sintaks HTML	62
11.1.3	Heading.....	63
11.1.4	Hyperlink.....	63
11.1.5	Tabel	64
11.1.6	Image	65
11.1.7	Audio / Video Elemen	66
11.1.8	Form.....	66
11.2	CSS.....	70
11.2.1	Pengenalan CSS.....	70
11.2.2	Font Propertis	72
11.2.3	List Properties	72
11.2.4	Alignment of Text	74
11.2.5	Colors	75
11.2.6	Span & Div.....	76
11.3	Bootstrap	76
11.3.1	Pemasangan Bootstrap.....	76
11.3.2	Bootstrap Container	77
11.3.3	Bootstrap Grid	77
11.3.4	Text Style.....	78
11.3.5	Table, Image, & Button.....	79
11.3.6	Bootstrap Form	82
11.4	Javascript	83
11.4.1	Pengenalan Javascript.....	83
11.4.2	Sintaks Umum pada Javascript	83
11.4.3	Object Orientation pada Javascript	85
11.4.4	Function pada Javascript	87
Modul 12	JSP	91
12.1	Pengenalan JSP	91

12.2	Java Web Server.....	91
Modul 13	Java Servlet & JDBC Part 1	93
13.1	Java Servlet	93
13.2	Memulai JDBC.....	94
13.3	Fetching Data	98
13.4	Insert Data	99
Modul 14	JDBC part 2.....	101
14.1	Pendahuluan	101
14.2	Update Data	101
14.3	Delete Data	102
Modul 15	JDBC Part 3 & Session	103
15.1	Pendahuluan	103
15.2	Session	103
Modul 16	Responsi & Progress Tugas Besar	105
16.1	Responsi & Progress Tugas Besar	105
Daftar Pustaka		106



Daftar Gambar

Gambar 1-1 Tampilan awal instalasi Java (JDK 22.0.2).....	2
Gambar 1-2 Pemilihan tempat penginstalan Java.....	3
Gambar 1-3 Tampilan Dialog <i>New Project</i>	3
Gambar 1-4 Tampilan Run Main Project.	4
Gambar 1-5 Tampilan hasil <i>compile</i> dan <i>run</i> program <i>hello world</i>	4
Gambar 4-1 Struktur <i>file</i> program harga token dan harga pulsa.	28
Gambar 5-1 Diagram Kelas.	30
Gambar 5-2 Hubungan Asosiasi.....	30
Gambar 5-3 Hubungan Agregasi.....	30
Gambar 5-4 Hubungan Komposisi	31
Gambar 5-5 Hubungan Generalisasi.....	31
Gambar 5-6 Contoh hubungan asosiasi.....	31
Gambar 6-1 Ilustrasi pada kelas manusia.	35
Gambar 6-2 Contoh kelas dengan <i>keyword super</i>	36
Gambar 7-1 Contoh kelas abstrak dalam UML.....	40
Gambar 7-2 Contoh kelas <i>interface</i> dalam UML.	43
Gambar 9-1 Hierarki dari <i>Collection</i>	52
Gambar 10-1 Hierarki dari tipe-tipe eksepsi.	57
Gambar 11-1 Struktur dasar HTML.....	61
Gambar 11-2 Tingkatan heading	63
Gambar 11-3 Tabel pada HTML.....	64
Gambar 11-4 Penggunaan <i>colspan</i> dan <i>rowspan</i> pada table HTML	65
Gambar 11-5 Penggunaan <i>image</i> pada HTML	65
Gambar 11-6 Penggunaan <i>audio</i> dan <i>video</i> pada HTML.....	66
Gambar 11-7 Contoh form pendaftaran	70
Gambar 11-8 Penulisan CSS.....	70
Gambar 11-9 Penerapan font properties	72
Gambar 11-10 Tampilan penggunaan list properties.....	73
Gambar 11-11 List properties dengan tambahan CSS.....	74
Gambar 11-12 Penerapan alignment of text.....	75
Gambar 11-13 Penerapan Span & Div	76
Gambar 11-14 Tampilan Dialog <i>New Project</i>	91
Gambar 11-15 Tampilan hasil <i>Compile</i> dan <i>run</i> java web.....	92
Gambar 12-1 Tampilan menu membuat Servlet	93
Gambar 12-2 Tampilan dialog <i>New Servlet</i>	93
Gambar 12-3 Tampilan utama Servlet.....	94
Gambar 12-4 Tampilan Menu Libraries	95
Gambar 12-5 Tampilan dialog <i>Add Library</i>	95
Gambar 12-6 Tampilan setelah <i>Add Library JDBC</i>	96
Gambar 12-7 Tampilan menu menambahkan Class.....	96
Gambar 12-8 Tampilan dialog <i>New Java Class</i>	97
Gambar 12-9 Tampilan dialog <i>New JSP</i>	90

Modul 1 Running Modul

Tujuan Praktikum

1. Mengetahui dan mengenal bahasa pemrograman java
2. Mengetahui cara instalasi dan konfigurasi java

1.1 Pengenalan Java

1.1.1 Sejarah Singkat Java



Java dibuat dan diperkenalkan pertama kali oleh sebuah tim Sun Microsystem yang dipimpin oleh **Patrick Naughton** dan **James Gosling** pada tahun 1991 dengan nama kode **Oak**. Tahun 1995 Sun mengubah nama Oak tersebut menjadi **Java**.

Teknologi java diadopsi oleh Netscape tahun 1996. JDK 1.1 diluncurkan tahun 1996, kemudian JDK 1.2, berikutnya J2EE (Java 2 Enterprise Edition) yang berbasis J2SE yaitu servlet, EJB dan JSP. Dan yang terakhir adalah J2ME (Java 2 Micro Edition) yang diadopsi oleh Nokia, Siemens, Motorola, Samsung, dan Sony Ericson



Tahukah kamu?

Ide pertama kali kenapa Java dibuat adalah karena adanya motivasi untuk membuat sebuah bahasa pemrograman yang bersifat *portable* dan *platform independent* (tidak tergantung mesin dan sistem operasi) yang dapat digunakan untuk membuat piranti lunak yang dapat ditanamkan (*embedded*) pada berbagai macam peralatan *electronic consumer* biasa, seperti *microwave*, *remote control*, telepon, *card reader* dan sebagainya.

1.1.2 Karakteristik Java

Karakteristik Java adalah sebagai berikut :

- 1) Berorientasi Objek, Java telah menerapkan konsep pemrograman berorientasi objek yang modern dalam implementasinya.
- 2) *Robust*, java mendorong pemrograman yang bebas dari kesalahan dengan bersifat *strongly typed* dan memiliki *run-time checking*
- 3) *Portable*, Program Java dapat dieksekusi di platform manapun selama tersedia Java Virtual Machine untuk *platform* tersebut.
- 4) *Multithreading*, Java mendukung penggunaan *multithreading* yang telah terintegrasi langsung dalam bahasa Java
- 5) Dinamis, Program Java dapat melakukan suatu tindakan yang ditentukan pada saat eksekusi program dan bukan pada saat kompilasi
- 6) Sederhana, Java menggunakan bahasa yang sederhana dan mudah dipelajari.
- 7) Terdistribusi, java didesain untuk berjalan pada lingkungan yang terdistribusi seperti halnya internet.
- 8) Aman, Java memiliki arsitektur yang kokoh dan pemrograman yang aman.

1.1.3 Garbage Collection

Teknik yang digunakan Java untuk penanganan objek yang telah tidak diperlukan untuk dimusnahkan (dikembalikan ke *pool memory*) disebut garbage collection. *Java Interpreter* melakukan pemeriksaan untuk mengetahui apakah objek di memori masih diacu oleh program. *Java Interpreter* akan mengetahui objek yang telah tidak dipakai oleh program. Ketika *Java interpreter* mengetahui objek itu, maka *Java interpreter* akan memusnahkan secara aman.

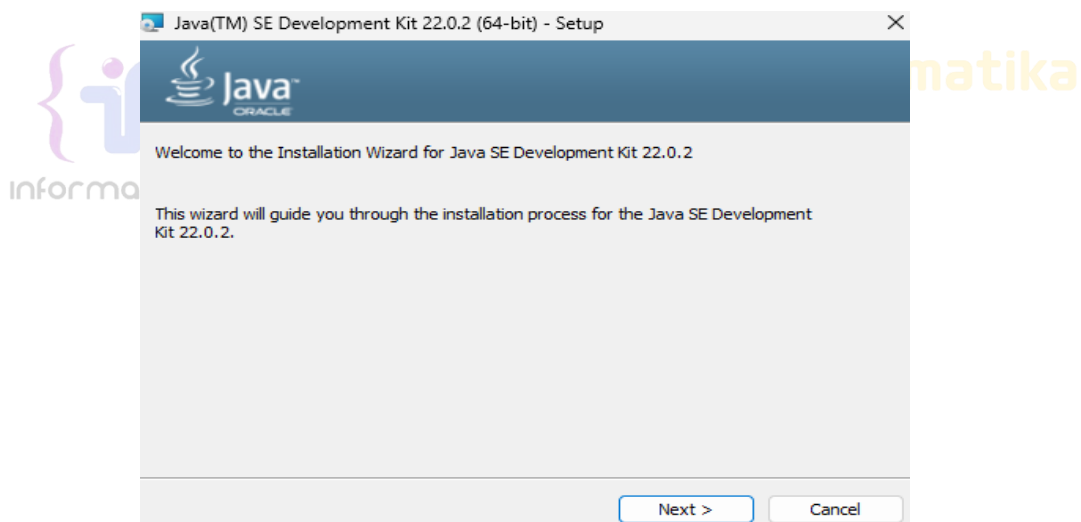
Java garbage collector berjalan sebagai thread berprioritas rendah dan melakukan kerja saat tidak ada kerja lain di program. Umumnya, *Java garbage collector* bekerja saat idle pemakai menunggu masukan dari *keyboard* atau *mouse*. *Garbage collector* akan bekerja dengan prioritas tinggi saat interpreter kekurangan memori. Hal ini jarang terjadi karena *thread* berprioritas rendah telah melakukan tugas dengan baik secara background.

1.2 Instalasi Netbeans

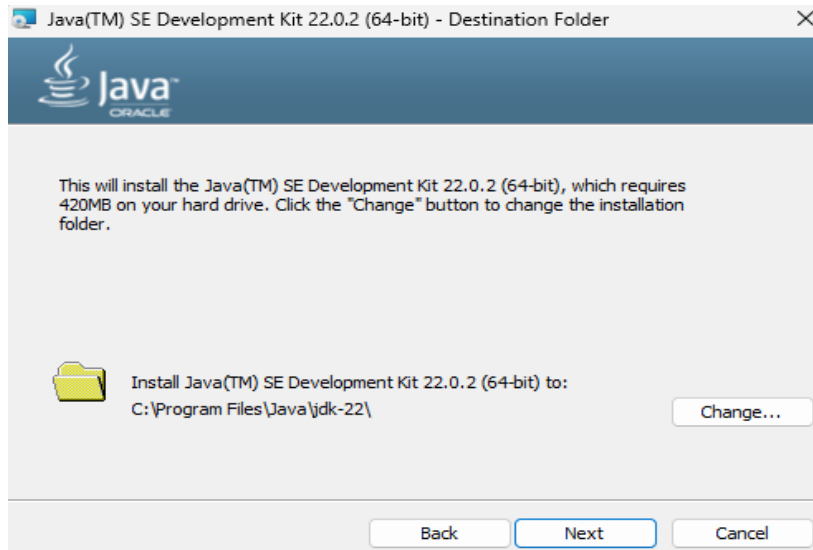
Sebelum *install* Netbeans Kita perlu *install* JDK terlebih dahulu.

1.2.1 Instalasi JDK (Java Development Kit)

Pada Windows, *file* instalasi berupa **self-installing file**, yaitu *file exe* yang bila dijalankan akan mengekstrak dirinya untuk dikopikan ke direktori. Prosedur instalasi akan menawarkan *default* untuk direktori instalasi seperti jdk 17.0.12, jdk 21.0.4, atau yang terbaru jdk 22.0.2. Kita sebaiknya tidak mengubahnya karena Kita dapat mengkoleksi beragam versi JDK, secara *default*.



Gambar 1-1 Tampilan awal instalasi Java (JDK 22.0.2).



Gambar 1-2 Pemilihan tempat penginstalan Java.

Selanjutnya klik “Next” dan tinggal mengikuti semua alur instalasi sampai proses instalasi selesai.

1.2.2 Netbeans

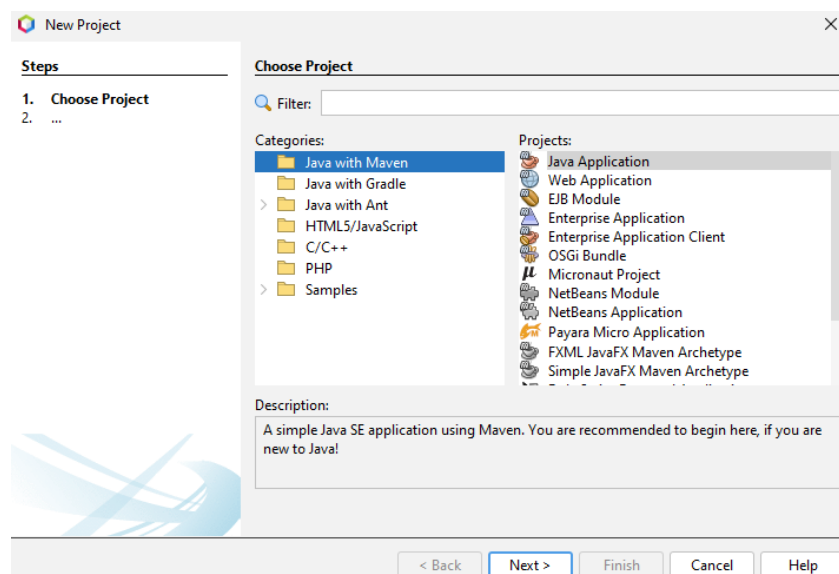
Setelah instalasi JDK selesai, tahap selanjutnya adalah melakukan instalasi Netbeans terbaru yang dapat diunduh di <http://www.netbeans.org>. Proses instalasi dapat dilakukan sama seperti instalasi *software* pada umumnya yang hanya mengikuti alur instalasi hingga proses selesai.

Di dalam netbeans, semua perancangan dan pemrograman dilakukan di dalam kerangka sebuah proyek. Proyek netbeans merupakan sekumpulan *file* yang dikelompokkan di dalam suatu kesatuan.

1) Membuat proyek

Sebelum membuat program Java dengan menggunakan Netbeans, langkah pertama adalah membuat proyek terlebih dahulu.

- a. Jalankan *menu File | Project* (Ctrl+Shift+N) untuk membuka suatu dialog *New Project*.



Gambar 1-3 Tampilan Dialog *New Project*.

- b. Dalam dialog, pilih *Categories : Java with Maven* dan *Projects : Java Application*. Lalu klik tombol *next*.
- c. Pilih dahulu lokasi *project* dengan menekan tombol *browse*. Akan tampil direktori dan Kita dapat memilih di direktori mana *project* disimpan. Setelah memilih, klik *open*.
- d. Kembali ke dialog *new project*, isikan *project name*. Misalnya disini *project name : HelloWorld*. Perhatikan bahwa *project folder* secara otomatis akan diset sesuai dengan *project name*.
- e. Centang pada *Set as Main Project* dan pada *Create Main Class*. Terakhir klik tombol *finish*.

Di dalam netbeans akan dibuka secara otomatis *file* utama Java bernama *main.java* yang berada pada *package helloworld*.

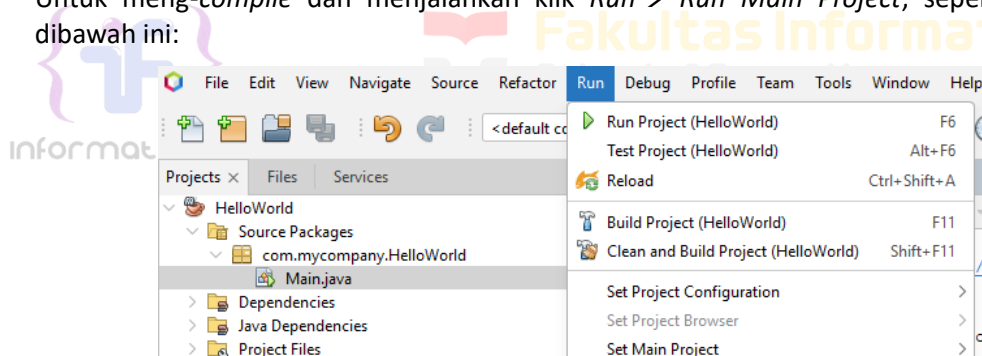
2) Memulai membuat program sederhana di Java

Setelah membuat proyek di Netbeans, selanjutnya membuat program sederhana. Di bawah ini merupakan contoh program yang akan menampilkan "*Hello Word*".

```
public class Main {
    public static void main(String[] args) {
        System.out.println("Hello Word");
    }
}
```

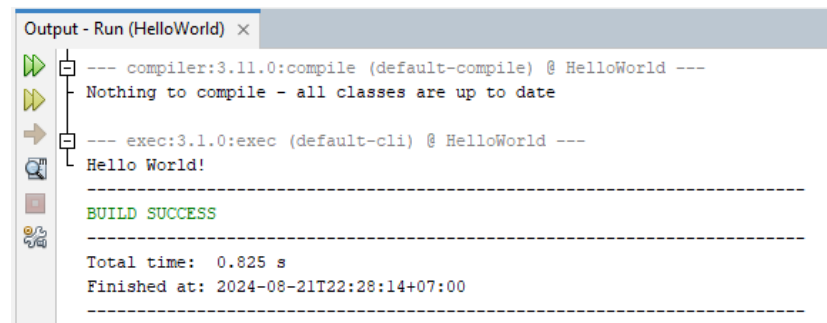
3) Meng-compile dan menjalankan program Java di Netbeans

Untuk meng-compile dan menjalankan klik *Run* → *Run Main Project*, seperti gambar dibawah ini:



Gambar 1-4 Tampilan Run Main Project.

Jika programnya sukses maka akan tampil tulisan *hello word* seperti di bawah ini :



Gambar 1-5 Tampilan hasil compile dan run program hello world.

Modul 2 Intro Pemrograman Berorientasi Objek

Tujuan Praktikum

1. Memahami tipe data, variabel, konstanta dan dapat mengimplementasikannya dalam pemrograman java.
2. Memahami dan mengimplementasikan operator, variabel array, pernyataan kondisional dan perulangan dalam pemrograman java.

2.1 Tipe Data dan Variabel

Hampir bisa dipastikan bahwa sebuah program selalu membutuhkan lokasi memori untuk menyimpan data yang sedang diproses. Kita tidak pernah tahu di lokasi memori mana komputer akan meletakkan data dari program Kita. Kenyataan ini menimbulkan kesulitan bagi Kita untuk mengambil data tersebut pada saat dibutuhkan. Maka dikembangkan konsep variabel. Setiap variabel memiliki jenis alokasi memori tersendiri, misalnya bilangan bulat, bilangan pecahan, karakter, dan sebagainya. Ini sering disebut dengan tipe data.

Secara umum ada tiga bentuk data:

1 2 3 4 5 6 7 8 9 Numerik	A B C D E F G H I J K L Karakter	TRUE FALSE Logika
---	--	---

- 1) Numerik, data yang berbentuk angka atau bilangan. Data numerik bisa dibagi atas dua kategori :
 - a. Bilangan bulat (*integer*), yaitu bilangan yang tidak mengandung angka pecahan.
 - b. Bilangan pecahan (*float*), yaitu bilangan yang mengandung angka pecahan.
- 2) Karakter, data yang berbentuk karakter atau deretan karakter. Karakter bisa dibagi menjadi dua kategori :
 - a. Karakter tunggal.
 - b. Deretan karakter.
- 3) Logika, yaitu tipe data dengan nilai benar (*true*) atau salah (*false*).

Pada dasarnya ada dua macam tipe variabel data dalam bahasa Java, yakni:

- a. Tipe primitif, meliputi : tipe boolean, tipe numerik (yang meliputi : Byte, short, int, long, char, float, double) dan tipe karakter (char).
- b. Tipe referensi, meliputi tipe variabel data : tipe kelas, tipe array, tipe interface. Ada pula tipe variabel data khusus yang disebut null types, namun variabel dalam Java tidak akan memiliki tipe *null* ini.

Bentuk umum pendeklarasian variabel:

```
tipeData namaVariabel1 =[nilaiAwal];  
tipeData namaVariabel1 =[nilaiAwal], [namaVariabel2 = [nilaiAwal], ...];
```

Contoh untuk mendeklarasikan sebuah variabel :

```
int dataint;  
char chardata;  
float x = 12,67;
```

Penamaan sebuah variabel dalam Java harus memenuhi aturan sebagai berikut :

- 1) Harus terdiri atas sederetan karakter Unicode yang diawali oleh karakter huruf atau garis bawah. *Unicode* merupakan sistem pengkodean karakter yang dapat dibaca oleh berbagai bahasa manusia.
- 2) Tidak boleh berupa *keyword* (kata yang dicadangkan), *null*, atau *literal true/false*.
- 3) Harus unik dalam suatu *scope*.

Dalam gaya pemrograman dengan Java, biasanya mengikuti format nama variabel, yaitu:

- 1) Diawali dengan huruf kecil
- 2) Jika nama variabel lebih dari satu kata, kata ke-2, ke-3 dan seterusnya diawali dengan huruf kapital dan ditulis menyatu.

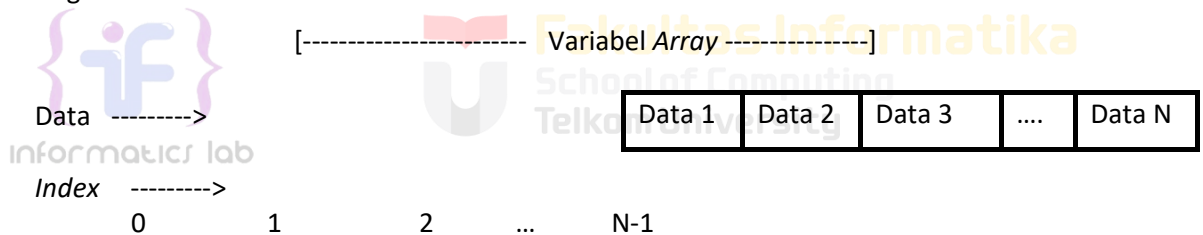
2.2 Konstanta

Variabel dalam Java bisa dijadikan konstanta, sehingga nilainya tidak akan dapat diubah-ubah dengan mendeklarasikannya sebagai variabel *final* seperti contoh :

```
final int x = 2;
```

2.3 Variabel Array

Untuk memudahkan pemahaman mengenai variabel *array*, digunakan ilustrasi kotak yang merepresentasikan setiap elemen variabel *array*. Sebuah variabel *array* bisa digambarkan berupa kotak sebagai berikut :



Index adalah sebuah angka yang menyatakan urutan sebuah elemen pada suatu variabel *array*. Karena seluruh kotak memiliki nama yang sama, maka untuk membedakannya diperlukan suatu cara yaitu dengan memberi nomor urut. Ibarat deretan rumah pada sebuah jalan, untuk membedakan antara rumah yang satu dengan rumah yang lain maka setiap rumah diberi nomor unik yang berbeda antara rumah satu dengan yang lainnya.

2.3.1 Deklarasi Variabel Array

Bentuk umum pendeklarasian variabel *array* di Java adalah :

```
tipeData[] namaVariabel = new tipeData [jumlahElemen];  
// atau  
tipeData namaVariabel[] = new tipeData [jumlahElemen];
```

Tipe data bisa berupa salah satu dari berbagai tipe data seperti *int*, *long*, *double* maupun nama kelas; baik kelas standar Java atau kelas buatan Kita sendiri. Materi kelas tidak akan dibahas dalam modul ini.

Cara pendeklarasian variabel *array* :

- 1) Mendeklarasikan variabel *array* tanpa menyebutkan berapa jumlah elemen yang diperlukan.

```
int[] variabelArray1;
```

Untuk deklarasi variabel *array* dengan cara ini, Kita harus menuliskan di salah satu baris program instruksi untuk memesan jumlah elemen untuk variabel tersebut. Sebelum terjadi pemesanan jumlah elemen, Kita tidak bisa menggunakan variabel *array* ini untuk menyimpan data.

```
variabelArray1 = new int[5];
```

- 2) Mendeklarasikan variabel *array* dengan menyebutkan jumlah elemen yang diperlukan

```
int[] variabelArray2 = new int[5];
```

Pada saat mendeklarasikan variabel *array* ini, maka Kita langsung memesan 5 elemen *array* yang akan Kita gunakan selanjutnya.

- 3) Mendeklarasikan variabel *array* secara otomatis

```
int[] variabelArray3 = {5, 3, 23, 99, 22};  
// atau  
int[] variabelArray3 = new int[]{1,23,45,4,3};
```

Variabel *array* ini memiliki 5 elemen, dan tiap elemen sudah terisi data. Berbeda dengan cara pendeklarasian yang pertama dan yang kedua.

2.3.2 Mengakses Variabel Array

Mengakses suatu variabel berarti mengisi data ke variabel tersebut atau mengambil data dari variabel tersebut. Mengakses variabel *array* berarti Kita mengakses elemen dari variabel *array* tersebut.

```
int tampung = var1[2];
```

Untuk mengisi nilai ke variabel *array*, sebutkan nomor indeks elemen yang akan diisi dengan nilai.

```
var1[5]=65
```

Untuk mengambil nilai dari variabel *array*, sebutkan nomor indeks elemen yang akan Kita ambil isinya.

```
var1[0] = var1[5] + var1[9];
```

Instruksi di atas akan menjumlahkan isi variabel *var1* elemen ke-6 dan ke-10 lalu menyimpan hasilnya ke elemen pertama.

2.3.3 Array 2 Dimensi

Bentuk umum pendeklarasian variabel *array* dua dimensi di Java adalah :

Pendeklarasian *array* dua dimensi bentuk **Rectangular**

```
tipeData[][] namaVariabel = new tipeData[jumlahBaris][jumlahKolom];  
// atau  
tipeData namaVariabel[][] = new tipeData[jumlahBaris][jumlahKolom];
```

Pendeklarasian *array* dua dimensi bentuk **Non-Rectangular**

```
tipeData [ ][ ] namaVariabel = new tipeData [jumlahBaris];  
namaVariabel [ ] = new tipeData[jumlahBaris];
```

Didalam *array* dua dimensi terdapat *array rectangular* dan *non-rectangular*. *Array* dua dimensi secara **Rectangular** artinya ukuran dimensi *array* semuanya sama yakni ukuran indeks pada baris dan kolom sama. Sedangkan **Non-Rectangular** artinya membuat *array* dengan ukuran indeks baris dan kolom yg tidak sama.

Cara pendeklarasian variabel *array* dua dimensi sama dengan cara pendeklarasian variabel *array* satu dimensi. Mengakses variabel *array* dua dimensi, mengisi variabel *array* dua dimensi, mengambil variabel *array* dua dimensi juga sama dengan cara yang berlaku pada variabel *array* satu dimensi, yaitu dengan menyebutkan nomor indeks dari elemen. Perbedaanya hanya pada dua dimensi harus menyebutkan indeks baris dan indeks kolom.

Menghitung jumlah elemen pada *array* dua dimensi.

```
long[][] gede = new long[5][5]; /*deklarasi variabel array dua dimensi*/
gede.length;                  // akan melaporkan jumlah baris, sedangkan
gede[i].length;               // akan melaporkan jumlah kolom pada baris ke-i
```

Contoh array 2 dimensi *rectangular* :

```
public class Array2
{
    public static void main(String[] args)
    {
        double m[][];
        m = new double[4][4];
        m[0][0] = 1;
        m[1][1] = 1;
        m[2][2] = 1;
        m[3][3] = 1;
        System.out.println(m[0][0]+" "+m[0][1]+" "+m[0][2]+" "+m[0][3]);
        System.out.println(m[1][0]+" "+m[1][1]+" "+m[1][2]+" "+m[1][3]);
        System.out.println(m[2][0]+" "+m[2][1]+" "+m[2][2]+" "+m[2][3]);
        System.out.println(m[3][0]+" "+m[3][1]+" "+m[3][2]+" "+m[3][3]);
    }
}
```

Contoh array 2 dimensi *non-rectangular*:

```
public class Array2
{
    public static void main(String[] args)
    {
        int twoDim [][] = new int[2][];
        twoDim[0] = new int[2];
        twoDim[1] = new int[3];

        twoDim[0][0] = 1 ;
        twoDim[0][1] = 4 ;
        twoDim[1][0] = 1 ;
        twoDim[1][1] = 4 ;
        twoDim[1][2] = 4 ;
        System.out.println(twoDim[0][0]);
        System.out.println(twoDim[0][1]);
        System.out.println(twoDim[1][0]);
        System.out.println(twoDim[1][1]);
        System.out.println(twoDim[1][2]);
    }
}
```

2.3.4 Array Bertipe Objek

Array juga bisa digunakan untuk menyimpan beberapa objek yang berasal dari kelas yang sama, sebagai contoh objek manusia memiliki *method* info sehingga dengan menggunakan *array* bertipe objek, objek manusia dapat dibentuk menjadi *array*. Berikut contoh program *array* bertipe objek.

```
//file : Manusia.java
public class Manusia {

    private String nama;
```

```

private int tinggi;

void setInfo(String nama, int tinggi){
    this.nama = nama;
    this.tinggi = tinggi;
}

void info(){
    System.out.println(this.nama+ " Memiliki Tinggi " + this.tinggi +"cm");
}
}

```

```

//file : Main.java
public class Main {
    public static void main(String[] args){

        manusia [] manusia = new manusia[4];
        for (int i = 0; i < manusia.length; i++) {
            manusia[i] = new manusia();
        }
        manusia[0].setInfo("Hermawan", 180);
        manusia[1].setInfo("Suciati", 160);
        manusia[2].setInfo("Boy", 170);
        manusia[3].setInfo("Neneng", 165);

        manusia[0].info();
        manusia[1].info();
        manusia[2].info();
        manusia[3].info();
    }
}

```

Output program:

```

Hermawan Memiliki Tinggi 180cm
Suciati Memiliki Tinggi 160cm
Boy Memiliki Tinggi 170cm
Neneng Memiliki Tinggi 165cm

```

2.4 Operator, Pernyataan Kondisional, dan Perulangan

2.4.1 Operator

Operator-operator yang digunakan pada java ada beberapa jenis diantaranya yaitu:

- 1) Operator Aritmatika
- 2) Operator Relasional
- 3) Operator Kondisional
- 4) Operator *Shift* dan *Bitwise*
- 5) Operator *Assignment*

Operator Aritmatika

Operator ini digunakan pada operasi-operasi **aritmatika** seperti penjumlahan, pengurangan, pembagian dll. Operator-operator yang digunakan yaitu :

Operator	Contoh	Keterangan
+	a + b	Menambahkan a dengan b
-	a – b	Mengurangkan a dengan b

*	a * b	Mengalikan a dengan b
/	a / b	Membagi a dengan b
%	a % b	Menghasilkan sisa pembagian antara a dengan b
++	a ++	Menaikkan nilai a sebesar 1
--	a --	Menurunkan nilai a sebesar 1
-	- a	Negasi dari a.

Operator Relasional

Untuk membandingkan 2 nilai (variabel) atau lebih digunakan operator Relasional, dimana operator ini akan mengembalikan atau menghasilkan nilai **True** atau **False**.

Jenis-jenisnya yaitu:

Operator	Contoh	Keterangan
>	a > b	True jika a lebih besar dari b
<	a < b	True jika a lebih kecil dari b
>=	a >= b	True jika a lebih besar atau sama dengan b
<=	a <= b	True jika a lebih kecil atau sama dengan b
==	a == b	True jika a sama dengan b
!=	a != b	True jika a tidak sama dengan b

Operator Kondisional

Operator ini menghasilkan nilai yang sama dengan operator relasional, hanya saja penggunaannya lebih pada **operasi-operasi boolean**. Jenisnya yaitu:

Operator	Contoh	Keterangan
&&	a && b	True jika a dan b, keduanya bernilai True
	a b	True jika a atau b bernilai True
!	! a	True jika a bernilai False

Operator Shift dan Bitwise

Dua operator ini digunakan untuk **memanipulasi nilai** dari bitnya, sehingga diperoleh nilai yang lain.

Operator Shift		
Operator	Penggunaan	Deskripsi
>>	a >> b	Menggeser bit a ke kanan sejauh b
<<	a << b	Menggeser bit a ke kiri sejauh b
>>>	a >>> b	Menggeser bit a ke kanan sejauh b
Operator Bitwise		
Operator	Penggunaan	Deskripsi
&	a & b	Bitwise AND
	a b	Bitwise OR
^	a ^ b	Bitwise XOR
~	~a	Bitwise Complement

Operator Assignment

Operator assignment dalam Java digunakan untuk **memberikan sebuah nilai ke sebuah variabel**. Operator assignment hanya berupa '=', namun selain itu dalam Java dikenal beberapa *shortcut assignment operator* yang penting, yang digambarkan dalam table berikut :

Operator	Contoh	Ekivalen dengan
+=	a += b	a = a + b
-=	a -= b	a = a - b
*=	a *= b	a = a * b
/=	a /= b	a = a / b

%=	a %= b	a = a % b
&=	a &= b	a = a & b
=	a = b	a = a b
^=	a ^= b	a = a ^ b
<<=	a <<= b	a = a << b
>>=	a >>= b	a = a >> b
>>>=	a >>>= b	a = a >>> b

2.4.2 Pernyataan Kondisional

Statement *if*

Statement *if* memungkinkan sebuah program untuk dapat memilih beberapa operasi untuk dieksekusi, berdasarkan beberapa pilihan. Dalam bentuknya yang paling sederhana, bentuk *if* mengandung sebuah pernyataan tunggal yang dieksekusi jika ekspresi bersyarat adalah benar.

Bentuk *if* yang paling sederhana yaitu :

```
if ( ekspresi_kondisional ) {
    statement 1;
    statement 2;
    .....
}
```

Statement *if-else*

Untuk melakukan beberapa operasi yang berbeda jika salah satu **ekspresi_kondisional** bernilai salah, maka digunakan statement *else*. Bentuk *if-else* memungkinkan kode Java memungkinkan dua alternatif operasi pemrosesan : satu **jika statement bersyarat adalah benar dan satu jika salah**.

- 1) Bentuk statement *if-else* dengan 2 pilihan operasi pemrosesan.

```
if ( ekspresi_kondisional ){
    statement 1;
    statement 2;
    .....
}else{
    statement 1;
    statement 2;
    .....
}
```

- 2) Bentuk statement *if-else* dengan beberapa pilihan operasi pemrosesan.

```
if ( ekspresi_kondisional_A ){
    statement 1;
    statement 2;
    .....
}else if( ekspresi_kondisional_B ){
    statement 1;
    statement 2;
    .....
}else{
    statement 1;
    statement 2;
    .....
}
```

Contoh :

```
class IfElse {
    public static void main(String args[]) {
        int month = 4;
        String season;
        if (month == 12 || month == 1 || month == 2) {
            season = "Dingin";
        } else if (month == 3 || month == 4 || month == 5) {
            season = "Semi";
        } else if (month == 6 || month == 7 || month == 8) {
            season = "Panas";
        } else if (month == 9 || month == 10 || month == 11) {
            season = "Gugur";
        } else {
            season = "";
        }
        System.out.println("Bulan April masuk musim " + season + ".");
    }
}
```

Statement Switch

Bentuk umum pernyataan *switch* adalah sebagai berikut :

```
switch ( expression ) {
    case value_1 :
        statement 1;
        statement 2;
        . . .
        break;
    case value_2 :
        statement 1;
        statement 2;
        . . .
        break;
[default:]
        statement 1;
        statement 2;
        . . .
        break;}
}
```



Fakultas Informatika
School of Computing
Telkom University

Contoh :

```
int hari = 7;
String hariString;
switch (hari) {
    case 1: hariString = "Senin";
        break;
    case 2: hariString = "Selasa";
        break;
    case 3: hariString = "Rabu";
        break;
    case 4: hariString = "Kamis";
        break;
    case 5: hariString = "Jumat";
        break;
    case 6: hariString = "Sabtu";
        break;
    case 7: hariString = "Minggu";
        break;
    default: hariString = "Invalid month";
        break;
}
System.out.println(hariString);
```


Kata-kunci *case* menandai posisi kode dimana eksekusi diarahkan. Sebuah konstanta integer atau konstanta karakter/string mengikutinya, atau ekspresi yang mengevaluasi konstanta integer atau konstanta karakter/string. Sedangkan kata default ekuivalensi dengan kata *else* pada statement *if*.

Ekspresi bersyarat

Kita menggunakan sebuah ekspresi bersyarat untuk menggantikan sebuah bentuk *if-else*. Sintaks adalah sebagai berikut :

```
exp1 ? exp2 : exp3
```

Dalam hal ini, jika *exp1* adalah benar, *exp2* dieksekusi. Jika *exp1* adalah salah, *exp3* dieksekusi.

Contoh:

```
int a = 4;
int b = 5;
int nilai = 3 > 2 ? a : b;
```

Outputnya :

```
nilai = 4
```

2.4.3 Perulangan

1) While

```
while( expression ){
    statement 1;
    statement 2;
    .....
}
```

Selama **ekspresi benar** *while* akan **dieksekusi**.

2) Do

```
do{
    statement 1;
    statement 2;
    .....
}while( expression );
```

Hasil dari *while* akan **dikembalikan** kepada **do**. *Syntax do-while loop* :

3) For

```
for ( initialization ; expression ; step ){
    statement 1;
    statement 2;
    ...
}
```

Pada java terdapat 2 statement yang biasanya digunakan pada setiap bentuk iterasi diatas. Statement tersebut yaitu :

- break*, dapat menghentikan perulangan walaupun kondisi untuk berhenti belum terpenuhi.
- continue*, dengan *statement* ini Kita bisa melewati operasi yang dilakukan dalam iterasi sesuai dengan kondisi tertentu.

Contoh Perulangan: Menampilkan angka 1-10

```
public class angka {
public static void main(String[] args) {
    int i;
    for(i=1;i<=10;i++){
        System.out.println(Integer.toString(i));
    }
    i=1;
    while(i<=10) {
        System.out.println(Integer.toString(i));
        i++;
    }
    i=1;
    do {
        System.out.println(Integer.toString(i));
        i++;
    }
    while(i<=10)
}
}
```

2.5 Compile, Run, dan Jar File

Perlu diketahui bahwa Netbeans merupakan IDE Java yang sudah *user friendly*, maka dari itu sebelum mengenal Netbeans perlu mengetahui fundamental dari *Compile*, *Run*, dan *Jar File*. Pada sub bab 1.2.2 program *HelloWorld* dapat dijalankan dengan satu kali klik tombol *Run*, namun pada dasarnya agar suatu program Java dalam dijalankan harus melewati tahap *compile* lalu *run* dan untuk mempermudah *running* program maka dibuatlah *Jar File* dari program yang dibuat.

2.5.1 Compile

Compile pada java berarti mengubah *source code* ke *byte codes* agar dapat dipahami oleh Java VM. Jika proses *compile* berhasil, maka akan terdapat sebuah berkas file berekstensi *class*. Java tidak membuat *file executeable (.exe)* melainkan *file .class* tersebut. Jika tidak berhasil, lakukan perbaikan *source code*, lalu coba *compile* kembali

Pada contoh program *HelloWorld*, jalankan dengan perintah sebagai berikut

```
➔ javac HelloWorld
```

2.5.2 Run

Setelah program berhasil di-*Compile* ke dalam *byte code* Java, program dapat dialankan pada Java VM. Untuk menjalankan program Java, jalankan perintah Java namafile dengan namafile adalah *file .class* yang akan di-*Run* (tanpa ekstensi *.class*).

Pada contoh program *HelloWorld*, jalankan dengan perintah sebagai berikut

```
➔ java HelloWorld
```

2.5.3 Jar File

Java Archive (JAR) merupakan *file archive* yang menyimpan *source* program Java agar dapat dijalankan secara *portable* dengan langsung membuka *file Jar* secara langsung, namun pada beberapa kasus *Output* yang hanya ditampilkan di konsol, *Jar File* perlu dijalankan di CMD/Terminal. Pada Netbeans pembuatan *Jar File* cukup dengan melakukan *Build Project* atau menekan tombol F11 dan hasilnya dapat di lihat di direktori penyimpanan *source project*, namun dengan menggunakan CMD/Terminal ada beberapa langkah yang perlu dilakukan.

Pada contoh program *HelloWorld*, untuk membuat *Jar File*, perlu dilakukan *Compile* terhadap *file* Java yang dibuat, selanjutnya perlu dilakukan pembuatan *manifest* untuk menentukan *main class* dengan perintah `echo Main-Class: namamainclass >manifest.txt`. Pada contoh program *HelloWorld*, pastikan directory di CMD/Terminal sudah berpindah ke directory tempat *source* program Java lalu jalankan dengan perintah pembuatan *manifest* sebagai berikut

```
➔ echo Main-Class: Main >manifest.txt
```

Setelah *manifest* dibuat, maka *Jar File* dapat dibuat dengan perintah `jar cvfm namafilejar.jar manifest.txt *.class`. Pada contoh program *HelloWorld*, jalankan perintah pembuatan *jar* dengan perintah sebagai berikut

```
➔ jar cvfm HelloWorld.jar manifest.txt *.class
```

Lalu jalankan *file Jar* menggunakan perintah sebagai berikut

```
➔ java -jar HelloWorld.jar
```

Maka *Output* yang ditampilkan akan sama dengan *Run* pada program Netbeans, yaitu tulisan "Hello World".



Modul 3 Kelas dan Objek

Tujuan Praktikum

1. Mengerti konsep dasar *Object Oriented Programming*.
2. Dapat membandingkan antara pemrograman prosedural dengan pemrograman berorientasi *Object*.
3. Memahami Kelas, *Object*, *Method* dan *Constructor*, serta mengimplementasikannya dalam suatu program Java.

3.1 Pendahuluan

Sebelum membahas tentang kelas terlebih dahulu Kita perlu mengetahui konsep pemrograman berorientasi objek. Suatu sistem yang dibangun dengan metode berorientasi objek adalah sebuah sistem yang komponennya di-enkapsulasi menjadi kelompok data dan fungsi, yang dapat mewarisi atribut dan sifat dari komponen lainnya dan komponen-komponen tersebut saling berinteraksi satu sama lain.

Beberapa karakteristik utama dalam pemrograman berorientasi objek yaitu:

3.1.1 Abstraksi

Abstraksi pada dasarnya adalah **menemukan hal-hal yang esensial pada suatu objek** dan mengabaikan hal-hal yang sifatnya insidental. Kita menentukan apa ciri-ciri (atribut) yang dimiliki oleh objek tersebut serta apa saja yang bisa dilakukan (metode/perilaku) oleh objek tersebut.

Contohnya adalah abstraksi pada manusia.

Ciri-ciri (atribut) : punya tangan, berat, tinggi, dll. (kata benda, atau kata sifat).

Fungsi (perilaku) : makan, minum, berjalan, dll. (kata kerja).

3.1.2 Enkapsulasi

Pengkapsulan adalah **proses pemaketan data objek bersama *method-methodnya***. Manfaat utama pengkapsulan adalah penyembunyian rincian-rincian implementasi dari pemakai/objek lain yang tidak berhak. Enkapsulasi memproteksi suatu proses dari kemungkinan interferensi atau penyalahgunaan dari luar sistem.

Fungsi dari enkapsulasi adalah:

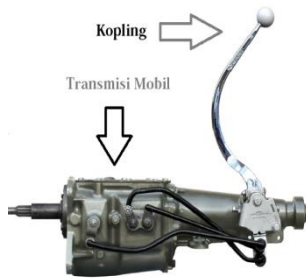
1) Penyembunyian data

Penyembunyian data (*data hiding*) mengacu perlindungan data internal objek. Objek tersebut disusun dari antarmuka *public Method* dan *private* data. Manfaat utama adalah bagian internal dapat berubah tanpa mempengaruhi bagian-bagian program yang lain.

2) Modularitas

Modularitas (*modularity*) berarti objek dapat dikelola secara **independen**. Karena kode sumber bagian internal objek dikelola secara terpisah dari antarmuka, maka Kita bebas melakukan modifikasi yang tidak menyebabkan masalah pada bagian-bagian lain. Manfaat ini mempermudah mendistribusikan objek-objek di sistem.

Contoh ilustrasi pada mobil :



Mobil memiliki sistem transmisi dan sistem ini menyembunyikan proses yang terjadi di dalamnya tentang bagaimana cara mobil tersebut bekerja, mulai dari bagaimana cara mobil mengatur percepatan dan apa yang dilakukan terhadap mesin untuk mendapat percepatan tersebut.

Kita sebagai pengguna hanya cukup memindah-mindahkan tongkat transmisi (kopling) untuk mendapatkan percepatan yang diinginkan.

Tongkat transmisi adalah satu-satunya *interface* yang digunakan dalam mengatur sistem transmisi mobil tersebut. Kita tidak dapat menggunakan pedal rem untuk mengakses sistem transmisi tersebut dan sebaliknya dengan mengubah-ubah sistem transmisi Kita tidak akan bisa menghidupkan radio mobil atau membuka pintu mobil. Konsep yang sama dapat pula Kita terapkan dalam pemrograman orientasi objek.

3.1.3 Pewarisan (Inheritance)

Pewarisan adalah **proses penciptaan kelas baru** (yang disebut subclass atau kelas turunan) **dengan mewarisi karakteristik** dari kelas yang telah ada (*superclass* atau kelas induk), ditambah karakteristik unik kelas baru itu. Karakteristik unik tersebut bisa merupakan perluasan atau spesialisasi dari *superclass*. Kelas turunan akan mewarisi anggota-anggota suatu kelas yang berupa data (atribut) dan fungsi (operasi) dan pada kelas turunan memungkinkan menambahkan data serta fungsi yang baru. Secara praktis berarti bahwa jika *superclass* telah mendefinisikan perilaku yang Kita perlukan, maka Kita tidak perlu mendefinisikan ulang perilaku itu, Kita cukup membuat kelas yang merupakan *subclass* dari *superclass* yang dimaksud.

3.1.4 Polymorphisms

Polymorphism berasal dari bahasa Yunani yang berarti **banyak bentuk**. Konsep ini memungkinkan digunakannya **antarmuka (*interface*) yang sama untuk memerintah suatu objek agar dapat melakukan aksi** atau tindakan yang mungkin secara prinsip sama tapi secara proses berbeda. Seringkali *Polymorphism* disebut dengan “satu interface banyak aksi”. Mekanisme *Polymorphism* dapat dilakukan dengan dengan beberapa cara, seperti *Overloading Method*, *Overloading Constructor*, maupun *Overriding Method*. Semua akan dibahas pada bab selanjutnya.

	Contoh ilustrasi pada mobil. Mobil yang ada di pasaran terdiri atas berbagai tipe dan merek, tetapi semuanya memiliki <i>interface</i> kemudi yang hampir sama seperti setir, tongkat transmisi, pedal gas, dll. Jika Kita dapat mengemudikan satu jenis mobil saja dari satu merek tertentu, maka boleh dikatakan Kita dapat mengemudikan hampir semua jenis mobil yang ada karena semua mobil tersebut menggunakan <i>interface</i> yang sama. Misal, ketika Kita menekan pedal gas pada mobil A, Kita telah berhasil menggerakkan mobil A tersebut dengan percepatan yang tinggi. Sebaliknya, ketika Kita menggunakan mobil B dan sama-sama menekan pedal gas, mobil B ini bergerak agak lambat. Jadi, dapat disimpulkan dengan sama-sama Kita menekan pedal gas pada dua mobil ini akan didapatkan hasil yang berbeda pada keduanya.

3.1.5 Message (Komunikasi Antar Objek)

Objek yang bertindak sendirian jarang berguna, kebanyakan objek memerlukan objek-objek lain untuk melakukan banyak hal. Oleh karena itu, objek-objek tersebut saling berkomunikasi dan berinteraksi lewat message. Ketika berkomunikasi, suatu objek mengirim pesan (dapat berupa pemanggilan *method*) untuk memberitahu agar objek lain melakukan sesuatu yang diharapkan. Seringkali pengiriman *message* juga disertai informasi untuk memperjelas apa yang dikehendaki. Informasi yang dilewatkan beserta *message* adalah parameter message.

3.2 Kelas

Kelas adalah cetak biru (rancangan) dari objek. Ini berarti Kita bisa membuat banyak objek dari satu macam kelas. Kelas mendefinisikan sebuah tipe dari objek. Di dalam kelas Kita dapat mendeklarasikan variabel dan menciptakan *Object* (instansiasi). Sebuah kelas mempunyai anggota(member) yang terdiri atas atribut dan *method*. Atribut adalah semua *field* identitas yang Kita berikan pada suatu kelas, misal kelas manusia memiliki *field* atribut berupa nama, maupun umur. *Method* dapat Kita artikan sebagai semua fungsi ataupun prosedur yang merupakan perilaku (*behaviour*) dari suatu kelas. Dikatakan fungsi bila *method* tersebut melakukan suatu proses dan mengembalikan suatu nilai (*return value*), dan dikatakan prosedur bila *method* tersebut hanya melakukan suatu proses dan tidak mengembalikan nilai (*void*).

Contoh pendeklarasian kelas:

```
class <classname>
{
    //declaration of data member
    //declaration of Methods
}
```

Kata `class` merupakan *keyword* pada Java yang digunakan untuk mendeklarasikan kelas dan `<classname>` digunakan untuk memberikan nama kelas. Sebagai contoh, dibawah ini terdapat pendeklarasian kelas Mahasiswa yang mempunyai *field* nama, nim dan *method* `getNamaMahasiswa()`.

```
class Mahasiswa
{
    String nama;           //data anggota
    int nim;               //data anggota
    void getNamaMahasiswa() {}; //Method
}
```

Setelah mengetahui apa itu kelas , selanjutnya Kita perlu mengetahui *Object* dan *Java Modifier*.

3.3 Object

Object (objek) secara lugas dapat **diartikan sebagai insatansiasi atau hasil ciptaan dari suatu kelas** yang telah dibuat sebelumnya. Dalam pengembangan program orientasi objek lebih lanjut, sebuah objek dapat dimungkinkan terdiri atas objek-objek lain. Seperti halnya objek mobil terdiri atas mesin, ban, kerangka mobil, pintu, karoseri dan lain-lain. Atau, bisa jadi sebuah objek merupakan turunan dari objek lain sehingga mewarisi sifat-sifat induknya. Misal motor dan mobil merupakan kendaraan bermotor, sehingga motor dan mobil mempunyai sifat-sifat yang dimiliki oleh kelas kendaraan bermotor dengan spesifikasi sifat-sifat tambahan sendiri.

Contoh *Object* yang lain: Komputer, TV, mahasiswa, ponsel, lbuku, dan lainnya.

Contoh pembuatan Objek dalam java:

```
Manusia objMns1 = new Manusia(); //membuat objek manusia
```

3.4 Field

Field adalah sebuah atribut. *Field* bisa berupa sebuah variabel kelas, variabel objek, variabel *Method* objek atau parameter dari sebuah fungsi. *Field* merupakan anggota dari kelas yang digunakan untuk menyimpan data. Terdapat dua *field* dalam kelas, dibawah ini adalah contoh kelas yang terdapat 2 jenis *field* tersebut.

```
public class Circle {
    public static final double PI= 3.14159;
    public static double radiansToDegrees(double rads) {
        return rads * 180 / PI;
    }
    public double r;
    public double area() {
        return PI * r * r;
    }
    public double circumference() {
        return 2 * PI * r;
    }
}
```

1) Class Field

Field yang dikaitkan dengan kelas dimana dia didefinisikan, bukan dengan sebuah *instance* dari kelas. Di bawah ini menyatakan pendeklarasian kelas *field* pada kelas `Circle` adalah:

```
public static final double PI= 3.14159;
```

Static *modifier* diatas menyatakan bahwa *field* tersebut merupakan *class field*. *Field* ini berkaitan dengan kelas itu sendiri tidak dengan *instance* dari kelas. Berdasarkan kelas `Circle` maka jika terdapat *Method* dari kelas `Circle` maka mengakses variabel `PI` dengan `Circle.PI`

2) Instance Field

Field apapun yang dideklarasikan tanpa menggunakan *static modifier*. Contoh *instance field* yang terdapat pada kelas `Circle` adalah:

```
public double r;
```

Field ini merupakan *field* yang terkait dengan *instance* kelas bukan dengan kelas itu sendiri. Berdasarkan kelas `Circle`, setiap objek `Circle` yang dibuat memiliki salinan sendiri *field* `r`. Contohnya `r` menyatakan jari-jari lingkaran. Jadi setiap objek `Circle` dapat memiliki jari-jari sendiri dari semua objek lingkaran lainnya.

3.5 Method

Method dikenal juga sebagai suatu *function* dan *procedure*. Dalam OOP, *Method* digunakan untuk memodularisasi program melalui pemisahan tugas dalam suatu kelas. Pemanggilan *Method* menspesifikasikan nama *Method* dan menyediakan informasi (parameter) yang diperlukan untuk melaksanakan tugasnya.

Deklarasi *Method* untuk yang mengembalikan nilai (fungsi)

```
[modifier] Type-data namaMethod(parameter1,parameter2,...parameterN)
{
    Deklarasi-deklarasi dan proses ;
    return nilai-kembalian
}
```

Type-data merupakan tipe data dari nilai yang dapat dikembalikan oleh *Method* ini.

Deklarasi *Method* untuk yang tidak mengembalikan nilai (prosedur).

```
[modifier] void namaMethod(parameter1,parameter2,...parameterN)
{
    Deklarasi-deklarasi dan proses ;
}
```

Contoh implementasi *Method*:

```
public int jumlahAngka(int x, int y)
{
    int z=x+y;
    return z;
}
```

Ada dua cara melewati argumen ke *Method*, yaitu:

1) Melewatkan secara Nilai (*Pass by Value*)

Digunakan untuk argumen yang mempunyai tipe data primitif (Byte, short, int, long, float, double, char, dan boolean). Prosesnya adalah *compiler* hanya menyalin isi memori (pengalokasian suatu variabel), dan kemudian menyampaikan salinan tersebut kepada *Method*. Isi *memory* ini merupakan data “sesungguhnya” yang akan dioperasikan.

Karena hanya berupa salinan isi *memory*, maka perubahan yang terjadi pada variabel akibat proses di dalam *method* tidak akan berpengaruh pada nilai variabel asalnya.

2) Melewatkan secara Referensi (*Pass by Reference*)

Digunakan pada *array* dan objek. Prosesnya isi *memory* pada variabel *array* dan objek merupakan penunjuk ke alamat *memory* yang mengandung data sesungguhnya yang akan dioperasikan. Dengan kata lain, variabel *array* atau objek menyimpan alamat *memory* bukan isi *memory*. Akibatnya, setiap perubahan variabel di dalam *method* akan mempengaruhi nilai pada variabel asalnya.

Contoh program:

```
//file TestPass.java
class TestPass {
    int i,j;

    TestPass(int a,int b) {
        i = a;
        j = b;
    }

    //passed by value dengan parameter berupa tipe data primitif
    void calculate(int m,int n) {
        m = m*10;
        n = n/2;
    }

    //passed by reference dengan berupa tipe data class
    void calculate(TestPass e) {
        e.i = e.i*10;
        e.j = e.j/2;
    }
}
```

```
// file PassedByValue.java
class PassedByValue {
    public static void main(String[] args) {
        int x,y;
        TestPass z;
        z = new TestPass(50,100);
        x = 10;
        y = 20;
    }
}
```



```

        System.out.println("Nilai sebelum passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);

        //passed by value
        z.calculate(x,y);
        System.out.println("Nilai sesudah passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);
        System.out.println("Nilai sebelum passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);

        //passed by reference
        z.calculate(z);

        System.out.println("Nilai sesudah passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);
    }
}

```

Output dari program diatas

```

Nilai sebelum passed by value :
x = 10
y = 20
Nilai sesudah passed by value :
x = 10
y = 20
Nilai sebelum passed by reference :
z.i = 50
z.j = 100
Nilai sesudah passed by reference :
z.i = 500
z.j = 50

```

Keterangan: 

Pada saat pemanggilan *method* `calculate()` dengan metode *pass by value*, hanya nilai dari variabel *x* dan *y* saja yang dilewatkan ke variabel *m* dan *n*, sehingga perubahan pada variabel *m* dan *n* tidak akan mengubah nilai dari variabel *x* dan *y*. Sedangkan pada saat pemanggilan *method* `calculate()` dengan metode *pass by reference* yang menerima parameter bertipe kelas `Test`. Pada waktu Kita memanggil *method* `calculate()`, nilai dari variabel *z* yang berupa referensi ke objek sesungguhnya dilewatkan ke variabel *a*, sehingga variabel *a* menunjukkan ke objek yang sama dengan yang ditunjuk oleh variabel *z* dan setiap perubahan pada objek tersebut dengan menggunakan variabel *a* akan terlihat efeknya pada variabel *z* yang terdapat pada kode yang memanggil *method* tersebut.

3.6 Keyword “this”

Dalam Java terdapat suatu besaran referensi khusus, disebut *this*, yang digunakan dalam *method* yang dirujuk untuk objek yang sedang berlaku. Nilai *this* merujuk pada objek di mana *method* yang sedang berjalan dipanggil.

Contoh implementasi program:

```

class Lagu {
    private String pencipta;
    private String judul;
    public void IsiParam(String judul,String pencipta) {
        this.judul = judul;
        this.pencipta = pencipta;
    }
}

```

```

        public void cetakKeLayar() {
            if(judul==null&& pencipta==null) return;
            System.out.println("Judul : " + judul +", pencipta : " + pencipta);
        }
    }
    class DemoLagu {
    public static void main(String[] args) {
        Lagu a = new Lagu();
        a.IsiParam("God Will Make A Way","Don Moen ");
        a.cetakKeLayar();
    }
}

```

Keluaran dari program:

```
Judul : God Will Make A Way, pencipta : Don Moen
```

Keterangan :

Perhatikan pada *method* `IsiParam()` di atas. Di sana Kita mendeklarasikan nama variabel yang menjadi parameternya sama dengan nama variabel yang merupakan *property* dari kelas Lagu (judul dan pencipta). Dalam hal ini, Kita perlu menggunakan *keyword* `this` (*keyword* ini merefer ke objek itu sendiri) agar dapat mengakses *property* judul dan pencipta di dalam *method* `IsiParam()` tersebut. Apa yang terjadi jika pada *method* `IsiParam()` *keyword* `this` dihilangkan ?

Keyword `this` juga dapat digunakan untuk memanggil suatu konstruktor dari konstruktor lainnya.

Contoh Program:

```

public class Buku {
    private String pengarang;
    private String judul;
    private Buku() {
        this("Rumah Kita", "GoodBles");           // this ini digunakan untuk
                                                    // memanggil konstruktor yang
                                                    // menerima dua parameter
    }
    private Buku(String judul,String pengarang)
    {
        this.judul = judul;
        this.pengarang = pengarang;
    }
    private void cetakKeLayar()
    {
        System.out.println("Judul : " + judul + " Pengarang : " + pengarang);
    }
    public static void main(String[] args)
    {
        Buku a,b ;
        a = new Buku("Jurassic Park", "Michael Chricton");
        b = new Buku();
        a.cetakKeLayar();
        b.cetakKeLayar();
    }
}

```

Keluaran dari program diatas:

```
Judul : Jurassic Park Pengarang : Michael Chricton
Judul : Rumah Kita Pengarang : GoodBles
```

Pada contoh program di atas, Kita juga dapat mengambil pelajaran bahwa penggunaan *modifier private* pada *constructor* mengakibatkan *constructor* tersebut hanya dapat dikenali dalam satu kelas saja, sehingga pembuatan objek hanya dapat dilakukan dalam lokal kelas tersebut.

Catatan penting lainnya : bahwa *method* `void main(String[] args)` haruslah bersifat **public static**.



Modul 4 Encapsulation

Tujuan Praktikum

1. Memahami *Modifier*, *Constructor*, dan *Package* serta mengimplementasikannya dalam suatu program Java.

4.1 Java Modifier

Modifier memberi dampak tertentu pada kelas, *interface*, *method*, dan variabel. **Java Modifier** terbagi menjadi kelompok berikut:

- 1) **Access modifier** berlaku untuk kelas, *method* dan variabel, meliputi *modifier public*, *protected*, *private*, dan tak ada modifier.
- 2) **Final modifier** berlaku kelas, *method* dan variabel, meliputi *modifier final*.
- 3) **Static modifier** berlaku untuk variabel dan *method*, meliputi *modifier static*.
- 4) **Abstract modifier** berlaku untuk kelas dan *method*, meliputi *modifier abstract*.
- 5) **Synchronized modifier** berlaku untuk *method*, meliputi *modifier synchronized*.
- 6) **Native modifier** berlaku untuk *method*, meliputi *modifier native*.
- 7) **Storage modifier** berlaku untuk variabel, meliputi *transient* dan *volatile*.

Berikut ini *modifier-modifier* yang terdapat pada java :

Modifier	Kelas dan Interface	Method dan Variabel
(tak ada <i>modifier</i>)	Tampak di Paketnya	Diwarisi oleh subclassnya di paket yang sama dengan kelasnya. Dapat diakses oleh <i>method-method</i> di kelas-kelas yang sepaket.
Public	Tampak di manapun	Diwarisi oleh semua subclassnya. Dapat diakses dimanapun.
Protected	Tidak dapat diterapkan	Diwarisi oleh semua subclassnya. Dapat diakses oleh <i>Method-Method</i> di kelas-kelas yang sepaket.
Private	Tidak dapat diterapkan	Tidak diwarisi oleh subclassnya Tidak dapat diakses oleh kelas lain.

Permitted Modifier :

Modifier	Kelas	Interface	Method	Variabel
<i>abstract</i>	Kelas dapat berisi <i>method abstract</i> . Kelas tidak dapat diinstantiasi Tidak mempunyai <i>constructor</i>	Optional untuk diberikan di <i>interface</i> karena <i>interface</i> secara inheren adalah <i>abstract</i> .	Tidak ada badan <i>method</i> yang didefinisikan. <i>Method</i> memerlukan kelas kongkret yang merupakan subclass yang akan mengimplementasikan <i>Method abstract</i>	Tidak dapat diterapkan.
<i>final</i>	Kelas menjadi tidak dapat digunakan untuk menurunkan kelas yang baru.	Tidak dapat diterapkan.	<i>Method</i> tidak dapat ditimpa oleh <i>Method</i> di subclass-subclassnya	Berperilaku sebagai konstanta
<i>static</i>	Tidak dapat diterapkan.	Tidak dapat diterapkan.	Mendefinisikan <i>method</i> (milik) kelas. Dengan	Mendefinisikan variabel milik kelas. Tidak

Modifier	Kelas	Interface	Method	Variabel
			demikian tidak memerlukan instant <i>Object</i> untuk menjalankannya. <i>Method</i> ini tidak dapat menjalankan <i>method</i> yang bukan <i>static</i> serta tidak dapat mengacu variabel yang bukan <i>static</i> .	memerlukan instant <i>Object</i> untuk mengacunya. Variabel ini dapat digunakan bersama oleh semua <i>instant</i> objek.
<i>synchronized</i>	Tidak dapat diterapkan.	Tidak dapat diterapkan.	Eksekusi dari <i>method</i> adalah secara mutual <i>exclusive</i> diantara semua <i>thread</i> . Hanya satu <i>thread</i> pada satu saat yang dapat menjalankan <i>method</i>	Tidak dapat diterapkan pada deklarasi. Diterapkan pada instruksi untuk menjaga hanya satu <i>thread</i> yang mengacu variabel pada satu saat.
<i>native</i>	Tidak dapat diterapkan.	Tidak dapat diterapkan.	Tidak ada badan <i>method</i> yang diperlukan karena implementasi dilakukan dengan bahasan lain.	Tidak dapat diterapkan.
<i>transient</i>	Tidak dapat diterapkan.	Tidak dapat diterapkan.	Tidak dapat diterapkan.	Variabel tidak akan diserialisasi
<i>volatile</i>	Tidak dapat diterapkan.	Tidak dapat diterapkan.	Tidak dapat diterapkan.	Variabel diubah secara asinkron. Kompilator tidak pernah melakukan optimasi atasnya.

Deklarasi *modifier* dalam Java:

```
[Modifier] class/interface [nama class/interface] {} /*deklarasi modifier di class/interface*/
[Modifier][TypeData][nama Atribut]; //deklarasi modifier di atribut
[Modifier][TypeData][nama Method](parameter_1,parameter_2,parameter_n) {}
//deklarasi modifier di Method
```

Contoh:

```
public class Manusia {} //contoh modifier di class
private int tinggi; //contoh modifier di atribut
public int getTinggi(){} //contoh modifier di Method
```

Contoh kelas yang mempunyai akses *modifier* ditunjukkan pada diagram kelas Manusia berikut ini.

Manusia
- nama : String - umur : int
+ setName : void + getName : String + setUmur : void + getUmur : int

Keterangan tanda:

- artinya memiliki *access modifier* **private**

+ artinya memiliki *access modifier* **public**

artinya memiliki *access modifier* **protected**

Contoh implementasi di dalam program:

```
public class Manusia {  
    //definisi atribut  
    private String nama;  
    private int umur;  
    //definisi Method  
    public void setName(String a){  
        nama=a;  
    }  
    public String getName(){  
        return nama;  
    }  
    public void setUmur(int a){  
        umur = a;  
    }  
    public int getUmur(){  
        return umur;  
    }  
}
```

Kesepakatan umum penamaan dalam suatu Kelas:

- 1) Nama Kelas – gunakan kata benda dan huruf pertama dari tiap kata ditulis dengan huruf besar dan memiliki access modifier public : Manusia
- 2) Pada umumnya, atribut diberi access modifier private, dan method diberi access modifier public. Hal ini diterapkan untuk mendukung konsep OO yaitu enkapsulasi mengenai data hiding. Jadi, Kita tidak langsung menembak data/atribut pada kelas tersebut, tetapi Kita memberikan suatu antarmuka *method* yang akan mengakses data/atribut yang disembunyikan tersebut.
- 3) Nama atribut - gunakan kata benda, dan diawali dengan huruf kecil : nama, umur
- 4) Nama *Method* – gunakan kata kerja; kecuali huruf yang pertama, huruf awal tiap kata ditulis kapital : `getName()`, `getUmur()`
- 5) Untuk *method* yang akan memberikan atau mengubah nilai dari suatu atribut, nama *method* Kita tambah dengan kata kunci "set": `setUmur(int a)`, `setName(String a)`
- 6) Untuk *Method* yang akan mengambil nilai dari atribut, nama *Method* Kita tambah dengan kata kunci "get" : `getUmur()`, `getName()`
- 7) Konstanta. Semuanya ditulis dengan huruf besar; pemisah antar kata menggunakan garis bawah: `MAX_VALUE`, `DECIMAL_DIGIT_NUMBER`

4.2 Constructor

Constructor adalah tipe khusus method yang **digunakan untuk menginstansiasi atau menciptakan sebuah objek**. Nama *constructor* adalah sama dengan nama kelasnya. Selain itu, *constructor* tidak bisa mengembalikan suatu nilai (*not return value*) bahkan *void* sekalipun. *Default*-nya, bila Kita tidak membuat *constructor* secara eksplisit, maka Java akan menambahkan *constructor default* pada program yang Kita buat secara implisit. *Constructor default* ini tidak memiliki parameter masukan sama sekali. Namun, bila Kita telah mendefinisikan minimal satu buah *constructor*, maka Java tidak akan menambah *constructor default*.

Constructor juga dimanfaatkan untuk membangun suatu objek dengan langsung mengeset atribut-atribut yang disandang pada objek yang dibuat tersebut. Oleh karena itu, *constructor* jenis ini haruslah memiliki parameter masukan yang akan digunakan untuk mengeset nilai atribut.

Access modifier yang dipakai pada *constructor* selayaknya adalah *public* karena *constructor* tersebut akan diakses di luar kelasnya (walaupun Kita juga bisa memberikan *access modifier* pada *constructor* dengan *private* artinya Kita tidak bisa memanggil *constructor* tersebut di luar kelasnya). Cara memanggil *constructor* adalah dengan menambahkan *keyword new*. *Keyword new* dalam deklarasi

ini artinya Kita mengalokasikan pada memory sekian blok *memory* untuk menampung objek yang baru Kita buat.

Deklarasi *constructor*:

```
[modifier] namaclass(parameter1){
    Body constructor;
}
[modifier] namaclass(parameter1,parameter2){
    Body constructor;
}
[modifier] namaclass(parameter1,parameter2,...,parameterN){
    Body constructor;
}
```

Contoh program sebelumnya dengan penambahan *constructor* eksplisit:

```
public class Manusia {
    private String nama;
    private int umur;
    //definisi constructor
    public Manusia(){ } //constructor pertama = default tanpa parameter
    public Manusia(String a){ //constructor kedua
        nama=a;
    }
    public Manusia(String a, int b){ //constructor ketiga
        nama=a;
        umur=b;
    }
    //definisi Method
    public void setName(String a){
        nama=a;
    }
    public String getName(){
        return nama;
    }
    public void setUmur(int a){
        umur=a;
    }
    public int getUmur(){
        return umur;
    }
}

public class DemoManusia {
    public static void main(String[] args) { // program utama
        Manusia arrMns[] = new Manusia[3]; // buat array of Object
        Manusia objMns1 = new Manusia(); // constructor pertama
        objMns1.setName("Markonah");
        objMns1.setUmur(76);
        Manusia objMns2 = new Manusia("Mat Conan"); //constructor kedua
        Manusia objMns3 = new Manusia("Bajuri", 13); //constructor ketiga
        arrMns[0] = objMns1;
        arrMns[1] = objMns2;
        arrMns[2] = objMns3;

        for(int i=0; i<3; i++) {
            System.out.println("Nama : "+arrMns[i].getName());
            System.out.println("Umur : "+arrMns[i].getUmur());
            System.out.println();
        }
    }
}
```

Jika program di atas di-*compile* dan di-*running* maka *output*-nya adalah sebagai berikut.

```
Nama : Markonah
```

```
Umur : 76
Nama : Mat Conan
Umur : 0
Nama : Bajuri
Umur : 13
```

Coba analisis mengapa hasilnya seperti di atas !

Seperti *source* di atas, Kita dapat mempunyai lebih dari satu *constructor*, masing-masing harus mempunyai parameter yang berbeda sebagai penandanya. Program secara otomatis akan memilih *constructor* mana yang akan dijalankan sesuai dengan parameter masukan pada waktu pembuatan objek. Hal seperti ini disebut **Overloading** terhadap *constructor*.

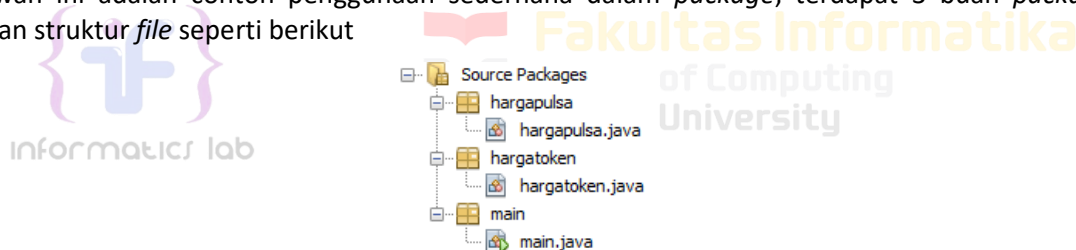
4.3 Package

Package digunakan untuk mengatur *class* yang telah dibuat dan akan bermanfaat jika *class* yang dibuat sangat banyak sehingga perlu dikelompokkan berdasarkan kategori tertentu. Selain pengelompokan *class*, penggunaan *package* dapat mengatasi konflik penamaan *method* jika suatu *method* ditulis dalam *package* yang berbeda.

Ada dua cara menggunakan suatu *package* yaitu :

- 1) *Class* yang menggunakan berada dalam direktori (*package*) yang sama dengan *Class* yang digunakan, Maka tidak diperlukan *import*.
- 2) *Class* yang menggunakan berada dalam direktori (*package*) yang berbeda dengan *Class* yang digunakan, Maka perlu dilakukan *import package*.

Dibawah ini adalah contoh penggunaan sederhana dalam *package*, terdapat 3 buah *package* dengan struktur *file* seperti berikut



Gambar 4-1 Struktur *file* program harga token dan harga pulsa.

Dengan isi tiap *class* sesuai dengan *source code* dibawah.

```
package main;
import hargatoken.hargatoken;
import hargapulsa.hargapulsa;
public class main {
    public static void main(String[] args){
        hargatoken objectToken = new hargatoken();
        objectToken.info();
        hargapulsa objectPulsa = new hargapulsa();
        objectPulsa.info();
    }
}
```

```
package hargatoken;
public class hargatoken {
    public void info(){
        System.out.println("Harga Token");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
        System.out.println("Rp. 200000");
    }
}
```



```

        System.out.println("Rp. 500000");
    }
}

```

```

package hargapulsa;
public class hargapulsa {
    public void info(){
        System.out.println("Harga Pulsa");
        System.out.println("Rp. 5000");
        System.out.println("Rp. 10000");
        System.out.println("Rp. 20000");
        System.out.println("Rp. 50000");
        System.out.println("Rp. 100000");
    }
}

```

Output program:

```

Harga Token
Rp. 20000
Rp. 50000
Rp. 100000
Rp. 200000
Rp. 500000
Harga Pulsa
Rp. 5000
Rp. 10000
Rp. 20000
Rp. 50000
Rp. 100000

```



Jangan pernah menggunakan kata-kata ini sebagai nama variabel, jika terjadi maka *compile* akan *error*

boolean	byte	char	double	float	int	long	short	public	private
protected	abstract	final	native	static	strictfp	synchronized	transient	volatile	if
else	do	while	switch	case	default	for	break	continue	assert
class	extends	implements	import	instanceof	interface	new	package	super	this
catch	finally	try	throw	throws	return	void	const	goto	enum

Modul 5 Relasi Antar Kelas

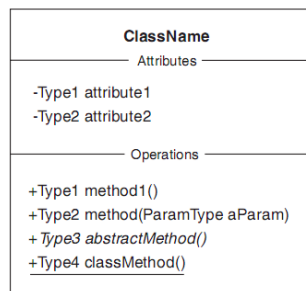
Tujuan Praktikum

2. Mengerti konsep dasar Object oriented programming
3. Memahami penggunaan kelas dalam pemrograman java
4. Memahami hubungan antar kelas baik asosiasi, agregasi, dan komposisi.

5.1 Diagram Kelas

Diagram kelas merupakan sebuah diagram yang **digunakan untuk memodelkan** kelas-kelas yang digunakan di dalam sistem beserta hubungan antar kelas dalam sistem tersebut.

Beberapa elemen penting dalam diagram kelas adalah kelas dan relasi antar kelas. Kelas digambarkan dengan simbol kotak seperti gambar di bawah ini:



Gambar 5-1 Diagram Kelas.

Baris pertama dari simbol diagram kelas menandakan nama dari kelas yang bersangkutan. Baris di bawahnya menyatakan atribut-atribut dari kelas tersebut apa saja, dan baris setelahnya menyatakan *method-method* yang terdapat pada kelas tersebut. Adapun simbol untuk *access modifier* adalah:

Simbol	Keterangan
+	simbol <i>access modifier public</i>
-	simbol <i>access modifier private</i>
#	simbol <i>access modifier protected</i> (untuk simbol ini tidak terdapat pada gambar diatas)

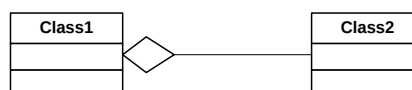
Sedangkan untuk menggambarkan **hubungan antar kelas** digunakan simbol garis antara dua kelas yang berelasi. Simbol garis tersebut antara lain:

- 1) Class1 **berasosiasi** dengan Class2 , digambarkan sebagai berikut:



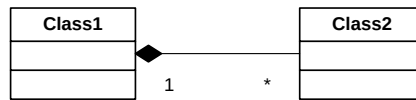
Gambar 5-2 Hubungan Asosiasi.

- 2) Class2 merupakan **elemen part-of** dari Class1 (Class1 berelasi **agregasi** dengan Class2), digambarkan sebagai berikut:



Gambar 5-3 Hubungan Agregasi.

3) Class1 dengan Class2 berelasi **komposisi**, digambarkan sebagai berikut:



Gambar 5-4 Hubungan Komposisi

4) Class1 merupakan **turunan** dari Class2, digambarkan sebagai berikut:



Gambar 5-5 Hubungan Generalisasi

5.2 Hubungan Antar Kelas

Dalam *Object Oriented Programming*, kelas-kelas yang terbentuk dapat memiliki hubungan satu dengan yang lainnya, sesuai dengan kondisi dari kelas-kelas yang bersangkutan. Ada beberapa jenis hubungan yang dapat terjadi antara kelas yang satu dengan kelas yang lainnya, antara lain:

- 1) Asosiasi
- 2) Agregasi
- 3) Komposisi
- 4) Generalisasi dan Spesialisasi (terkait dengan pewarisan)

5.2.1 Asosiasi

Asosiasi merupakan hubungan antara dua kelas terpisah yang terbentuk melalui objek pada tiap kelas. Jika dua kelas dalam suatu model perlu berkomunikasi satu sama lain, harus ada hubungan di antara kelas hal tersebut dapat diwakili oleh **asosiasi**. Contoh hubungan asosiasi ini adalah:



Gambar 5-6 Contoh hubungan asosiasi.

Pada diagram kelas di atas, terlihat hubungan antara kelas Dosen dengan kelas Mahasiswa. Method pada Kelas Mahasiswa dipanggil pada kelas Mahasiswa sehingga memiliki hubungan asosiasi.

Implementasi dari diagram kelas tersebut dalam Java adalah sebagai berikut:

File Mahasiswa.java

```
public class Mahasiswa {
    private String nim;
    private String nama;

    public void setName(String nama){
        this.nama = nama;
    }
    public void setNim(String nim){
        this.nim = nim;
    }
    public String getName(){
        return this.nama;
    }
}
```

```

    public String getNim(){
        return this.nim;
    }
}

```

File Dosen.java

```

public class Dosen {
    private String kodeDosen;
    private String namaDosen;

    //Setter
    public void setKodeDosen(String kodeDosen){
        this.kodeDosen = kodeDosen;
    }
    public void setNamaDosen(String namaDosen){
        this.namaDosen = namaDosen;
    }
    //Getter
    public String getKodeDosen(){
        return this.kodeDosen;
    }
    public String getNamaDosen(){
        return this.namaDosen;
    }

    public void giveScore(Mahasiswa s, int nilai){
        // ini asosiasi, method milik class Student dipanggil di class Dosen,
        // tp objek Student tidak menjadi atribut dr class Lecture

        s.setNilai(nilai);
    }
    public int getScore(Mahasiswa s){
        // ini asosiasi, method milik class Student dipanggil di class Lecture,
        // tp objek Student tidak menjadi atribut dr class Lecture
        return s.getNilai();
    }

    public static void main(String[] args) {
        Mahasiswa m = new Mahasiswa();

        m.setNim("130118383");
        m.setNama("Budi");

        Dosen d = new Dosen();
        d.giveScore(m, 90);

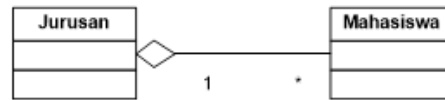
        System.out.println("Nim :"+m.getNim());
        System.out.println("Nama :"+m.getNama());
        System.out.println("Nilai :"+d.getScore(m));
    }
}

```

Pada implementasi di atas terlihat bahwa method pada kelas Mahasiswa dipanggil pada kelas Dosen pada method getScore() dan giveScore() sehingga method pada kelas Dosen menerima nilai dari objek kelas Mahasiswa.

5.2.2 Agregasi

Agregasi merupakan **hubungan** antara **dua kelas** yang **lebih ketat** dan terdapat hubungan **bagian (part)** dan **keseluruhan (whole)**. Contoh hubungan agregasi ini adalah:



Gambar 5-7 Contoh hubungan agregasi.

Pada diagram kelas di atas, terlihat hubungan antara kelas Jurusan dengan kelas Mahasiswa. Kelas mahasiswa merupakan bagian dari kelas jurusan, tetapi kelas jurusan dan kelas mahasiswa dapat diciptakan sendiri-sendiri.

Implementasi dari diagram kelas tersebut dalam Java adalah sebagai berikut:

File Mahasiswa.java

```
public class Mahasiswa {
    private String NIM, Nama;
    public Mahasiswa(String no, String nm) {
        this.NIM = no;
        this.Nama = nm;
    }

    public String GetNIM() {
        return (NIM);
    }

    public String GetNama() {
        return (Nama);
    }
}
```

File Jurusan.java

```
public class Jurusan {
    private String KodeJurusan,>NamaJurusan;
    private Mahasiswa[] Daftar = new Mahasiswa[10];
    public Jurusan(String kode, String nama) {
        this.KodeJurusan = kode;
        this>NamaJurusan = nama;
    }

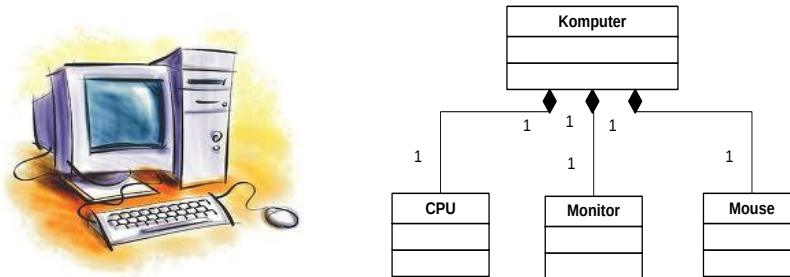
    private static int JmlMhs = 0;
    public void AddMahasiswa(Mahasiswa m) {
        this.Daftar[JmlMhs] = m;
        this.JmlMhs++;
    }

    public void DisplayMahasiswa() {
        int i;
        Sistem.out.println("Kode Jurusan : "+this.KodeJurusan);
        Sistem.out.println("Nama Jurusan : "+this>NamaJurusan);
        Sistem.out.println("Daftar Mahasiswa :");
        for (i=0;i<JmlMhs;i++)
            Sistem.out.println(Daftar[i].GetNIM()+"
"+Daftar[i].GetNama());
    }
}
```

Pada implementasi terlihat bahwa kelas jurusan memiliki atribut yang memiliki tipe kelas mahasiswa, sehingga kelas mahasiswa merupakan bagian dari kelas jurusan.

5.2.3 Komposisi

Komposisi merupakan **bentuk khusus dari agregasi** dimana terdapat hubungan antar kelas yang lebih kuat, karena yang menjadi **part (bagian)** tidak akan ada tanpa adanya kelas yang menjadi whole. Contoh hubungan komposisi adalah sebagai berikut:



Gambar 5-8 Contoh hubungan komposisi.

Dari diagram kelas di atas terlihat bahwa kelas CPU, Monitor, dan Mouse semuanya merupakan bagian dari kelas Komputer dan ketika kelas Komputer musnah maka kelas CPU, *Monitor*, dan *Mouse* akan ikut musnah. Implementasi dari diagram kelas tersebut dalam Java adalah sebagai berikut:

File CPU.java

```
public class CPU {
    private String Merk;
    private int Kecepatan;
    public CPU(String m, int k) {
        this.Merk = m;
        this.Kecepatan = k;
    }
    public void DisplaySpecCPU() {
        Sistem.out.println(this.Merk + ", " + this.Kecepatan);
    }
}
```

File Monitor.java

```
public class Monitor {
    private String Merk;
    public Monitor(String m) {
        this.Merk = m;
    }

    public void DisplaySpecMonitor() {
        Sistem.out.println(this.Merk);
    }
}
```

File Mahasiswa.java

```
//Printer.java
public class Mouse {
    private String Merk, Type;
    public Printer(String m, String t) {
        this.Merk = m;
        this.Type = t;
    }

    public void DisplaySpecMouse() {
        Sistem.out.println(this.Merk + ", " + this.Type);
    }
}
```

Pada implementasi di atas, terlihat bahwa kelas CPU, Monitor, dan Printer merupakan atribut dari kelas Komputer dan baru diciptakan pada saat instansiasi objek dari kelas Komputer.

Modul 6 Inheritance

Tujuan Praktikum

1. Mengerti dan memahami *inheritance* secara mendalam.
2. Memahami konsep *inheritance* dan *polymorfisme* serta dapat mengimplementasikan dalam suatu program.

6.1 Pendahuluan

Pada dasarnya Kita sebagai manusia sudah terbiasa untuk melihat objek yang berada di sekitar Kita tersusun secara hierarki berdasarkan kelas-nya masing-masing. Dari sini kemudian timbul suatu konsep tentang pewarisan yang merupakan suatu proses dimana suatu *class* diturunkan dari kelas lainnya sehingga ia mendapatkan ciri atau sifat dari kelas tersebut.

Dari hierarki di atas, semakin ke bawah, kelas akan semakin bersifat spesifik. Misalnya, sebuah Kelas Pria memiliki seluruh sifat yang dimiliki oleh manusia, demikian halnya juga Wanita.

Penurunan sifat tersebut dalam Bahasa Pemrograman Java disebut dengan *Inheritance* yaitu satu dalam Pilar Dasar OOP (*Object Oriented Programing*), yang dalam implementasinya merupakan sebuah hubungan “adalah bagian dari” atau “*is a relationship*” *Object* yang di-*inherit* (diturunkan).

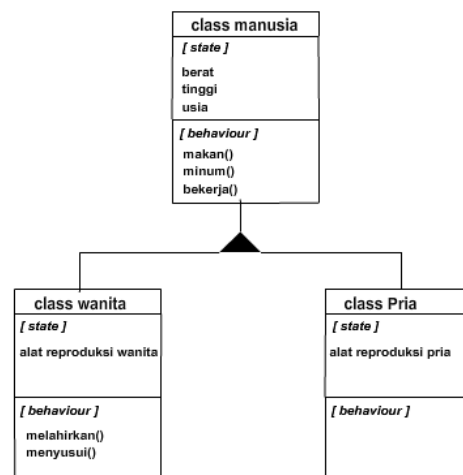
Latar belakang diperlukannya suatu *inheritance* dalam pemrograman Java adalah untuk **menghindari duplikasi *Object*** baik itu *field*, variabel maupun *method* yang sebenarnya merupakan *Object* yang bisa diturunkan dari hanya sebuah kelas. Jadi *inheritance* bukan sebuah Kelas yang diturunkan oleh sebuah *Literial*, tapi lebih menunjukkan ke hubungan antar *Object* itu sendiri.

Sedangkan suatu kemampuan dari sebuah *Object* untuk membolehkan mengambil beberapa bentuk yang berbeda agar tidak terjadi duplikasi *Object* Kita kenal sebagai *Polymorphism*.

Antara penurunan sifat (*inheritance*) maupun *Polymorphism* merupakan konsep yang memungkinkan digunakannya suatu *interface* yang sama untuk memerintah objek agar melakukan aksi atau tindakan yang mungkin secara prinsip sama namun secara proses berbeda. Dalam konsep yang lebih umum sering kali *Polymorphism* disebut dalam istilah tersebut.

6.2 Inheritance

Inheritance merupakan suatu cara untuk menurunkan suatu kelas yang lebih umum menjadi suatu kelas yang lebih spesifik. *Inheritance* adalah salah satu ciri utama suatu bahasa program yang berorientasi pada objek, dan Java pasti menggunakannya. Java mendukung *inheritance* dengan memungkinkan satu kelas untuk bersatu dengan kelas lainnya ke dalam deklarasinya. Dalam inheritance terdapat dua istilah yang sering digunakan. Kelas yang menurunkan disebut kelas dasar (*base class/super class*), sedangkan kelas yang diturunkan disebut kelas turunan (*derived class/subclass*). Karakteristik pada *superclass* akan dimiliki juga oleh *subclass*-nya.



Gambar 6-1 Ilustrasi pada kelas manusia.

Manusia adalah *superclass* dari pria dan wanita. Pria dan wanita mewarisi state dan *behaviour* kelas manusia.

Contoh pewarisan dalam program Java:

```
class A {
    int x;
    int y;
    void tampilXY() {
        System.out.println("nilai x: "+x+" nilai y: "+y);
    }
}
class B extends A {
    int z;
    void jumlahXY()
    {
        System.out.println("jumlah: "+(x+y+z));
    }
}
public class DemoInheritance {
    public static void main(String[] args) {
        A superclass=new A();
        B subclass=new B();
        System.out.println("superclass :");

        superclass.x=3;
        superclass.y=4;
        superclass.tampilXY();
        System.out.println("subclass :");

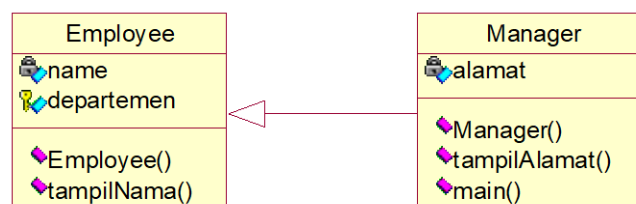
        //member superclass dapat diakses dari subclassnya
        subclass.x=1;
        subclass.y=2;
        subclass.tampilXY();

        //member tambahan hanya ada di subclass
        subclass.z=5;
        subclass.jumlahXY();
    }
}
```

6.3 Keyword Super

Keyword *super* digunakan untuk merefer *superclass* dari suatu kelas, yaitu untuk merefer *member* dari suatu *superclass*, baik atribut maupun *method*.

Contoh :



Gambar 6-2 Contoh kelas dengan *keyword super*.

Dari UML diatas dapat dilihat bahwa kelas **Employee** merupakan *superclass* dari kelas **Manager**.

```
class Employee {
    private String name;
    String departemen;
    public Employee (String s) {
        name = s;
    }
    public void tampilNama()
    {
        System.out.println("nama : "+name);
    }
}
class Manager extends Employee {
    private String alamat;

    public Manager(String nama, String s) {
        /*memanggil konstruktor employee*/
        super(nama);
        alamat = s;
    }

    public void tampilAlamat() {
        /*menginisialisasi variabel departemen yang ada pada superclass*/
        super.departemen="Personalia";

        /*memanggil Method tampilNama() yang ada pada superclass*/
        super.tampilNama();
        /*menampilkan variabel departemen yang telah diinisialisasi*/
        System.out.println("alamat : "+alamat);
        System.out.println("departemen : "+super.departemen);
    }

    public static void main(String[] args) {
        /*membuat objek*/
        Manager adi = new Manager("adi", "sukabirus");
        adi.tampilAlamat();
    }
}
```

Output Program :

```
Nama : adi
Alamat : sukabirus
Departemen : Personalia
```

6.4 Overloading Method

Di dalam java Anda diperbolehkan membuat dua atau lebih *method* dengan nama yang sama dalam satu kelas, tetapi jumlah dan tipe data argumen masing – masing *method* haruslah berbeda satu dengan yang lainnya. Hal itu yang dinamakan dengan *Overloading Method*.

Contoh program:

```
class Lagu {
    String pencipta;
    String judul;

    void IsiParam(String param1) {
        judul = param1;
        pencipta = "Tidak dikenal";
    }

    void IsiParam(String param1,String param2) {
        judul = param1;
        pencipta = param2;
    }

    void CetakKeLayar() {
        System.out.println("Judul : " + judul + ", pencipta : " + pencipta);
    }
}

class DemoOver2 {
public static void main(String[] args) {
    Lagu d,e;
    d = new Lagu();
    e = new Lagu();

    d.IsiParam("Lagu 1");
    e.IsiParam("kepastian yang kutunggu","GiGi");

    d.CetakKeLayar();
    e.CetakKeLayar();
}
}
```

Pada contoh program di atas, Kita melakukan *Overloading* terhadap *method* *IsiParam()*. Di mana *IsiParam()* yang pertama menerima satu buah parameter bertipe String dan *IsiParam()* yang kedua menerima dua buah parameter bertipe String. Sehingga inilah hasil eksekusi program yang didapat:

```
Judul : Lagu 1, pencipta : Tidak dikenal
Judul : kepastian yang kutunggu: GiGi
```

6.5 Overriding Method

Di dalam java, jika dalam suatu *subclass* Anda mendefinisikan sebuah *method* yang sama dengan yang dimiliki oleh superclass, maka *method* yang Anda buat dalam *subclass* tersebut dikatakan meng-*override* superclass-nya. Sehingga jika Anda mencoba memanggil *method* tersebut dari *instance subclass* yang Anda buat, maka *method* milik *subclass*-lah yang akan dipanggil, bukan lagi *method* milik superclass-nya.

Contoh program:

```
class A {
    public void tampilkanKeLayar() {
        System.out.println("Method milik class A dipanggil...");
    }
}

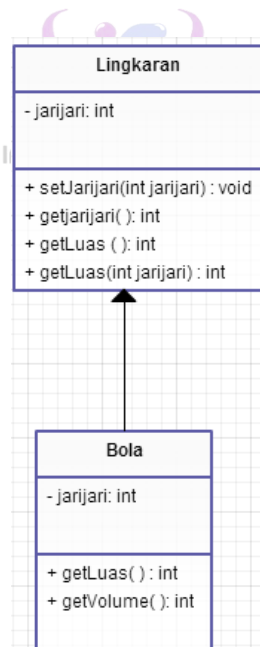
class B extends A {
    public void tampilkanKeLayar() {
        super.tampilkanKeLayar(); // Method milik superclas dipanggil
        System.out.println("Method milik class B dipanggil...");
    }
}

class DemoInheritance {
    public static void main(String[] args) {
        B subOb = new B();
        subOb.tampilkanKeLayar();
    }
}
```

Output program diatas:

```
Method milik class A dipanggil...
Method milik class B dipanggil...
```

Terlihat pada contoh di atas, **kelas B** yang merupakan turunan dari **kelas A**, meng-*override* *method* `tampilkanKeLayar()`, sehingga pada waktu Kita memanggil *method* tersebut dari variabel *instance* **kelas B** (variabel `subOb`), yang terpanggil adalah *method* yang sama yang ada pada **kelas B**.



Perhatikan kelas diagram disamping. Jawablah pertanyaan dibawah ini!

- 1) Sebutkan *method* mana yang melakukan *override*?
- 2) Sebutkan *method* mana yang melakukan *overload*?
- 3) Buatlah *syntax* yang mengaplikasikan kelas diagram tersebut.

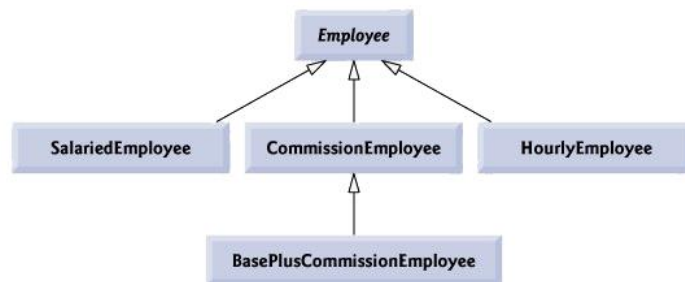
Modul 7 Abstract dan Interface

Tujuan Praktikum

1. Memahami dan mengerti konsep *interface* dan mampu mengimplementasikan dalam Java.
2. Memahami dan mengerti konsep abstract dan mampu mengimplementasikan dalam Java.
3. Mampu membedakan antara kelas abstract dan *interface*.

7.1 Kelas Abstrak

Kata kunci abstrak digunakan untuk *method* atau *class* yang **belum memiliki implementasi**. Abstrak *method* dideklarasikan pada abstrak *class*. Kelas yang dideklarasikan sebagai abstrak tidak akan bisa dibentuk *Object* dalam Java. Jadi kelas abstrak tidak akan bisa digunakan di dalam program Kita yang lain, kecuali kelas abstrak tersebut diturunkan dan turunan dari kelas abstrak tersebut yang bisa dijadikan *Object*. Sebagai ilustrasi dari kelas abstrak perhatikan gambar di bawah ini :



Gambar 7-1 Contoh kelas abstrak dalam UML.

Berdasarkan gambar diatas kelas **Employee** digunakan untuk merepresentasikan konsep umum pegawai. Kelas **Employee** diperluas ke kelas **SalariedEmployee**, **CommissionEmployee** dan **HourlyEmployee**. **BasePlusCommissionEmployee** yang memperluas **CommisionEmployee** merepresentasikan tipe akhir pegawai. Berdasarkan gambar kelas **Employee** diketik miring menandakan bahwa kelas tersebut merupakan kelas abstrak.

Berikut contoh program dari kelas abstrak *employee* dan Kelas *SalariedEmployee*.

Kelas Abstrak *Employee*

```
public abstract class Employee
{
    private String firstName;
    private String lastName;
    private String socialSecurityNumber;
    // three-argument constructor
    public Employee( String first, String last, String ssn )
    {
        firstName = first;
        lastName = last;
        socialSecurityNumber = ssn;
    } // end three-argument Employee constructor
    // set first name
    public void setFirstName( String first )
    {
        firstName = first;
    } // end Method setFirstName
    // return first name
    public String getFirstName()
    {
        return firstName;
    } // end Method getFirstName
    // set last name
    public void setLastName( String last )
    {
        lastName = last;
    } // end Method setLastName
    // return last name
    public String getLastName()
    {
        return lastName;
    } // end Method getLastName
    // set social security number
    public void setSocialSecurityNumber( String ssn )
    {
        socialSecurityNumber = ssn; // should validate
    } // end Method setSocialSecurityNumber

    // return social security number
    public String getSocialSecurityNumber()
    {
        return socialSecurityNumber;
    } // end Method getSocialSecurityNumber

    // return String representation of Employee Object
    public String toString()
    {
        return String.format( "%s %s\nsocial security number: %s",
            getFirstName(), getLastName(), getSocialSecurityNumber() );
    } // end Method toString

    // abstract Method overridden by subclasses
    public abstract double earnings(); // no implementation here
} // end abstract class Employee
```

Kelas *SalariedEmployee* yang merupakan turunan dari Kelas Abstrak.

```
public class SalariedEmployee extends Employee {

    private double weeklySalary;

    // four-argument constructor
    public SalariedEmployee( String first, String last, String ssn,
        double salary )
    {
        super( first, last, ssn ); // pass to Employee constructor
        setWeeklySalary( salary ); // validate and store salary
    } // end four-argument SalariedEmployee constructor

    // set salary
    public void setWeeklySalary( double salary )
    {
        weeklySalary = salary < 0.0 ? 0.0 : salary;
    } // end Method setWeeklySalary

    // return salary
    public double getWeeklySalary()
    {
        return weeklySalary;
    } // end Method getWeeklySalary

    // calculate earnings; override abstract Method earnings in Employee
    public double earnings()
    {
        return getWeeklySalary();
    } // end Method earnings

    // return String representation of SalariedEmployee Object
    public String toString()
    {
        return String.format( "salaried employee: %s\n%s: $%,.2f",
            super.toString(), "weekly salary", getWeeklySalary() );
    } // end Method toString
} // end class SalariedEmployee
```

7.2 Interface

Interface serupa dengan kelas. Kelas mendefinisikan kemampuan sebuah *Object*, sementara *interface* mendefinisikan method atau konstanta yang akan diimplementasikan pada *Object* yang lain. *Interface* membantu mendefinisikan sifat *Object* dengan mendeklarasikan seperangkat karakteristik *Object* tersebut. Sebagai contoh kasus, radio, TV, dan *speaker* komputer memiliki pengontrol volume. Untuk alasan ini, mungkin Kita menginginkan *device-device* ini mengimplementasikan *interface* yang bernama `VolumeControl()`.

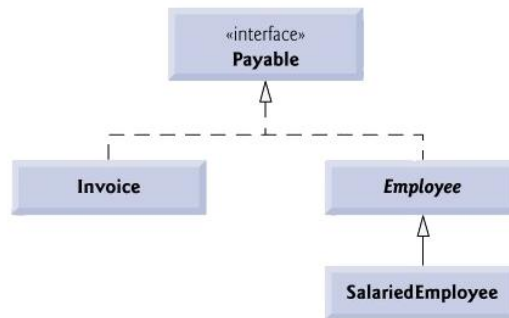
Interface pada Java 8 memiliki beberapa batasan:

- 1) Semua atribut adalah *public*, *static* dan *final* (semua atribut bertindak sebagai konstanta)
- 2) Hanya memiliki *default method*, dan *static method*
- 3) Tidak boleh ada deklarasi konstruktor.

Berbeda dengan *class* biasa, *interface* mengizinkan **multiple inheritance**, yaitu sebuah kelas dapat mempunyai dua *interface* induk sekaligus. Hal tersebut tidak dapat dilakukan pada sebuah kelas, dimana sebuah kelas hanya boleh mempunyai satu kelas induk.

Syntax yang diperlukan hampir sama dengan cara **membuat kelas**. Perbedaannya hanya *method* pada **interface** tidak memiliki isi/ **variabel**. Sebuah contoh dengan tiga item: *interface*, kelas yang

mengimplementasikan *interface*, dan kelas yang menggunakan kelas yang disalurkan. Sebagai ilustrasi perhatikan gambar UML berikut:



Gambar 7-2 Contoh kelas *interface* dalam UML.

Berdasarkan gambar diatas hierarki dimulai dengan *interface Payable*. Relasi UML yang terlihat antara kelas dan interface dinamakan **realization**. Sebuah kelas dikatakan merealisasikan atau mengimplementasikan method pada sebuah **interface**.

Berikut contoh program dari UML diatas (Hanya *interface Payable* dan Kelas Invoice) .

Interface Payable

```

public interface Payable
{
    double getPaymentAmount(); // calculate payment; no implementation
} // end interface Payable
    
```

Kelas Invoice yang mengimplementasikan Payable

```

public class Invoice implements Payable
{
    private String partNumber;
    private String partDescription;
    private int quantity;
    private double pricePerItem;

    // four-argument constructor
    public Invoice( String part, String description, int count,
        double price )
    {
        partNumber = part;
        partDescription = description;
        setQuantity( count ); // validate and store quantity
        setPricePerItem( price ); // validate and store price per item
    } // end four-argument Invoice constructor

    // set part number
    public void setPartNumber( String part )
    {
        partNumber = part;
    } // end Method setPartNumber

    // get part number
    public String getPartNumber()
    {
        return partNumber;
    } // end Method getPartNumber

    // set description
    public void setPartDescription( String description )
    {
        partDescription = description;
    } // end Method setPartDescription
}
    
```

```

// get description
public String getPartDescription()
{
    return partDescription;
} // end Method getPartDescription

// set quantity
public void setQuantity( int count )
{
    quantity = ( count < 0 ) ? 0 : count; // quantity cannot be negative
} // end Method setQuantity

// get quantity
public int getQuantity()
{
    return quantity;
} // end Method getQuantity

// set price per item
public void setPricePerItem( double price )
{
    pricePerItem = ( price < 0.0 ) ? 0.0 : price; // validate price
} // end Method setPricePerItem

// get price per item
public double getPricePerItem()
{
    return pricePerItem;
} // end Method getPricePerItem

// return String representation of Invoice Object
public String toString()
{
    return String.format( "%s: \n%s: %s (%s) \n%s: %d \n%s: $%,.2f",
        "invoice", "part number", getPartNumber(), getPartDescription(),
        "quantity", getQuantity(), "price per item", getPricePerItem() );
} // end Method toString

// Method required to carry out contract with interface Payable
public double getPaymentAmount()
{
    return getQuantity() * getPricePerItem(); // calculate total cost
} // end Method getPaymentAmount
} // end class Invoice

```

```

public interface Product {
    static final String MAKER = "My Corp";
    static final String PHONE = "555-123-4567";
    public int getPrice(int id);
}

public class Shoe implements Product {
    public int getPrice(int id) {
        if (id == 1)
            return(5);
        else
            return(10);
    }

    public String getMaker() {
        return (MAKER);
    }
}

```



```

public class Store {
    static Shoe hightop;
    public static void init() {
        hightop = new Shoe();
    }
}

public static void main(String argv[]) {
    init();
    getInfo(hightop);
    orderInfo(hightop);

    public static void getInfo(Shoe item) {
        System.out.println("This Product is made by " + item.MAKER);
        System.out.println("It costs $" + item.getPrice(1) + '\n');
    }

    public static void orderInfo(Product item) {
        System.out.println("To order from "+item.MAKER+" call "+item.PHONE+".");
        System.out.println("Each item costs $" + item.getPrice(1));
    }
}

```

Interface sebelum Java 8 hanya dapat memiliki *method* abstrak. Semua *method* di *Interface* bersifat publik dan abstrak secara *default*. Namun pada Java 8 memungkinkan *interface* untuk memiliki *method default* dan *static*. Hal tersebut dikarenakan *default method* pada *interface* dapat memungkinkan pengembang menambahkan *method* baru ke *interface* tanpa mempengaruhi kelas yang mengimplementasikan *interface*.

Default method pada *Interface* :

```

interface MyInterface{
    default void newMethod(){
        System.out.println("Newly added default method");
    }
    void existingMethod(String str);
}

public class Example implements MyInterface{
    // implementasi abstract method
    public void existingMethod(String str){
        System.out.println("String is: "+str);
    }
    public static void main(String[] args) {
        Example obj = new Example();
        obj.newMethod();
        obj.existingMethod("Java 8 is easy to learn");
    }
}

```

Method **newMethod** pada *interface* **MyInterface** yang berarti kita tidak perlu mengimplementasikan *method* tersebut di kelas implementasi **Example**. Dengan menggunakan metode ini kita dapat menambahkan *method default* ke dalam *interface* tanpa perlu khawatir dengan kelas yang mengimplementasikan *interface* tersebut.

Static method pada *Interface* :

```
interface MyInterface{
    default void newMethod(){
        System.out.println("Newly added default method");
    }
    static void anotherNewMethod(){
        System.out.println("Newly added static method");
    }
    void existingMethod(String str);
}
public class Example implements MyInterface{
    // implementing abstract method
    public void existingMethod(String str){
        System.out.println("String is: "+str);
    }
    public static void main(String[] args) {
        Example obj = new Example();

        obj.newMethod();
        MyInterface.anotherNewMethod();
        obj.existingMethod("Java 8 is easy to learn");
    }
}
```

Static method pada *interface* hampir sama dengan *default method* sehingga kita tidak perlu mengimplementasikannya di kelas **Example**. Kita dapat dengan aman menambahkannya ke *interface* yang ada tanpa mengubah kode di kelas **Example**. Karena *static method* tidak dapat menyimpannya di kelas **Example**.

Perbedaan **abstract class** dan **interface** antara lain:

- 1) Semua *interface* Method tidak memiliki body sedangkan beberapa *abstract class* dapat memiliki Method dengan implementasi.
- 2) Sebuah *interface* hanya dapat mendefinisikan *constant* sedangkan sebuah *abstract class* tampak seperti kelas biasa yang dapat mendeklarasikan variabel.
- 3) *Interface* tidak memiliki hubungan *inheritance* secara langsung dengan sebuah *kelas* tertentu, sedangkan *abstract class* bisa jadi hasil turunan dari *abstract class* induknya.
- 4) *Interface* memungkinkan terjadinya pewarisan jamak (*multiple inheritance*) sedangkan *abstract class* tidak.

Modul 8 Presentasi Tugas Besar (Diagram Kelas)

Tujuan Praktikum
1. Mahasiswa mempresentasikan diagram kelas yang akan digunakan untuk membangun aplikasi untuk tugas besar

8.1 Presentasi Tugas Besar

Pada Modul 8 ini, mahasiswa diharapkan sudah memahami diagram kelas dan dapat merencanakan aplikasi yang akan dibuat untuk tugas besar dengan cara membuat kelas diagramnya terlebih dahulu. Selain itu, mahasiswa juga diharapkan dapat menjelaskan dan mempresentasikan kelas diagram yang telah dibuat kepada asisten praktikum.



Modul 9 Polymorphism, Static Class, & Collection

Tujuan Praktikum

1. Memahami dan mengerti konsep *polymorphism* dan mampu mengimplementasikan dalam Java.
2. Menjelaskan pengertian *inner class* dan menjelaskan penggunaannya.
3. Menjelaskan pengertian *static variable* dan *static method* dan menjelaskan penggunaannya.
4. Menjelaskan pengertian collection dan menjelaskan jenis-jenis collection serta perbedaan penggunaannya.
5. Menjelaskan pengertian generics dan menjelaskan penggunaan generics.

9.1 Pendahuluan

Polymorphism berasal dari bahasa Yunani yang berarti banyak bentuk. Dalam PBO, konsep ini memungkinkan digunakannya suatu *interface* yang sama untuk memerintah objek agar melakukan aksi atau tindakan yang mungkin secara prinsip sama namun secara proses berbeda.

Dalam Konsep yang lebih umum sering kali *Polymorphism* disebut dalam istilah satu *interface* banyak aksi. Contoh yang konkrit dalam dunia nyata yaitu mobil.

Mobil yang ada dipasaran terdiri atas berbagai tipe dan berbagai merk, namun semuanya memiliki *interface* kemudi yang sama, seperti: stir, tongkat transmisi, pedal gas dan rem. Jika seseorang dapat mengemudikan satu jenis mobil saja dari satu merk tertentu, maka orang itu akan dapat mengemudikan hampir semua jenis mobil yang ada, karena semua mobil tersebut menggunakan *interface* yang sama.

Interface yang sama tidak berarti cara kerjanya juga sama. Misal pedal gas, jika ditekan maka kecepatan mobil akan meningkat, tapi proses peningkatan kecepatan ini berbeda-beda untuk setiap jenis mobil. Dengan OOP maka dalam melakukan pemecahan suatu masalah Kita tidak melihat bagaimana cara menyelesaikan suatu masalah tersebut (terstruktur) tetapi objek-objek apa yang dapat melakukan pemecahan masalah tersebut.

Sebagai contoh anggap Kita memiliki departemen yang memiliki manager, sekretaris, petugas administrasi data dan lainnya. Misal manager ingin memperoleh data dari bag administrasi maka manager tersebut dapat menyuruh petugas bag administrasi untuk mengambilnya. Pada kasus tersebut, manager tidak harus mengetahui bagaimana cara mengambil data tetapi manager bisa mendapatkan data tersebut melalui objek petugas administrasi.

Jadi untuk menyelesaikan suatu masalah dengan kolaborasi antar objek-objek yang ada karena setiap objek memiliki deskripsi tugasnya sendiri.

9.2 Referensi Variabel Casting

Terdapat 2 tipe variabel reference yaitu **Downcasting** dan **Upcasting**.

Downcasting: apabila terdapat variabel *reference* yang mengacu pada sub tipe objek, maka dapat dimasukkan ke dalam sub tipe variabel *reference*. Dengan kata lain **downcasting** hanya dapat dilakukan bila antara type dari *Object* dan type dari variabel *reference* masih dalam sebuah inheritance. Bila ada *Polymorphisme* sebagai berikut:

```
A ref = new B();  
// Dimana B adalah subclass dari A, maka dari variabel ref Kita hanya akan dapat  
meng-invoke (menjalankan) Method-Method yang dideklarasikan di kelas A. Bila Kita  
ingin meng-invoke Method yang hanya terdapat di kelas B, maka Kita tidak dapat  
secara langsung memanggil Method yang hanya ada di B seperti sebagai berikut:  
ref.MethodDiB(); //COMPILE TIME ERROR !!!!
```

Potongan program di atas akan menghasilkan *compile time error* sbb:

Cannot find symbol

Agar dapat **meng-*invoke* method** yang ada di **kelas B**, Kita dapat menggunakan **casting** seperti sebagai berikut:

```
//cara 1
B b = (B) ref;    //CASTING !!!!
b.MethodDiB();
//Atau :
//cara 2
((B)ref).MethodDiB(); //CASTING !!!!
```

Contoh :

```
class A {
    void MethodDiA() {
        System.out.println("A.MethodDiA()");
    }
}
class B extends A {
    void MethodDiB() {
        System.out.println("B.MethodDiB()");
    }
}
class Polymorphisme01 {
    public static void main(String[] args) {
        A ref = new B();
        //cara 1
        B b = (B) ref;
        b.MethodDiB();
        //cara 2
        ((B)ref).MethodDiB();
    }
}
```

Program di atas akan menghasilkan :

B.MethodDiB()
B.MethodDiB()

Downcasting hanya dapat dilakukan bila **antara type** dari *Object* dan *type* dari variabel *reference* masih dalam sebuah **inheritance**. Bila tidak, maka akan terjadi *compile time error* (*inconvertible types*). Contoh:

```
class A {}
class B extends A {}
class C {}

class Test {
    void lakukanSesuatu() {
        A ab = new B();
        B b = (B) ab;    //Downcast berhasil
        C c = (C) ab;    //Downcast GAGAL
    }
}
```

Compile time error yang dihasilkan adalah :

inconvertible types

Ada kemungkinan *downcast* tidak *error* saat *compile time*, akan tetapi melemparkan **ClassCastException**. Hal ini terjadi karena *type* dari *Object* yang di-assign ke suatu variabel *reference* adalah *superclass* dari *type* dari variabel *references* tersebut.

Contoh:

```
class A {}
class B extends A {}

class Test {
    public static void main(String[] args) {
        Test t = new Test();
        t.lakukanSesuatu();
    }

    void lakukanSesuatu() {
        A a = new A();
        /*Statement di bawah dapat compile, akan tetapi
        *saat dijalankan akan melemparkan exception
        *ClassCastException !!!!!
        */
        B b = (B) a;
    }
}

/*
Saat compile berhasil !!! (dengan javac)
Saat dijalankan (dengan java) akan menghasilkan :
java.lang.NoClassDefFoundError: inheritance/VarRefCast03
Exception in thread "main"
Java Result: 1
*/
```

Kesimpulan untuk *downcast* adalah :

- 1) *Downcast* pasti eksplisit.
- 2) *Downcast* akan berhasil saat *compile time* bila antara *Object* dengan variabel *reference* masih dalam 1 buah jalur *inheritance*.
- 3) *Downcast* akan gagal saat runtime (melemparkan *exception*) bila ternyata *type* dari *Object* yang di-assign ke variabel *reference* ternyata adalah *superclass* dari variabel *reference* itu.

Upcasting: mengisikan variabel *reference* ke variabel *reference* lainnya dengan tipe superclassnya.

Upcasting bisa dilakukan dengan dua cara :

- 1) **Implicit upcasting** (otomatis)
- 2) **Eksplisit upcasting**

Contoh :

```
class A {}
class B extends A {}
class VarRefCast01 {
    public static void main(String[] args) {
        B b = new B();
        //implicit upcasting
        //dari variabel reference ke variabel reference lainnya
        A ab1 = b;
        //implicit upcasting
        //dari suatu Object ke variabel reference
        A ab2 = new B();
        //explicit upcasting
        //dari variabel reference ke variabel reference lainnya
        A ab3 = (A) b;
        //explicit upcasting
        //dari Object ke variabel reference
        A ab4 = (A) new B();
    }
}
```

9.3 Static

9.3.1 Static Variabel

Static variabel yaitu pendefinisian variabel milik kelas yang tidak memerlukan *instant object* untuk mengacunya. Variabel ini dapat digunakan bersama oleh semua *instant object*. Static variabel juga biasa disebut variabel *class*, serta *local* variabel tidak dapat didefinisikan sebagai static.

Contoh penggunaan static variabel sebagai berikut:

```
public class Share{
    private int privateInt;
    private static int staticInt;
    public Share(int pr, int si){
        privateInt = pr;
        staticInt = si;
    }
    public String toString(){
        return privateInt + "" + staticInt;
    }
}

public class Driver{
    public static void main(String args[]){
        Share s1 = new Share(4,4);
        System.out.println(s1.toString());
        Share s2 = new Share(8,2);
        System.out.println(s1.toString());
        System.out.println(s2.toString());
        Share s3 = new Share(6,22);
        System.out.println(s1.toString());
        System.out.println(s2.toString());
        System.out.println(s3.toString());
    }
}
```

Output :

```
> 4 4
> 4 2
> 8 2

> 4 22
> 8 22
> 6 22
```

9.3.2 Static Method

Static *method* merupakan pendefinisian *method* (milik) kelas, dengan demikian tidak memerlukan *instant object* untuk menjalankannya. *Method* ini tidak dapat menjalankan *method* yang bukan static serta tidak dapat mengacu variabel yang bukan static.

Contoh penggunaan static *method* adalah sebagai berikut :

```
public class CounterMachine{
    public static void count(){
```

```

        counter++;}
    }

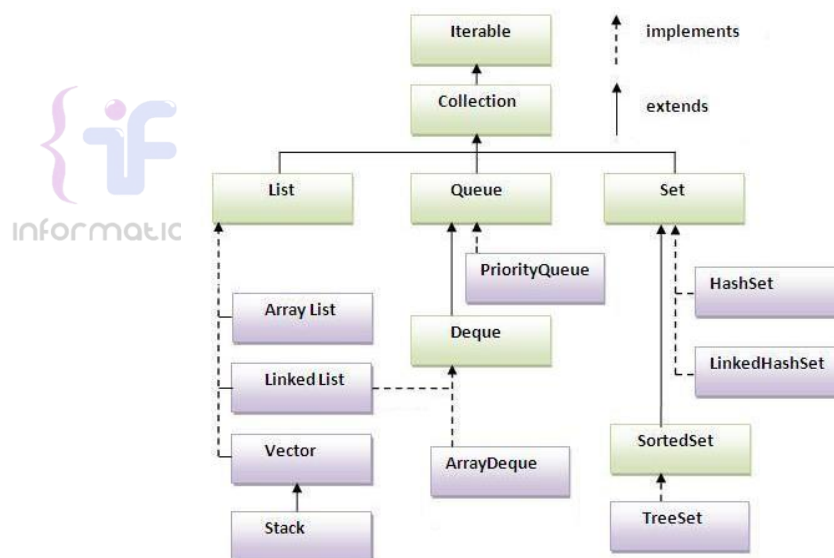
    public class Driver{
        public static void main(String args[]){
            for(int i = 0; i< 10; i++){
                if( i % 2 == 0){
                    CounterMachine.count();
                }
            }
            System.out.println(CounterMachine.counter);
        }
    }
}

```

9.4 Collection

Collection adalah objek khusus yang digunakan untuk menampung banyak objek yang bertipe apapun. *Collection* dapat menampung banyak objek bertipe *Object*. Seperti diketahui, semua kelas di Java merupakan turunan dari kelas *Object*, maka dengan prinsip polimorfisme, *collection* dapat menampung objek bertipe apapun.

Collection didefinisikan sebagai *interface*. Kita dapat menggunakan kelas turunan dari *Collection*. Contoh hierarki yang ada pada *Collection* dapat dilihat pada Gambar 9-1.



Gambar 9-1 Hierarki dari *Collection*.

Dengan menggunakan *generics*, Kita dapat membatasi tipe yang dapat ditampung oleh *Collection*. Contohnya sebagai berikut.

```

Collection<String> str = new HashSet<String>();
List<Integer> arr_i = new List ();
ArrayList<Employee> emp = new ArrayList ();

```

Beberapa *method* yang ada pada *Collection* antara lain:

Table 9-1 Beberapa *Method Collection*.

Nama Method	Deskripsi
boolean add(Object item)	Menambahkan item ke dalam <i>collection</i>
boolean remove(Object item)	Menghapus item dari dalam <i>collection</i>
boolean addAll(Collection c)	Menambahkan semua item dari c ke dalam <i>collection</i>

<code>boolean removeAll(Collection c)</code>	Menghapus semua item yang ada pada <i>c</i> dari dalam <i>collection</i>
<code>boolean retainAll(Collection c)</code>	Menghapus semua item yang tidak ada pada <i>c</i> dari dalam <i>collection</i>
<code>boolean contains(Object item)</code>	Mengecek apakah item ada di dalam <i>collection</i>

Terdapat beberapa cara yang dapat digunakan untuk mengakses semua elemen dari *Collection*.

```
import java.util.*;

public class JavaApplication1 {
    public static void main(String[] args) {
        List list = new ArrayList();
        list.add(200);
        list.add("Hello");
        list.add(235);
        list.add("Donny");

        // normal loop
        System.out.println("Normal loop");
        for (int i = 0; i < list.size(); i++) {
            Object o = list.get(i);
            System.out.println(o);
        }

        // loop using for-element
        System.out.println("for-element loop");
        for (Object o : list) {
            System.out.println(o);
        }

        // Loop using iterator
        System.out.println("loop using iterator");
        Iterator itr = list.iterator();
        while (itr.hasNext()) {
            Object o = itr.next();
            System.out.println(o);
        }

        // loop using lambda Expression
        System.out.println("loop using lambda expression");
        list.forEach(o -> System.out.println(o));

        // loop using reference
        System.out.println("loop using reference");
        list.forEach(System.out::println);
    }
}
```

9.4.1 Melakukan Sorting pada Collection

Untuk melakukan *sorting* pada *Collection*, Kita dapat menggunakan dua acara:

- 1) Menggunakan *interface Comparable*

Untuk melakukan *sorting*, dapat menggunakan *method* `Collections.sort(list)` atau `list.sort()`. Jika *list* mengandung elemen bertipe *String*, maka *list* akan diurutkan berdasarkan urutan abjad. Jika *list* mengandung elemen bertipe *Date*, maka *list* akan diurutkan berdasarkan urutan tanggal. Untuk membangun pengurutan sendiri pada *list* bertipe kelas tertentu, maka kelas tersebut harus mengimplementasikan *interface Comparable* dan meng-*override* *method* `compareTo(Object obj)`. *Method* `compareTo(Object obj)` akan mengembalikan:

- Nilai < 0, jika objek yang memanggil diurutkan sebelum objek obj
- Nilai = 0, jika objek yang memanggil sama dengan objek obj
- Nilai > 0, jika objek yang memanggil diurutkan setelah objek obj

Jika parameter pada `compareTo()` adalah *String* atau *Date*, maka pada `compareTo()` panggil:

```
return this.getString().compareTo(obj.getString())
```

Jika parameter pada `compareTo()` adalah angka, maka pada `compareTo()` melakukan pengurangan, seperti:

```
return this.getNumber() - obj.getNumber()
```

Contoh penggunaan *interface Comparable* sebagai berikut.

```
import java.util.*;
public class Employee implements Comparable<Employee> {
    private String name;
    private int salary;
    public Employee(String name, int salary) {
        this.name = name;
        this.salary = salary;
    }
    public String getName() {
        return name;
    }
    public int getSalary() {
        return salary;
    }
    @Override
    public String toString() {
        return "name=" + name + ", salary=" + salary;
    }
    @Override
    public int compareTo(Employee e) {
        return name.compareTo(e.name);
    }
    public static void main(String[] args) {
        List<Employee> listEmp = new ArrayList();
        listEmp.add(new Employee("bobby", 3000));
        listEmp.add(new Employee("erick", 1600));
        listEmp.add(new Employee("rey", 2500));
        listEmp.add(new Employee("anna", 3500));
        Collections.sort(listEmp);
        listEmp.forEach(System.out::println);
    }
}
```

Output yang muncul sebagai berikut:

```
name=anna, salary=3500.0
name=bobby, salary=3000.0
name=erick, salary=1600.0
name=rey, salary=2500.0
```

2) Menggunakan *interface Comparator<T>*

Interface comparable memungkinkan untuk melakukan *sorting* pada satu *property* saja. Untuk melakukan *sorting* pada *multiple property*, gunakan *interface Comparator*. Caranya adalah dengan membuat kelas yang mengimplementasikan *interface Comparator<T>* dan meng-*override Method* `compare(T obj1, T obj2)`. Contohnya Kita dapat membangun kelas `comparator<>` untuk kelas `Employee ()` sebagai berikut:

```
import java.util.*;
public class SalaryComparator implements Comparator<Employee> {
    @Override
    public int compare(Employee e1, Employee e2) {
        return e1.getSalary() - e2.getSalary();
    }
}
```

Penggunaan kelas Comparator pada *method* main sebagai berikut.

```
public static void main(String[] args) {
    List<Employee> listEmp = new ArrayList();
    listEmp.add(new Employee("bobby", 3000));
    listEmp.add(new Employee("erick", 1600));
    listEmp.add(new Employee("rey", 2500));
    listEmp.add(new Employee("anna", 3500));
    Collections.sort(listEmp);
    System.out.println("Sorted by name");
    listEmp.forEach(System.out::println);
    Collections.sort(listEmp, new SalaryComparator());
    System.out.println("Sorted by salary");
    listEmp.forEach(System.out::println);
}
```

Output yang dihasilkan sebagai berikut.

```
Sorted by name
name=anna, salary=3500
name=bobby, salary=3000
name=erick, salary=1600
name=rey, salary=2500
Sorted by salary
name=erick, salary=1600
name=rey, salary=2500
name=bobby, salary=3000
name=anna, salary=3500
```



9.4.2 Melakukan Filtering pada Collection

Untuk melakukan *filtering* pada *Collection*, Kita dapat menggunakan dua cara. Misalkan Kita ingin melakukan *filtering* pada *list Employee* berdasarkan kriteria tertentu. Misalkan Kita ingin mencari daftar *Employee* yang memiliki gaji minimal 3000. Maka, Kita dapat menggunakan dua cara.

- 1) Menggunakan cara konvensional seperti berikut.

```
System.out.println("Employee with salary minimal 3000");
for(Employee e: listEmp) {
    if(e.getSalary() >= 3000) {
        System.out.println(e);
    }
}
```

- 2) Menggunakan ekspresi lambda seperti berikut.

```
System.out.println("Employee with salary minimal 3000");
List<Employee> temp = listEmp.stream().
    filter(e -> e.getSalary() >= 3000).collect(Collectors.toList());
temp.forEach(System.out::println);
```

Atau Kita ingin mencari **Employee** dengan nama tertentu. Maka Kita dapat menggunakan dua cara:

- 1) Menggunakan cara konvensional seperti berikut.

```
System.out.println("Employee erick");
for(Employee e: listEmp) {
    if(e.getName().equals("erick")) {
```

```
        System.out.println(e);  
    }  
}
```

2) Menggunakan ekspresi lambda seperti berikut.

```
System.out.println("Employee erick");  
System.out.println(listEmp.Stream().  
    filter(e -> e.getName().equals("erick")).  
    findFirst().orElse(null));
```



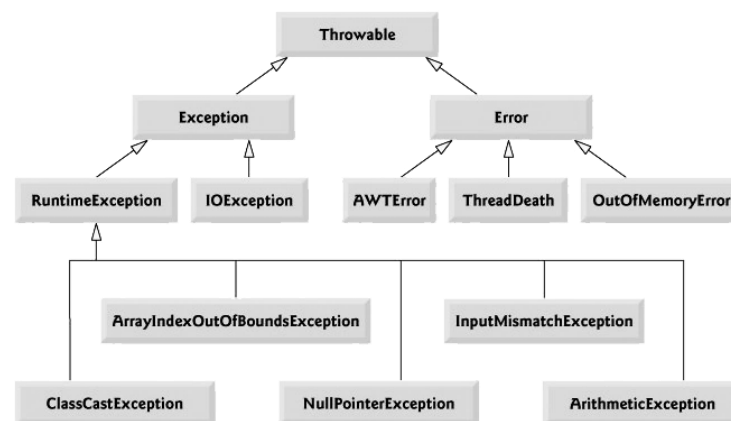
Modul 10 Exception

Tujuan Praktikum

1. Memahami konsep *Exception*

10.1 Exception

Exception adalah **kondisi abnormal / tidak wajar yang terjadi pada saat pengeksekusian** suatu perintah. Dalam java ada lima **keywords** yang digunakan untuk menangani eksepsi ini yaitu : **try, catch, finally, throw, dan throws**. Berikut adalah gambar hierarki dari tipe - tipe eksepsi.



Gambar 10-1 Hierarki dari tipe-tipe eksepsi.

Superclass tertinggi adalah **class Throwable**, tetapi kelas ini tidak akan pernah menggunakan kelas secara langsung. Dibawah **class Throwable** terdapat dua *subclass* yaitu **class Error** dan **class Exception**. **Class Error** merupakan tipe eksepsi yang seharusnya tidak ditangani dengan blok *try catch* dalam program, karena berhubungan dengan Java *run-time system*. Hal ini dikarenakan eksepsi yang terjadi sangat kritis. Sedangkan **class Exception** merupakan tipe eksepsi yang memang sebaiknya ditangani oleh program Anda secara langsung.

10.2 Eksepsi yang tidak dicek

Semua eksepsi yang bertipe *RuntimeException* dan turunannya tidak harus ditangani dalam program Anda.

```
class DemoEksepsi
{
    public static void main(String[] args)
    {
        int[] arr = new int[1];
        System.out.println(arr[1]);
    }
}
```

Output-nya:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 1
    at DemoEksepsi.main(DemoEksepsi.java:6)
```

10.3 Eksepsi yang dicek

Semua tipe eksepsi yang bukan turunan dari *class RuntimeException* merupakan eksepsi yang harus ditangani oleh program Anda. Java bahkan tidak mengizinkan Anda mengompilasi program yang Anda buat, jika tidak menangani eksepsi tersebut. (biasanya yang berhubungan dengan pengaksesan I/O)

10.4 Penggunaan Blok Try Catch

Untuk menangani eksepsi yang terjadi dalam program, Anda cukup meletakkan kode yang ingin Anda inspeksi terhadap kemungkinan eksepsi di dalam blok *try*, diikuti dengan blok *catch* yang menentukan jenis eksepsi apa yang ingin Anda tangani dalam blok *catch* tersebut.

Contoh program:

```
class DemoEksepsi {
    public static void main(String[] args) {
        try {
            int[] arr = new int[1];
            System.out.println(arr[1]);
            System.out.println("Baris ini tidak akan pernah dieksekusi...");
        } catch(ArithmeticException e) {
            System.out.println("Terjadi eksepsi karena indeks di luar kapasitas array");
        }
        System.out.println("Sesudah blok try catch");
    }
}
```

Output-nya:

```
Terjadi eksepsi karena indeks di luar kapasitas array
Sesudah blok try catch
```

Dapat terjadi kode yang terdapat dalam **blok try mengakibatkan lebih dari satu jenis eksepsi**. Dalam hal ini, Kita dapat menuliskan lebih dari satu blok *catch* untuk setiap blok *try*. Berikut adalah contohnya.

```
class DemoEksepsi {
    public static void main(String[] args){
        try {
            int x= args.length;
            int y= 100/x;
            int[] arr={10,11};
            y = arr[x];
            System.out.println("Tidak terjadi eksepsi");
        } catch(ArithmeticException e) {
            System.out.println("Terjadi eksepsi karena pembagian dengan nol");
        } catch(ArrayIndexOutOfBoundsException e) {
            System.out.println("Setelah blok try catch");
        }
    }
}
```

Output program di atas tergantung jumlah parameter yang diberikan. Jika tanpa parameter, maka output-nya :

```
C:\Java_projects> java DemoEksepsi
Terjadi eksepsi karena pembagian dengan nol
Setelah blok try catch
```

Jika dieksekusi dengan sebuah parameter maka *output*-nya:

```
C:\Java_projects> java DemoEksepsi param1
Tidak terjadi eksepsi
Setelah blok try catch
```

Jika dieksekusi dengan dua buah parameter maka *output*-nya:

```
C:\Java_projects> java DemoEksepsi
Terjadi eksepsi karena indeks di luar kapasitas array
Setelah blok try catch
```

10.5 Penggunaan Throws

Penggunaan *keyword* ini berhubungan erat dengan penggunaan *exception* yang dicek oleh Java. Setiap *method* yang mungkin menyebabkan suatu *exception* dan tidak menangani *exception* tsb, dalam arti *exception* tsb akan dilempar ke luar, maka *method* tersebut harus menjelaskan kemungkinan ini agar si pemanggil *method* ini dapat mengetahui dan bersiap-siap untuk menangani *exception* yg mungkin terjadi. Ini dilakukan dengan cara menggunakan *keyword throws* pada saat pendeklarasian *method*.

Apabila Kita tidak menyertakan blok *try-catch* di dalam *method* yang mengandung kode-kode yang mungkin menimbulkan eksepsi, maka Kita harus menyertakan klausa *throws* pada saat pendeklarasian *method* yang bersangkutan. Jika tidak, maka program tidak dapat dikompilasi.

Contoh program:

```
class Coba {

    public void tampil() throws Exception {
        int x=0;
        if (x<5)
            throw new Exception("Lebih kecil 5");
    }
}

public class DemoException {

    public static void main (String args[]) {
        Coba c = new Coba();
        try {
            c.tampil();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
        System.out.println("Program Selesai");
    }
}
```

Output program :

```
Lebih kecil 5
Program Selesai
```

10.6 Pemakaian Finally

Penggunaan blok *try catch* terkadang membingungkan karena Kita tidak dapat menentukan dengan pasti alur mana yang akan dieksekusi. Apalagi penggunaan *throw* yang mengakibatkan kode setelah *throw* tidak akan dieksekusi atau justru terjadi kesalahan pada blok *catch*, menyebabkan program akan berhenti. Untuk mengatasi problem ini, Java memperkenalkan *keyword finally*. Dimana semua kode yang ada dalam blok *finally* “pasti” akan dieksekusi apapun yang terjadi di dalam blok *try catch*.

Contoh:

```
public class DemoFinally{
    public static void main(String[] args){
        int x = 3;
        int[] arr = {10,11,12};
        try {
            System.out.println(arr[x]);
            System.out.println("Tidak Terjadi Eksepsi");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Terjadi Eksepsi");
            System.out.println(arr[x-4]);
        } finally {
            System.out.println("Program Selesai");
        }
    }
}
```

Output program

```
Terjadi Eksepsi
Program Selesai
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: -1
    at DemoFinally.main(coba2.java:13)
```



Modul 11 Web Frontend

Tujuan Praktikum

1. Mahasiswa mampu memahami konsep dan implementasi pada web.
2. Mahasiswa mampu memahami sintaks dan elemen-elemen HTML.
3. Mahasiswa mampu memahami konsep dan implementasi CSS pada web.
4. Mahasiswa mampu mengenali jenis-jenis dan penggunaan CSS.
5. Mengenal komponen-komponen dasar dalam pembuatan *server pages* dalam Java.

11.1 HTML

11.1.1 Pengenalan HTML

HTML atau *HyperText Markup Language* merupakan bahasa dasar yang digunakan untuk membangun sebuah *web* dimana HTML menangani elemen-elemen dasar pada pembangunan sebuah *website*. Struktur HTML paling dasar adalah sebagai berikut:



Gambar 11-1 Struktur dasar HTML

11.1.1.1 Tag HTML

Tag dalam HTML secara normal memiliki sepasang *tag* di mana *tag* pertama merupakan *tag* pembuka dan yang kedua merupakan *tag* penutup. Konten yang ingin ditampilkan pada laman *web* diletakkan di antara kedua *tag* tersebut.

`<nama_tag>` letakkan konten di sini ... `</nama_tag>`

Tag dalam HTML tidak semuanya berbentuk pasangan, ada beberapa *tag* yang hanya berdiri sendiri seperti *tag* "`<br/>`" yang berguna untuk berpindah baris.

11.1.1.2 Elemen HTML

Elemen HTML merupakan *tag* HTML yang telah memiliki konten atau isi di antara kedua *tag* pembuka dan penutupnya. Elemen HTML dapat berupa teks atau juga dapat menyisipkan *tag* HTML lain pada elemen tersebut.

```

<!DOCTYPE html>
<html>

<head>
  <!-- Contoh elemen berisi tag lain -->
  <title>Page Title</title>
</head> asdff

<body>
  <h1>My First Heading</h1>
  <!-- Contoh Elemen berisi Teks -->
  <p>My first paragraph.</p>
</body>

</html>

```

11.1.1.3 Atribut HTML

Atribut HTML merupakan tambahan informasi dari sebuah *tag* HTML. Bentuk atribut untuk setiap *tag* HTML berbeda-beda sehingga kegunaan atribut juga berbeda seperti menambahkan informasi warna elemen, ukuran lebar, ukuran panjang dan lain-lain. Namun, mayoritas atribut yang sering muncul untuk setiap *tag* HTML adalah atribut “id” dan “class” karena kedua atribut ini berperan besar dalam pengembangan laman *web* dengan CSS dan JavaScript. Atribut HTML dideklarasikan di dalam *tag* pembuka pada setiap elemen HTML dengan format **nama_atribut="value"**, setiap nilai atribut diapit oleh petik dua.

```

<a href="www.google.co.id" > Google.co.id </a>
<input type="button" id="btnSubmit" class="btnSubmit1" value="Kirim"/>

```

11.1.2 Dasar Sintaks HTML

```

<!DOCTYPE html>
<html>

<head>
  <title>Page Title</title>
</head>

<body>
  <h1>My First Heading</h1>
  <p>My first paragraph.</p>
</body>

</html>

```

Seperti yang sudah dijelaskan sebelumnya struktur dasar HTML antara lain berupa:

- Deklarasi <! DOCTYPE html> mendefinisikan dokumen menjadi HTML5
- Elemen <html> adalah elemen dasar dari halaman HTML
- Elemen <head> berisi informasi meta tentang dokumen
- Elemen <title> menentukan judul untuk dokumen
- Elemen <body> berisi konten halaman yang terlihat

11.1.3 Heading

Heading pada HTML merupakan *tag* yang berguna untuk menampilkan judul dari konten laman *web* yang dibangun. *Heading* dalam sebuah laman *web* berperan penting untuk aplikasi mesin pencarian karena sistem mesin pencarian bekerja dengan menggunakan *Heading* laman *web* kita sebagai *index* pencarian. Dalam HTML terdapat enam tingkatan *Heading* di mana semakin kecil nilai *heading* nya maka semakin penting dan semakin besar ukurannya pada laman *web*.

```
<h1>Heading 1</h1>
<h2>Heading 2</h2>
<h3>Heading 3</h3>
<h4>Heading 4</h4>
<h5>Heading 5</h5>
<h6>Heading 6</h6>
```



Gambar 11-2 Tingkatan heading

11.1.4 Hyperlink

Hyperlink dalam HTML memungkinkan halaman *web* berpindah laman atau bernavigasi menuju laman *web* yang lain. *Tag* yang digunakan adalah *tag* `<a>...</a>`

```
<a href="www.google.co.id"> Visit Google </a>
```

Output :
Visit Google

Jika panah kursor kita arahkan pada teks tersebut, kursor akan berubah menjadi bentuk tangan.

Dalam *tag Hyperlink* pada HTML ada satu atribut yang harus digunakan agar konten yang ada di antara *tag hyperlink* berjalan dan dapat melakukan navigasi menuju laman *web* lain yaitu atribut **href**. Atribut ini bernilai *url* atau alamat dari laman *web* tujuan.

11.1.5 Tabel

Tabel pada HTML merupakan salah satu elemen penting khususnya digunakan untuk menampilkan data yang membutuhkan bentuk tabel. Tabel pada HTML didefinisikan dengan tag `<table></table>` dengan setiap pendefinisian baris menggunakan tag `<tr></tr>`, pendefinisian *heading* tabel menggunakan tag `<th></th>` dan pendefinisian kolom menggunakan tag `<td></td>`.

```
<table width="80%" height="50%" border="1">
  <tr>
    <th>Nama Lengkap</th>
    <th>Kota Kelahiran</th>
    <th>Age</th>
  </tr>
  <tr>
    <td>Budi</td>
    <td>Jakarta</td>
    <td>35</td>
  </tr>
  <tr>
    <td>Andi</td>
    <td>Semarang</td>
    <td>52</td>
  </tr>
  <tr>
    <td>Rasyid</td>
    <td>Surabaya</td>
    <td>22</td>
  </tr>
</table>
```



Nama Lengkap	Kota Kelahiran	Age
Budi	Jakarta	35
Andi	Semarang	52
Rasyid	Surabaya	22

Gambar 11-3 Tabel pada HTML

Dalam tabel HTML kita dapat melakukan operasi *Merge Cell* yang biasanya dapat dilakukan pada aplikasi perkantoran seperti Microsoft Word atau Excel dengan cara menambahkan atribut **colspan** dan **rowspan** pada tag pembuka kolom yaitu `<td>` nilai dari atribut tersebut berupa jumlah kolom atau baris yang akan digabungkan.

```

<table width="80%" height="50%" border="1">
  <tr>
    <th rowspan="2">Nama Lengkap</th>
    <th colspan="2">Gelar Pendidikan</th>
    <th rowspan="2">Age</th>
  </tr>
  <tr>
    <th> Sarjana </th>
    <th> Magister </th>
  </tr>
  <tr>
    <td>Budi</td>
    <td>S.Kom</td>
    <td>M.Sc</td>
    <td>35</td>
  </tr>
  <tr>
    <td>Andi</td>
    <td>S.SiKom</td>
    <td>M.T</td>
    <td>52</td>
  </tr>
</table>

```

Nama Lengkap	Gelar Pendidikan		Age
	Sarjana	Magister	
Budi	S.Kom	M.Sc	35
Andi	S.SiKom	M.T	52

Gambar 11-4 Penggunaan colspan dan rowspan pada table HTML

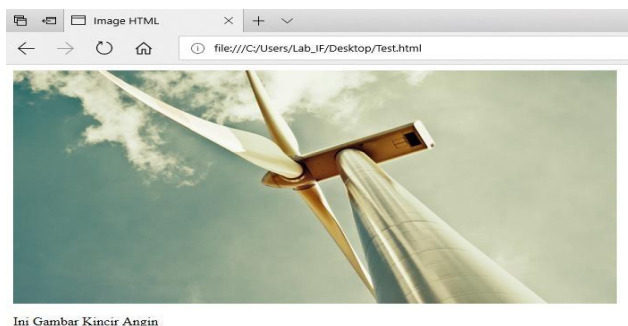
11.1.6 Image

Menampilkan gambar pada halaman *web* merupakan sebuah improvisasi dalam pembuatan desain sebuah *web* yang dapat memperindah tampilan *website*. *Tag* HTML yang digunakan adalah `<img/>` *tag* ini tidak memiliki pasangan penutup maka dari itu diakhir *tag* pembuka ditambahkan garis miring seperti di atas. Terdapat satu atribut wajib yang harus ditambahkan seperti atribut **href** pada *tag* *Hyperlink* yaitu atribut **src** yang bernilai alamat direktori gambar disimpan.

```


<p>Ini Gambar Kincir Angin</p>

```



Gambar 11-5 Penggunaan image pada HTML

11.1.7 Audio / Video Elemen

Sebelum berkembangnya teknologi HTML5, untuk menyisipkan audio atau video, diperlukan sebuah *plugin* seperti *Flash Player* namun sekarang dengan HTML5 memiliki *tag* yang dapat menyisipkan audio atau video ke dalam laman *web*. Untuk audio menggunakan *tag* <audio> untuk *tag* pembuka dan <source> untuk memanggil *url* atau alamat direktori *file*. Sedangkan untuk video menggunakan *tag* <video>.

```
<audio controls>
  <source src="horse.ogg" type="audio/ogg">
  <source src="horse.mp3" type="audio/mpeg"> Your browser does not support the
  audio element.
</audio>

<video width="400" controls>
  <source src="mov_bbb.mp4" type="video/mp4">
  <source src="mov_bbb.ogg" type="video/ogg"> Your browser does not support
  HTML5 video.
</video>

<p>
  Video courtesy of
  <a href="https://www.bigbuckbunny.org/" target="_blank">Big Buck Bunny</a>.
</p>
```



Gambar 11-6 Penggunaan audio dan video pada HTML

11.1.8 Form

Form pada HTML digunakan sebagai wadah untuk menampung dan mengumpulkan data-data dari pengguna jika diperlukan untuk disimpan dalam sebuah *database*. *Tag* dasar untuk pemanggilan *form* adalah <form> ... </form> dan diantara *tag form* tersebut merupakan tempat mendefinisikan elemenelemen yang dibutuhkan *form* yang akan dibuat nantinya. Atribut utama dari *tag form* yaitu **action**, atribut ini bernilai tujuan data akan diolah dengan bahasa pemrograman *web* saat tombol "Submit" ditekan, selain itu atribut **method** yang hanya bernilai POST atau GET ini juga sangat dibutuhkan untuk pengolahan data dengan bahasa pemrograman *web*. Pembahasan lebih lanjut ada pada modul PHP.

11.1.8.1 Elemen Form

Elemen-elemen form pada html adalah sebagai berikut:

Tag	Type	Fungsionalitas
<input/>	type = "text"	Menampilkan elemen untuk input data teks
	type = "password"	Menampilkan elemen untuk input data password
	type = "email"	Menampilkan elemen untuk input data email
	type = "radio"	Menampilkan elemen untuk pemilihan data berbentuk radio
	type = "checkbox"	Menampilkan elemen untuk pemilihan data berbentuk <i>checkbox</i>
	type = "submit"	Menampilkan elemen tombol untuk pengolahan data <i>form</i>
<select> <option> ... </option> </select>	-	Menampilkan elemen untuk pemilihan data berbentuk <i>dropdown list</i>
<textarea> ... </textarea>	-	Menampilkan elemen untuk input data dalam bentuk paragraf panjang.
<button> ... </button>	type = "button"	

11.1.8.2 Atribut Elemen Form

Atribut-atribut elemen *form* yang sering digunakan antara lain sebagai berikut:

Tabel 11-1 Atribut elemen form

Atribut	Value	Fungsionalitas
id	Bebas	Menambahkan informasi ID dari elemen untuk kebutuhan CSS, JavaScript atau Bahasa Pemrograman Web yang lain.

name	Bebas	Menambahkan informasi Name dari elemen untuk kebutuhan JavaScript atau Bahasa Pemrograman Web yang lain.
class	Bebas	Menambahkan informasi Class dari elemen untuk kebutuhan CSS, JavaScript atau Bahasa Pemrograman Web yang lain.
placeholder	Bebas	Menampilkan informasi sementara tentang elemen sebelum memberikan inputan pada elemen.
value	Bebas	Memberikan nilai <i>default</i> pada elemen khususnya elemen type dan option
disabled	-	Membuat elemen menjadi tidak dapat dioperasikan
readonly	-	Membuat elemen type tidak dapat dirubah atau diberi inputan
checked	-	Membuat elemen <i>checkbox</i> atau radio button menjadi terpilih secara <i>default</i> .
selected	-	Membuat elemen option pada <i>tag select</i> menjadi terpilih secara <i>default</i> .

```

<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Formulir Pendaftaran Praktikan</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <h1>Formulir Pendaftaran Praktikan</h1>
  <form action="#" method="POST">
    <div class="form-group">
      <label for="nama_id">Nama:</label>
      <input type="text" name="nama_input" id="nama_id" placeholder="Input Nama"
value="Praktikan" readonly>
    </div>
    <div class="form-group">
      <label for="uname_id">Username:</label>
      <input type="text" name="uname_input" id="uname_id" placeholder="Input
Username" value="Praktikum" disabled>
    </div>
    <div class="form-group">
      <label for="password_id">Password:</label>
      <input type="password" name="password_input" id="password_id"
placeholder="Input Password">

```



```

</div>
<div class="form-group">
  <label for="email_id">Email:</label>
  <input type="email" name="email_input" id="email_id" placeholder="Input
Email">
</div>
<fieldset>
  <legend>Jenis Kelamin</legend>
  <div>
    <input type="radio" name="jk_input" id="pria" value="Pria">
    <label for="pria">Pria</label>
  </div>
  <div>
    <input type="radio" name="jk_input" id="wanita" value="Wanita">
    <label for="wanita">Wanita</label>
  </div>
</fieldset>
<fieldset>
  <legend>Hobi</legend>
  <div>
    <input type="checkbox" name="hobi_input" id="renang" value="Renang">
    <label for="renang">Renang</label>
  </div>
  <div>
    <input type="checkbox" name="hobi_input" id="bersepeda"
value="Bersepeda">
    <label for="bersepeda">Bersepeda</label>
  </div>
  <div>
    <input type="checkbox" name="hobi_input" id="memancing"
value="Memancing">
    <label for="memancing">Memancing</label>
  </div>
</fieldset>
<div class="form-group">
  <label for="jp_id">Jenjang Pendidikan:</label>
  <select name="jp_input" id="jp_id">
    <option value="" selected>-----Pilih-----</option>
    <option value="D3">Tamat D3</option>
    <option value="S1">Tamat S1</option>
    <option value="S2">Tamat S2</option>
    <option value="S3">Tamat S3</option>
  </select>
</div>
<div class="form-group">
  <label for="kritik_saran">Kritik & Saran:</label>
  <textarea id="kritik_saran" name="kritik_saran" rows="5"></textarea>
</div>
<div class="form-actions">
  <button type="button" class="btn-cancel">Cancel</button>
  <button type="submit" class="btn-submit">Submit</button>
</div>
</form>
</body>
</html>

```

Formulir Pendaftaran Praktikan

Nama :

Username :

Password :

Email :

Jenis Kelamin : ☐ Pria ☒ Wanita

Hobi : ☒ Renang
☐ Bersepeda
☒ Memancing

Jenjang Pendidikan :

Pilih

Pilih

Tamat D3

Tamat S1

Tamat S2

Tamat S3

Kritik & Saran :

Gambar 11-7 Contoh form pendaftaran

11.2 CSS

11.2.1 Pengenalan CSS

Cascading Style Sheets (CSS) merupakan bahasa yang membantu memperindah tampilan dari laman *web* yang telah dibangun dengan HTML. CSS mendeskripsikan bagaimana bentuk tampilan elemen HTML seharusnya saat ditampilkan pada laman *browser*. Format penulisan CSS secara umum ditunjukkan pada gambar berikut.



Gambar 11-8 Penulisan CSS

Selector merupakan elemen HTML yang akan ditambahkan CSS kemudian diikuti dengan *declaration block* yang terdiri dari *property* elemen yang akan dirubah beserta *value* dari *property*-nya. Setiap deklarasi *selector* dapat merubah banyak nilai *property* sekaligus dengan dipisahkan dengan titik koma dan untuk semua *declaration block* dari satu *selector* berada di antara kurung kurawal.

11.2.1.1 Cara Menyisipkan CSS

Terdapat tiga cara untuk menyisipkan atau mendefinisikan CSS ke dalam HTML, antara lain:

a) External Style Sheet

Eksternal Style Sheet merupakan cara menyisipkan atau mendefinisikan CSS ke dalam HTML dengan memanggil file dengan ekstensi **.css** ke dalam *file* HTML. Pemanggilannya diletakkan di antara elemen `<head></head>` dengan menggunakan *tag* `<link/>`.

```
<head>
  <link rel="stylesheet" type="text/css" href="myStyleSheet.css">
</head>
```

b) Internal Style sheet

Internal Style Sheet merupakan cara menyisipkan atau mendefinisikan CSS ke dalam HTML dengan menggunakan *tag* `<style>` `</style>` pada elemen `<head>``</head>`. Biasanya digunakan ketika satu laman membutuhkan *style* CSS yang berbeda dari yang telah dipanggil pada *Eksternal Style Sheet*.

```
<head>
  <style>
    body {
      background-color: blue;
    }

    h1 {
      color: maroon;
      margin-left: 40px;
    }
  </style>
</head>
```

c) Inline Style

Inline Style menyisipkan atau mendefinisikan CSS ke dalam HTML dengan menambahkan atribut **style** pada elemen yang ingin ditambahkan CSS. Biasanya digunakan hanya untuk satu elemen yang membutuhkan *style* CSS yang berbeda dari yang telah didefinisikan pada *Internal Style* atau *Eksternal Style*.

```
<h1 style="color:lightblue; font-size:30px;">Praktikum Web
Programming</h1>
```

11.2.1.2 Selector

Selector pada CSS digunakan untuk menemukan elemen HTML untuk diberi CSS berdasarkan *selector* yang didefinisikan. Bentuk *selector* ada beberapa antara lain nama elemen HTML, atribut ID dan atribut Class.

```
/*Selector dengan Elemen
HTML*/
p {
  text-align: center;
  color: red;
}

/*Selector dengan Id Elemen
HTML*/
#para1 {
  text-align: center;
  color: red;
}

/*Selector dengan Class Elemen
HTML*/
p.center {
  text-align: center;
  color: red;
}
```

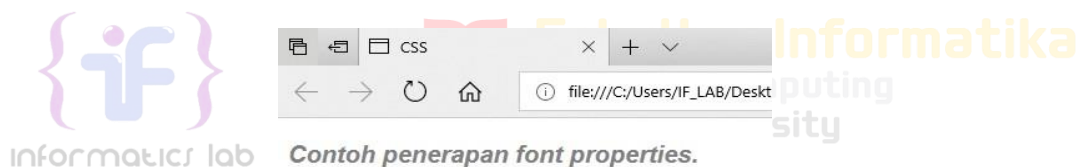
11.2.2 Font Propertis

Sebuah laman *web* tentunya tidak lepas oleh penggunaan teks, oleh karena itu memiliki tampilan teks yang tepat sangat diperlukan agar sebuah *web* memiliki tampilan yang baik dan menarik. CSS dapat menangani kebutuhan tampilan teks dengan *font properties*.

Font Properties	Keterangan
Font-family	Menentukan jenis font yang digunakan
Font-size	Mengatur ukuran font
Font-style	Mengatur style font (normal, italic, oblique)
Font-weight	Mengatur style font (normal atau bold)

Contoh penerapannya sebagai berikut:

```
p.example {  
  font-family: Arial;  
  font-size: 20px;  
  color: light;  
  font-style: italic;  
  font-weight: bold;  
}
```



Gambar 11-9 Penerapan font properties

11.2.3 List Properties

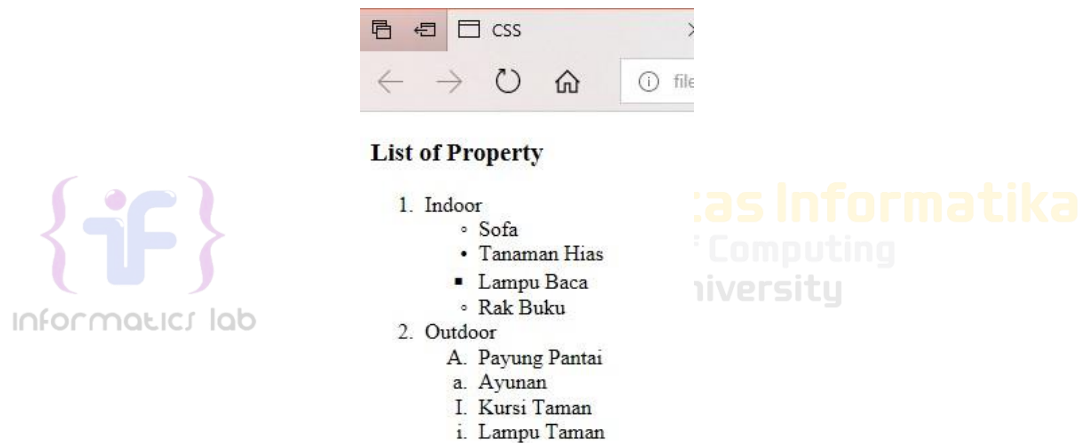
Dalam HTML terdapat elemen yang berguna membuat sebuah *list* menggunakan simbol dan karakter. *Tag* yang digunakan adalah *tag* `<ul></ul>` atau `<ol></ol>`. *Tag* `<ul>` digunakan ketika akan menggunakan *list* dengan penanda berupa simbol atau bisa dikatakan *unordered list*, sedangkan *tag* `<ol>` digunakan ketika akan menggunakan *list* dengan penanda karakter yang memiliki urutan atau bisa dikatakan *ordered list*. Namun di dalam *tag* tersebut juga harus didefinisikan tag pendukung yaitu `<li></li>` untuk mendefinisikan elemen-elemen *list* yang akan ditampilkan. Untuk setiap *tag ordered list* atau *unordered list* memiliki satu atribut untuk mendefinisikan tipe simbol atau karakter yang akan digunakan yaitu atribut ***type***. Contoh penerapan dan tipe masing-masing *tag* sebagai berikut:

```
<h3>List of Property</h3>  
<ol type="1">  
  <li>Indoor  
    <ul type="circle">  
      <li>Sofa</li>  
    </ul>  
    <ul type="disc">  
      <li>Tanaman Hias</li>  
    </ul>  
    <ul type="square">
```

```

        <li>Lampu Baca</li>
    </ul>
    <ul type="none">
        <li>Rak Buku</li>
    </ul>
</li>
<li>Outdoor
    <ol type="A">
        <li>Payung Pantai</li>
    </ol>
    <ol type="a">
        <li>Ayunan</li>
    </ol>
    <ol type="I">
        <li>Kursi Taman</li>
    </ol>
    <ol type="i">
        <li>Lampu Taman</li>
    </ol>
</li>
</ol>

```



Gambar 11-10 Tampilan penggunaan list properties

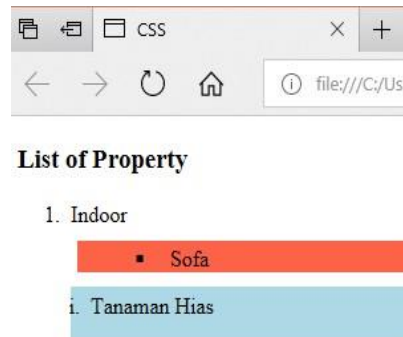
Dengan ditambahkan CSS pada elemen *list*, maka *list* yang ditampilkan dapat lebih menarik, berikut CSS properties untuk elemen *list*.

Lists Specified Properties	Keterangan
List-style-image	Membuat sebuah gambar menjadi penanda <i>list</i>
List-style-position	Mengatur posisi penanda list di dalam konten atau di luar konten
List-style-type	Mengatur jenis penanda <i>list</i>

Lists General Properties	Keterangan
Background-color	Mengatur warna latar belakang elemen <i>list</i>

Padding	Mengatur ruang jarak elemen konten dengan pembatas pada bagian dalam
Margin	Mengatur ruang jarak elemen konten dengan pembatas pada bagian luar

Contoh penerapannya sebagai berikut:



Gambar 11-11 List properties dengan tambahan CSS

```

U1.firstList {
    background-color: tomato;
    margin: 10px 5px 10px 5px;
    list-style-type: lower-alpha;
    list-style-position: inside;
}

ol.secondList {
    background-color: lightblue;
    list-style-type: lower-roman;
    padding: 5px 5px 15px 15px;
    list-style-position: inside;
}

```

11.2.4 Alignment of Text

Pengaturan *alignment* pada sebuah teks juga dapat ditangani oleh CSS dengan *properties* sebagai berikut:

Properties	Value	Keterangan
Text-align	Center	Membuat teks menjadi rata tengah

	Left	Membuat teks menjadi rata kiri
	Right	Membuat teks menjadi rata kanan
	Justify	Membuat paragraf menjadi rata kanan dan kiri

Contoh penerapannya pada gambar 11-12:



Gambar 11-12 Penerapan alignment of text

```
h1 {
    text-align: center;
}
h2 {
    text-align: left;
}
h3 {
    text-align: right;
}
```

11.2.5 Colors

Jika berbicara desain antar muka *web*, permasalahan tentang warna merupakan salah satu hal yang penting. Pada dasarnya Tag HTML dapat menangani pengaturan warna latar belakang atau teks menggunakan atribut dari HTML sendiri, namun CSS dapat menangani lebih baik dengan menawarkan pengaturan yang lebih lengkap.

Properties	Keterangan	Value	Color Names (Red, Green, Orange dll.)
Background-color	Mengatur warna latar belakang elemen HTML		RGB Value (R, G, B)
			Hex Value (#FFFF00)
			HSL Value (Hue, Saturation, Light)
			RGBA (dengan <i>Opacity</i>)
Color	Mengatur warna teks elemen HTML		HSLA (dengan <i>Opacity</i>)

Contoh penerapannya sebagai berikut :

```
body{
    background-color: HSL(20%, 40%, 70%);
    color: orange;
}

#teks{
    color: #2F3CDF;
}
```

```

/*dengan opacity sebesar 0.5*/
input.text-field{
    background-color: RGBA(32, 55, 122, 0.5);
}

```

11.2.6 Span & Div

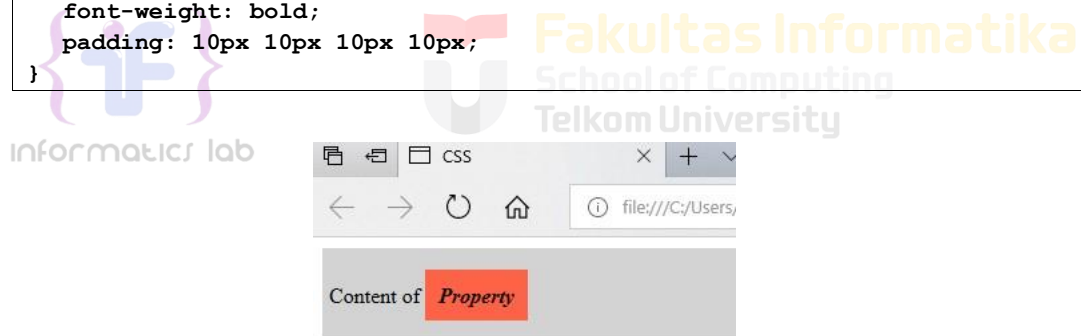
Span merupakan elemen HTML yang dapat menangani perubahan konten elemen pada satu baris. *Tag* yang digunakan adalah `<span></span>`. Sedangkan *Div* merupakan elemen HTML yang digunakan untuk membuat *section* untuk beberapa elemen HTML di dalamnya. Tag yang digunakan yaitu `<div></div>`.

```

<div class="section1">
    <p> Content of <span class="mark"> Property </span> </p>
</div>

/*CSS Properties*/
.section1 {
    background-color: lightgrey;
    padding: 10px 5px 10px 5px;
}
.mark {
    background-color: tomato;
    font-style: italic;
    font-weight: bold;
    padding: 10px 10px 10px 10px;
}

```



Gambar 11-13 Penerapan Span & Div

11.3 Bootstrap

Bootstrap merupakan sebuah *front-end framework* gratis untuk pengembangan antar muka *web* yang lebih cepat dan lebih mudah. Dikembangkan oleh Mark Otto dan Jacom Thornton di Twitter dan dirilis sebagai produk *open source* pada Agustus 2011 di GitHub. Bootstrap mencakup *template* desain berbasis HTML dan CSS untuk tipografi, *form*, *button*, navigasi, *modal*, *image carousells* dan masih banyak lagi, serta terdapat opsional *plugin* JavaScript. Selain itu, Bootstrap memiliki kemampuan untuk membuat desain responsif yang secara otomatis menyesuaikan diri agar terlihat baik di segala perangkat, mulai dari perangkat ponsel hingga *desktop pc*.

11.3.1 Pemasangan Bootstrap

Bootstrap merupakan produk yang mengusung konsep *open source* sehingga untuk pemasangannya dapat dilakukan dengan beberapa cara sebagai berikut:

1. Unduh di <http://getbootstrap.com>, selanjutnya pasang pada *project web* kalian seperti memanggil *External Style Sheet* pada CSS.
2. Memanggil Bootstrap CDN (*Content Delivery Network*), sehingga kita tidak perlu mengunduh dan memasangnya pada laman *website*, hanya memanggil *source* dari Bootstrap. Cara ini membutuhkan koneksi internet untuk menghasilkan perubahan tampilan CSS.

```
<!-- Pemanggilan Bootstrap dengan CDN -->

<!-- CSS -->
<link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
9ndCyUaIbzAi2FUVXJi0CjmCapSmO7SnpJef0486qhLnuZ2cdeRhO02iuK6FUUVM"
crossorigin="anonymous">

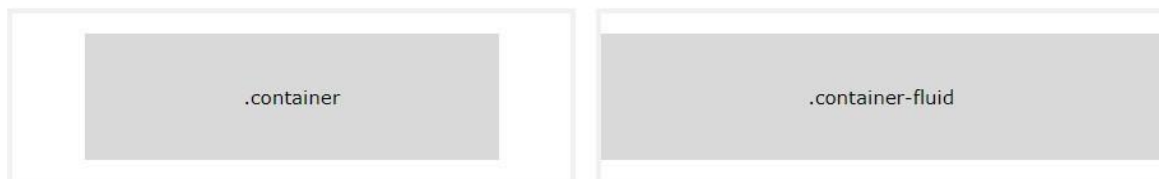
<!-- jQuery library -->
<script src="https://code.jquery.com/jquery-3.7.0.min.js" integrity="sha256-
2PmVv0kuTBOenSvLm6bvfbSSHrUJ+3A7x6P5Ebd07/g=" crossorigin="anonymous"></script>

<!-- JavaScript -->
<script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/js/bootstrap.bundle.min.js"
integrity="sha384-
geWF76RCwLtnZ8qwWowPQNguL3RmwHVBC9FhGdlKrxdiJJigb/j/68SIy3Te4Bkz"
crossorigin="anonymous"></script>
```

11.3.2 Bootstrap Container

Bootstrap *container* adalah elemen paling dasar yang dibutuhkan dalam *layouting* menggunakan Bootstrap *Grid*. *Container* berbentuk class CSS yang sisipkan pada elemen HTML <div>. Pada gambar 3-7 terdapat dua *class container* pada Bootstrap yang dapat dipilih yaitu:

1. Class **.container** menyediakan *container* yang *responsive* dengan lebar yang tetap.
2. Class **.container-fluid** menyediakan *container* dengan lebar yang penuh mencakup seluruh area pandang.



Gambar 11-14 Class container

11.3.3 Bootstrap Grid

Sistem *grid* pada Bootstrap menggunakan rangkaian *container*, *rows* dan *column* untuk tata letak dan keselarasan elemen atau konten. Dibangun dengan *flexbox* dan sangat responsif terhadap perangkat yang digunakan untuk menampilkan laman *web*. Struktur dasar *grid* pada Bootstrap sebagai berikut.

```
<div class="row">
  <div class="col-*-*#"></div>
  <div class="col-*-*#"></div>
</div>
<div class="row">
  <div class="col-*-*#"></div>
  <div class="col-*-*#"></div>
  <div class="col-*-*#"></div>
```

</div>

Pertama diawali dengan <div class="container">. Kemudian buat sebuah baris sebelum mendeklarasikan sebuah kolom dengan menggunakan <div class="row">. Terakhir buat elemen div dengan mendefinisikan class "col-*-#". Tanda * dan # mewakili jenis dan ukuran *column* yang akan digunakan, *value* yang dapat didefinisikan dapat dilihat pada tabel berikut:

Tabel 11-2 Sistem grid Bootstrap

	Extra Small	Small	Medium	Large	Extra Large
Max container width	None (auto)	540px	720px	960px	1140px
Class prefix	.col-	.col-sm-	.col-md-	.col-lg-	.col-xl-
# per columns	12				
Nestable	Yes				
Column Ordering	Yes				

Contoh penerapannya sebagai berikut:

```
<div class="container">
<div class="row">
  <div class="col-12 col-md-8">.col-12 .col-md-8</div>
  <div class="col-6 col-md-4">.col-6 .col-md-4</div>
</div>
<div class="row">
  <div class="col-6 col-md-4">.col-6 .col-md-4</div>
  <div class="col-6 col-md-4">.col-6 .col-md-4</div>
  <div class="col-6 col-md-4">.col-6 .col-md-4</div>
</div>
<div class="row">
  <div class="col-6">.col-6</div>
  <div class="col-6">.col-6</div>
</div>
</div>
```

.col-12 .col-md-8		.col-6 .col-md-4	
.col-6 .col-md-4		.col-6 .col-md-4	
.col-6		.col-6	

Gambar 11-15 Hasil penerapan Bootstrap grid

11.3.4 Text Style

Bootstrap menyediakan banyak *class* untuk mengatur *style* sebuah teks elemen HTML. Beberapa contohnya antara lain :

Tabel 11-3 Text style

Class	Keterangan
.text-left	Mengatur teks menjadi rata kiri dalam sebuah elemen.
.text-center	Mengatur teks menjadi rata tengah dalam sebuah elemen.

.text-right	Mengatur teks menjadi rata kanan dalam sebuah elemen.
.text-lowercase	Mengatur seluruh teks pada elemen menjadi huruf kecil
.text-uppercase	Mengatur seluruh teks pada elemen menjadi huruf besar
.text-capitalize	Menjadikan huruf pertama besar untuk setiap kata pada sebuah elemen.
.font-weight-bold	Mengatur ketebalan huruf menjadi <i>bold</i>
.font-weight-light	Mengatur ketebalan huruf menjadi <i>light</i>
.font-weight-normal	Mengatur ketebalan huruf menjadi normal
.font-italic	Mengatur gaya teks menjadi miring
.h1 s.d .h6	Mengatur seluruh teks pada sebuah elemen sehingga memiliki tampilan selengkapnya tag elemen H1 s.d H6 pada HTML.

11.3.5 Table, Image, & Button

Bootstrap menyediakan *class* untuk pengaturan *style* elemen tabel, gambar dan tombol menjadi lebih menarik.

1. Bootstrap Table

Tabel pada Bootstrap dipanggil dengan *class* **.table** secara *default*, namun ada beberapa *class* tambahan yang dapat didefinisikan pada elemen tabel yang lain berikut dapat dilihat pada Tabel berikut:

Tabel 11-4 Elemen tabel

Class	Keterangan
.table-dark	Membuat tampilan tabel memiliki latar belakang gelap
.thead-light	Membuat elemen <thead> pada tabel memiliki latar belakang cerah
.thead-dark	Membuat elemen <thead> pada tabel memiliki latar belakang gelap
.table-striped	Membuat tampilan tabel memiliki latar belakang setiap row yang berbeda
.table-bordered	Membuat tampilan tabel sederhana dengan border tipis
.table-hover	Membuat tampilan tabel yang akan berubah warna latar belakang row saat didekati cursor.
.table-sm	Membuat tampilan tabel sederhana yang dengan ukuran <i>padding</i> yang minim

Contoh penerapannya sebagai berikut:

```

<!--Tabel Hover Style -->
<table class="table table-hover">
  <thead>
    <tr>
      <th scope="col">#</th>
      <th scope="col">Nama Lengkap</th>
      <th scope="col">Asal Kota</th>
      <th scope="col">Umur</th>
    </tr>
  </thead>

```

```

<tbody>
  <tr>
    <th scope="row">1</th>
    <td>Budi Rojadi</td>
    <td>Semarang</td>
    <td>35 th</td>
  </tr>
  <tr>
    <th scope="row">2</th>
    <td>Yulia Santi</td>
    <td>Bekasi</td>
    <td>32</td>
  </tr>
  <tr>
    <th scope="row">3</th>
    <td>Fahri Abdilah</td>
    <td>Medan</td>
    <td>38 th</td>
  </tr>
</tbody>
</table>

```

#	Nama Lengkap	Asal Kota	Umur
1	Budi Rojadi	Semarang	35 th
2	Yulia Santi	Bekasi	32
3	Fahri Abdilah	Medan	38 th

Gambar 11-16 Hasil penerapan Class.table-hover

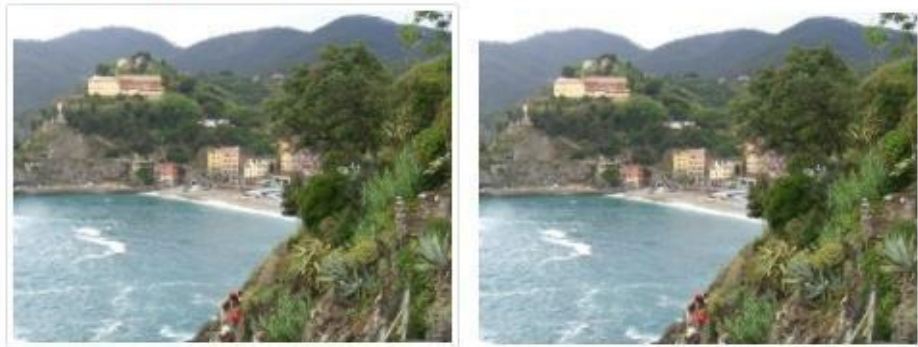
#	Nama Lengkap	Asal Kota	Umur
1	Budi Rojadi	Semarang	35 th
2	Yulia Santi	Bekasi	32

Gambar 11-17 Hasil penerapan Class.thead-dark

2. Bootstrap Image

Bootstrap dapat menangani desain gambar agar *responsif* pada setiap perangkat yang menampilkan laman *web*. Dengan menambahkan *class .img-fluid* pada elemen *tag * pada HTML maka gambar yang didefinisikan pada laman *web* akan memiliki ukuran yang *responsif* menyesuaikan ukuran layar perangkat. *Class* tersebut mengatur ukuran gambar dengan menyesuaikan ukuran dari *parent element* sebagai wadah atau *container* elemen gambar. Terdapat *class .thumbnail* yang berguna menjadikan gambar menjadi berukuran kecil dan sedikit memiliki border disekitarnya dapat dilihat pada gambar berikut.

Thumbnail & Fluid



Gambar 11-18 Image class.thumbnail dan class.fluid

```
<div class="container">
  <h2>Thumbnail & Fluid</h2>
  
  
</div>
```

3. Bootstrap Button

Tampilan *button* pada elemen HTML dapat dirubah dengan menambahkan beberapa *class* untuk *button* oleh Bootstrap. Bootstrap membuat tampilan *button* menjadi lebih menarik dan memberikan *user experience* yang baik. *Class* yang digunakan secara *default* adalah *.btn* namun dengan disertai *class* lain seperti berikut untuk memberikan perubahan warna dan ukuran *button*:

Tabel 11-5 Class button

Class	Keterangan
.btn-primary	Membuat tampilan button dengan desain utama
.btn-danger	Membuat tampilan button dengan desain berwarna merah
.btn-success	Membuat tampilan button dengan desain berwarna hijau
.btn-warning	Membuat tampilan button dengan desain berwarna kuning
.btn-info	Membuat tampilan button dengan desain sebuah informasi
.btn-link	Membuat tampilan button dengan desain sebuah <i>hyperlink</i>
.btn-xs	Membuat tampilan button berukuran <i>extra small</i>
.btn-sm	Membuat tampilan button berukuran <i>small</i>
.btn-md	Membuat tampilan button berukuran medium
.btn-lg	Membuat tampilan button berukuran besar
.btn-block	Membuat tampilan button menjadi sebuah blok besar yang mengisi seluruh ruang pada <i>parent</i>

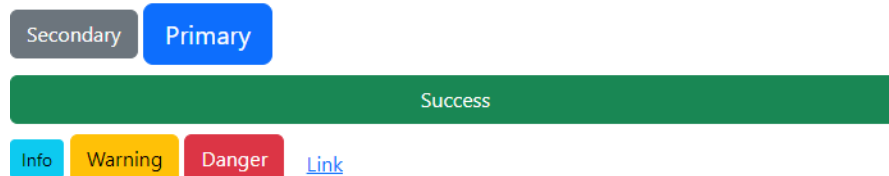
Contoh penerapan *class button*:

```

<div class="container">
  <h4>Button Styles</h4>
  <button type="button" class="btn btn-secondary btn-md">Secondary</button>
  <button type="button" class="btn btn-primary btn-lg">Primary</button>
  <button type="button" class="btn btn-success btn-block">Success</button>
  <button type="button" class="btn btn-info btn-xs">Info</button>
  <button type="button" class="btn btn-warning">Warning</button>
  <button type="button" class="btn btn-danger">Danger</button>
  <button type="button" class="btn btn-link">Link</button>
</div>

```

Button Styles



Gambar 11-19 Hasil penerapan class button

11.3.6 Bootstrap Form

Bootstrap menyediakan perubahan elemen *form* pada HTML baik pada segi tata letak tampilan atau tampilan antarmuka elemen-elemen dalam *form*. Class **.form-group** didefinisikan untuk setiap *parent* atau *container* untuk elemen *form*, sedangkan class **.form-control** didefinisikan untuk setiap elemen dalam *tag* `<form>`. Ada tiga *class* yang dapat dipilih untuk mengatur tata letak tampilan *form* yaitu :

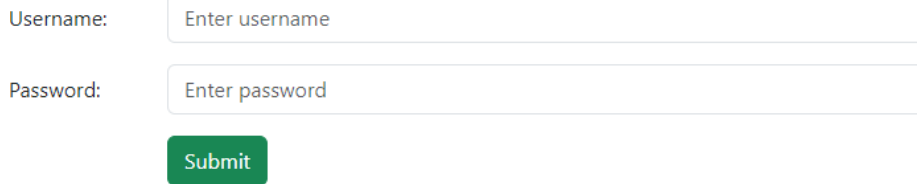
1. **Vertical Form**, ini merupakan tampilan *default* saat *tag form* tidak didefinisikan *class*.
2. **Inline Form**, ini merupakan tampilan *form* dengan class **.form-inline** yang membuat *form* memiliki tampilan seluruh elemen dalam satu garis.
3. **Horizontal Form**, ini merupakan tampilan *form* dengan class **.form-horizontal** yang membuat *form* memiliki tampilan elemen label dengan *input type* dalam satu baris. Pada tampilan ini butuh pendefinisian class **.control-label** untuk setiap elemen `<label>` Contoh :

```

<div class="container">
  <h3>Horizontal form</h3>
  <form class="form-horizontal" action="/action_page.php">
    <div class="form-group">
      <label class="control-label col-sm-2" for="email">Username:</label>
      <div class="col-sm-10">
        <input type="text" class="form-control" id="uname" placeholder="Enter
username" name="uname">
      </div>
    </div>
    <div class="form-group">
      <label class="control-label col-sm-2" for="pwd">Password:</label>
      <div class="col-sm-10">
        <input type="password" class="form-control" id="pwd" placeholder="Enter
password" name="pwd">
      </div>
    </div>
    <div class="form-group">
      <div class="col-sm-offset-2 col-sm-10">
        <button type="submit" class="btn btn-success">Submit</button>
      </div>
    </div>
  </form>
</div>

```

Horizontal form



Username:

Password:

Gambar 11-20 Contoh Bootstrap Form

11.4 Javascript

11.4.1 Pengenalan Javascript

11.4.1.1 Sejarah Singkat Javascript

Javascript, seperti namanya, merupakan bahasa pemrograman *scripting*. Dan seperti bahasa *scripting* lainnya, Javascript umumnya digunakan hanya untuk program yang tidak terlalu besar, biasanya hanya beberapa ratus baris. Javascript pada umumnya mengontrol program yang berbasis Java. Jadi memang pada dasarnya Javascript tidak dirancang untuk digunakan dalam aplikasi skala besar.

Meskipun dibuat dengan tujuan awal untuk mengendalikan program Java, komunitas Javascript menggunakan bahasa ini untuk tujuan lain, memanipulasi gambar dan isi dari dokumen HTML. Singkatnya, pada akhirnya Javascript digunakan untuk satu tujuan utama, “menghidupkan” dokumen HTML dengan mengubah konten statis menjadi dinamis dan interaktif. Bersamaan dengan perkembangan Internet dan dunia *web* yang pesat, Javascript akhirnya menjadi bahasa utama dan satu-satunya untuk membuat HTML menjadi interaktif di dalam browser.

11.4.1.2 Prinsip Dasar Javascript

Prinsip dasar yang terdapat pada bahasa pemrograman javascript adalah sebagai berikut.

1. Javascript mendukung paradigma pemrograman imperatif (Javascript dapat menjalankan perintah program baris demi baris, dengan masing-masing baris berisi satu atau lebih perintah), fungsional (struktur dan elemen-elemen dalam program sebagai fungsi matematis yang tidak memiliki keadaan (*state*) dan data yang dapat berubah (*mutable data*)), dan orientasi objek (segala sesuatu yang terlibat dalam program dapat disebut sebagai “objek”).
2. Javascript memiliki model pemrograman fungsional yang sangat ekspresif.
3. Pemrograman berorientasi objek (PBO) pada Javascript memiliki perbedaan dari PBO pada umumnya.

Program kompleks pada Javascript umumnya dipandang sebagai program-program kecil yang saling berinteraksi.

11.4.2 Sintaks Umum pada Javascript

11.4.2.1 Tipe data dasar

Seperti kebanyakan bahasa pemrograman lainnya, Javascript memiliki beberapa tipe data untuk dimanipulasi. Seluruh nilai yang ada dalam Javascript selalu memiliki tipe data. Tipe data yang dimiliki oleh Javascript adalah sebagai berikut:

- *Number* (bilangan)
- *String* (serangkaian karakter)

- *Boolean* (benar / salah)
- *Object*
- *Function* (fungsi)
- *Array*
- *Date*
- *RegExp* (*regular expression*)
- *Null* (tidak berlaku / kosong)
- *Undefined* (tidak didefinisikan)

Kebanyakan dari tipe data yang disebutkan di atas sama seperti tipe data sejenis pada bahasa pemrograman lainnya. Misalnya, sebuah boolean terdiri dari dua nilai saja, yaitu *true* dan *false*.

11.4.2.2 Variabel

Seperti pada bahasa pemrograman lainnya, variabel dalam Javascript merupakan sebuah tempat untuk menyimpan data sementara. Variabel dibuat dengan kata kunci *var* pada Javascript.

```
var a;           // a berisi undefined
var nama = "Budi"; // nama berisi "Budi"
```

Nilai yang ada di dalam variabel dapat diganti dengan mengisi nilai baru, dan bahkan dapat diganti tipe datanya juga.

```
nama = "Anton"; // nama sekarang berisi string "Anton"
nama = 1;       // nama sekarang berisi integer 1
```

Walaupun kemampuan untuk menggantikan tipe data ini sangat memudahkan kita dalam mengembangkan aplikasi, fitur ini harus **digunakan** dengan sangat hati-hati. Perubahan tipe data yang tidak diperkirakan dengan baik dapat menyebabkan berbagai kesalahan (*error*) pada program, misalnya jika kita mencoba mengakses method **charAt()** (fungsi yang mengembalikan nilai *char* pada indeks sebuah *string*) setelah mengubah tipe pada contoh di atas.

11.4.2.3 Array

Array merupakan sebuah tipe data yang digunakan untuk menampung banyak tipe data lainnya. Berbeda dengan tipe data *object*, *array* pada Javascript merupakan sebuah tipe khusus. Walaupun memiliki *method* dan properti, *array* bukanlah objek, melainkan sebuah tipe yang “mirip objek”. Pembuatan *array* dalam Javascript dilakukan dengan menggunakan kurung siku (*[]*):

```
var data = ["satu", 2, true];
```

Elemen *array* pada Javascript tidak harus memiliki tipe data yang sama seperti contoh diatas. Selain itu Javascript juga mendukung untuk membuat *array* di dalam *array* yang biasa dikenal dengan *array* dua dimensi seperti contoh dibawah ini.

```
var arr2 = [["satu", "dua"], ["tiga", "empat"]];
```

Pengaksesan elemen dalam *array* dilakukan dengan menggunakan kurung siku. Nilai yang kita berikan dalam kurung siku adalah urutan elemen penulisan *array* (indeks), yang dimulai dari nilai 0. Jika indeks yang diakses tidak ada, maka kita akan mendapatkan nilai *undefined*.

```
data[2];      // mengembalikan true
arr2[0][1];   // mengembalikan "dua"
data[10];     // mengembalikan undefined
```

Sebagai sebuah objek khusus, *array* juga memiliki *method* dan properti. Beberapa *method* dan properti yang populer misalnya *length*, *pop()*, dan *push()*.


```
var data = ["a", "b", "c"];  
// data.length mengembalikan 3  
data.push("d"); // mengembalikan 4 data menjadi ["a", "b", "c", "d"]  
data.pop(); // mengembalikan "d", data menjadi ["a", "b", "c"]
```

11.4.2.4 Pengendalian Struktur

Javascript memiliki perintah-perintah pengendalian struktur (*control structure*) yang sama dengan bahasa dalam keluarga C. Perintah `if` dan `else` digunakan untuk percabangan, sementara perintah `for`, `for-in`, `while`, dan `do-while` digunakan untuk perulangan.

Percabangan pada Javascript bisa dikatakan sama persis dengan C atau Java:

```
var gelar;  
var pendidikan = "S2";  
if (pendidikan === "S1") {  
    gelar = "Sarjana";  
} else if (pendidikan === "S2") {  
    gelar = "Master";  
} else if (pendidikan === "S3") {  
    gelar = "Doktor";  
} else {  
    gelar = "Tidak Diketahui";  
}  
gelar; // gelar berisi "Master"
```

Satu hal yang perlu diperhatikan, tiga buah sama dengan (`===`) digunakan pada operasi perbandingan di Javascript. Javascript mendukung dua operator perbandingan sama dengan, yaitu `==` dan `===`. Perbedaan utamanya adalah `==` mengubah tipe data yang dicek menjadi nilai terdekat, sementara `===` memastikan tipe data dari dua nilai yang dibandingkan sama. Untuk mendapatkan nilai perbandingan paling akurat, selalu gunakan `===` ketika mengecek nilai.

Sama seperti `if`, perulangan `do`, `do-while`, dan `for` memiliki cara pemakaian yang dapat dikatakan sama persis dengan C atau Java:

```
while (true) {  
    // tak pernah berhenti  
} var input; do {  
    input = get_input();  
} while (inputIsValid(input)) for  
(var i = 0; i < 5; i++) {  
    // berulang sebanyak 5 kali }
```

11.4.3 Object Orientation pada Javascript

Javascript memiliki dua jenis tipe data utama, yaitu tipe data dasar dan objek. Tipe data dasar pada Javascript adalah angka (*numbers*), rentetan karakter (*strings*), boolean (*true* dan *false*), *null*, dan *undefined*. Nilai-nilai selain tipe data dasar secara otomatis dianggap sebagai objek. Objek dalam Javascript didefinisikan sebagai *mutable properties collection*, yang artinya adalah sekumpulan properti (ciri khas) yang dapat berubah nilainya. Karena nilai-nilai selain tipe data dasar merupakan objek, maka pada Javascript sebuah *Array* adalah objek. Fungsi adalah objek dan *Regular expression* juga merupakan objek.

11.4.3.1 Pembuatan Object pada Javascript

Notasi pembuatan objek pada Javascript sangat sederhana, yaitu sepasang kurung kurawal yang membungkus properti. Notasi pembuatan objek ini dikenal dengan nama *object literal*. *Object literal* dapat digunakan kapanpun pada ekspresi Javascript yang valid:

```
var objek_kosong = {};  
var mobil = {  
  "warna-badan": "merah",  
  "nomor-polisi": "BK1234AB"  
};
```

Nama properti dari sebuah objek harus berupa *string*, dan boleh berisi *string* kosong (""). Jika merupakan nama Javascript yang legal, kita tidak memerlukan petik ganda pada nama properti. Petik ganda seperti pada contoh ("warna-badan") hanya diperlukan untuk nama Javascript ilegal atau kata kunci seperti "if" atau "var". Misalnya, "nomor-polisi" memerlukan tanda petik, sementara nomor_polisi tidak. Contoh lain, variasi tidak memerlukan tanda petik, sementara "var" perlu.

Sebuah objek dapat menyimpan banyak properti, dan setiap properti dipisahkan dengan tanda koma (,). Jika ada banyak properti, nilai dari properti pada setiap objek boleh berbeda-beda:

```
var jadwal = {  
  platform: 34,  
  telah_berangkat:  
false,  
  tujuan: "Medan",  
  asal: "Jakarta"  
};
```



Karena dapat diisi dengan nilai apapun (termasuk objek), maka kita dapat membuat objek yang mengandung objek lain (*nested object*; objek bersarang) seperti berikut:

```
var jadwal =  
{  
  platform: 34,  
  telah_berangkat: false,  
  asal:  
  {  
    kode_kota:  
"MDN",      nama_kota:  
"Medan",    waktu:  
"2013-12-29 14:00"  
  },      tujuan:  
  {  
    kode_kota:  
"JKT",      nama_kota:  
"Jakarta",  waktu:  
"2013-12-29 17.30"  
  }  
};
```

11.4.3.2 Akses Nilai Property

Akses nilai properti dapat dilakukan dengan dua cara, yaitu.

1. Penggunaan kurung siku ([]) setelah nama objek. Kurung siku kemudian diisi dengan nama properti, yang harus berupa *string*. Cara ini biasanya digunakan untuk nama properti yang adalah nama ilegal atau kata kunci Javascript.
2. Penggunaan tanda titik (.) setelah nama objek diikuti dengan nama properti. Notasi ini merupakan notasi yang umum digunakan pada bahasa pemrograman lainnya. Notasi ini tidak dapat digunakan untuk nama ilegal atau kata kunci Javascript.

Contoh penggunaan kedua cara pemanggilan di atas adalah sebagai berikut:

```
mobil["warna-badan"] // Hasil: "merah"
jadwal.platform      // Hasil: 34
```

Sebagai bahasa dinamis, Javascript tidak akan melemparkan pesan kesalahan jika kita mengakses properti yang tidak ada dalam objek. Kita akan menerima nilai *undefined* jika mengakses properti yang tidak ada:

```
jadwal.nomor_kursi // Hasil: undefined mobil
["jumlah-roda"]   // Hasil: undefined
```

Pengaksesan properti pada Javascript juga dapat digunakan secara dinamis untuk mengubah nilai dari properti tersebut. Perubahan nilai properti juga dapat dilakukan untuk properti yang bahkan tidak ada pada objek tersebut:

```
mobil["jumlah-roda"] = 4;
mobil.bahan_bakar = "Bensin";
```

11.4.3.3 Prototype pada Javascript

Pada Javascript yang mengimplementasikan PBO kita tidak lagi perlu menuliskan kelas, dan langsung melakukan penurunan terhadap objek. Misalkan kita memiliki objek mobil yang sederhana seperti berikut:

```
var mobil = { nama:
  "Mobil", jumlahBan: 4 };
```

Kita dapat langsung menurunkan objek tersebut dengan menggunakan fungsi `Object.create` seperti berikut:

```
var truk =
  Object.create(mobil);
// truk.nama === "Mobil"
// truk.jumlahBan === 4
```

11.4.4 Function pada Javascript

Sebuah fungsi membungkus satu atau banyak perintah. Setiap kali fungsi dipanggil, maka perintah-perintah yang ada di dalam fungsi tersebut dijalankan. Secara umum fungsi digunakan untuk penggunaan kembali kode (*code reuse*) dan penyimpanan informasi (*information hiding*). Implementasi fungsi kelas pertama juga memungkinkan penggunaan fungsi sebagai unit-unit yang dapat dikombinasikan, seperti layaknya sebuah lego. Dukungan terhadap pemrograman berorientasi objek juga berarti fungsi dapat digunakan untuk memberikan perilaku tertentu dari sebuah objek.

11.4.4.1 Pembuatan Fungsi pada Javascript

Sebuah fungsi pada Javascript dibuat dengan cara seperti berikut:

```
function tambah(a, b)
{
    hasil = a + b;
    return hasil;
}
```

Cara penulisan fungsi seperti diatas dikenal dengan nama *function declaration*, atau deklarasi fungsi. Terdapat empat komponen yang membangun fungsi di atas, yaitu:

1. Kata kunci *function*, yang memberitahu Javascript bahwa akan dibuat sebuah fungsi.
2. Nama fungsi, dalam contoh di atas adalah tambah. Dengan memberikan sebuah fungsi nama maka pemanggilan fungsi dapat dirujuk dengan nama tersebut. Harus diingat bawa nama fungsi bersifat opsional, yang berarti **fungsi pada Javascript tidak harus diberi nama**.
3. Daftar parameter fungsi, yaitu a, b pada contoh di atas. Daftar parameter ini selalu dikelilingi oleh tanda kurung (). Parameter boleh kosong, tetapi tanda kurung wajib tetap dituliskan. Parameter fungsi akan secara otomatis didefinisikan menjadi variabel yang hanya bisa dipakai di dalam fungsi. Variabel pada parameter ini diisi dengan nilai yang dikirimkan kepada fungsi secara otomatis.
4. Sekumpulan perintah yang ada di dalam kurung kurawal {}. Perintah-perintah ini dikenal dengan nama badan fungsi. Badan fungsi dieksekusi secara berurut ketika fungsi dijalankan.

Penulisan deklarasi fungsi (*function declaration*) seperti di atas merupakan cara penulisan fungsi yang umumnya kita gunakan pada bahasa pemrograman imperatif dan berorientasi objek. Tetapi selain deklarasi fungsi Javascript juga mendukung cara penulisan fungsi lain, yaitu dengan memanfaatkan ekspresi fungsi (*function expression*). Ekspresi fungsi merupakan cara pembuatan fungsi yang memperbolehkan menuliskan fungsi tanpa nama. Fungsi yang dibuat tanpa nama dikenal dengan sebutan fungsi anonim atau fungsi lambda. Berikut adalah cara membuat fungsi dengan ekspresi fungsi:

```
var tambah = function (a, b)
{
    hasil = a + b;
    return hasil;
};
```

Terdapat hanya sedikit perbedaan antara ekspresi fungsi dan deklarasi fungsi:

1. Penamaan fungsi. Pada deklarasi fungsi, nama fungsi langsung diberikan sesuai dengan sintaks yang disediakan Javascript. Penggunaan ekspresi fungsi pada dasarnya menyimpan sebuah fungsi anonim ke dalam variabel dan nama fungsi adalah nama variabel yang dibuat. Perlu diingat juga bahwa pada dasarnya ekspresi fungsi adalah fungsi anonim. Penyimpanan ke dalam variabel hanya diperlukan karena kita akan memanggil fungsi nantinya.
2. Ekspresi fungsi dapat dipandang sebagai sebuah ekspresi atau perintah standar bagi Javascript, sama seperti penulisan kode `var i = 0`. Deklarasi fungsi merupakan konstruksi khusus untuk membuat fungsi. Hal ini berarti pada akhir dari ekspresi fungsi kita harus menambahkan, sementara pada deklarasi fungsi hal tersebut tidak penting.

11.4.4.2 Pemanggilan Fungsi

Sebuah fungsi dapat dipanggil untuk menjalankan seluruh kode yang ada di dalam fungsi tersebut, sesuai dengan parameter yang kita berikan. Pemanggilan fungsi dilakukan dengan cara menuliskan nama fungsi tersebut, kemudian mengisi argumen yang ada di dalam tanda kurung.

Misalkan fungsi tambah yang kita buat pada bagian sebelumnya:

```
var tambah = function (a, b)
{
    hasil = a + b;
    return hasil;
};
```

dapat dipanggil seperti berikut:

```
tambah(3, 5);
```

Yang terjadi pada kode di atas adalah nilai a dan b masing-masing digantikan dengan 3 dan 5. Seperti yang dapat dilihat, hal ini berarti pengisian argumen pada saat pemanggilan fungsi harus berurut sesuai dengan deklarasi fungsi.

Sama seperti sebuah variabel, fungsi juga mengembalikan nilai ketika dipanggil. Dalam kasus di atas, tambah(3, 5) akan mengembalikan nilai 8. Nilai ini tentunya dapat disimpan ke dalam variabel baru, atau bahkan dikirimkan sebagai sebuah argumen ke fungsi lain lagi:

```
var simpan = tambah(3, 5); // simpan === 8
tambah(simpan, 2);         // mengembalikan 10
tambah(tambah(3, 5), 2)    // juga mengembalikan 10
tambah(tambah(2, 3), 4)    // mengembalikan 9
```

Fungsi akan mengembalikan nilai ketika kata kunci *return* ditemukan. Pengembalian nilai fungsi dapat dilakukan kapanpun, dan fungsi akan segera berhenti ketika kata kunci *return* ditemukan. Berikut adalah contoh kode yang memberikan gambaran tentang pengembalian nilai fungsi:

```
var naikkan = function (n) {
    var hasil = n + 10; return
    hasil;
    // kode di bawah tidak dijalankan lagi
    hasil = hasil * 100;
} naikkan(10); // mengembalikan
20 naikkan(25); // mengembalikan
35
```

Sebuah ekspresi dapat juga diberikan langsung kepada *keyword return*, dan ekspresi tersebut akan dijalankan sebelum nilai dikembalikan. Hal ini berarti fungsi tambah maupun naikkan yang sebelumnya bisa disederhanakan dengan tidak lagi menyimpan nilai di variabel hasil terlebih dahulu:

```
var naikkan = function (n) {  
  return n + 10;  
} var tambah = function (a,  
b) {      return a + b;  
} tambah(4, 4);           // mengembalikan 8  
naikkan(10);              // mengembalikan 20  
tambah(naikkan(5), 7);    // mengembalikan 22
```



Modul 12 JSP

12.1 Pengenalan JSP

JSP adalah teknologi yang memungkinkan kita untuk menambahkan konten dinamis ke halaman web. Tanpa JSP, jika kita ingin memperbarui tampilan atau konten halaman HTML statis, kita harus melakukannya secara manual. Misalnya, jika hanya ingin mengubah tanggal atau gambar, kita perlu mengedit file HTML secara langsung. Namun, dengan JSP, kita bisa membuat konten yang dapat berubah secara otomatis berdasarkan banyak faktor, seperti waktu, informasi yang diberikan oleh pengguna, riwayat interaksi pengguna dengan situs, atau bahkan jenis browser yang digunakan. Kemampuan ini sangat penting untuk menyediakan layanan online yang bisa menyesuaikan respon sesuai dengan kebutuhan dan preferensi setiap pengguna. Salah satu hal penting dalam menyediakan layanan online yang berarti adalah kemampuan sistem untuk mengingat data yang terkait dengan layanan dan penggunanya. Karena itulah, basis data menjadi sangat penting dalam pembuatan halaman web yang dinamis.

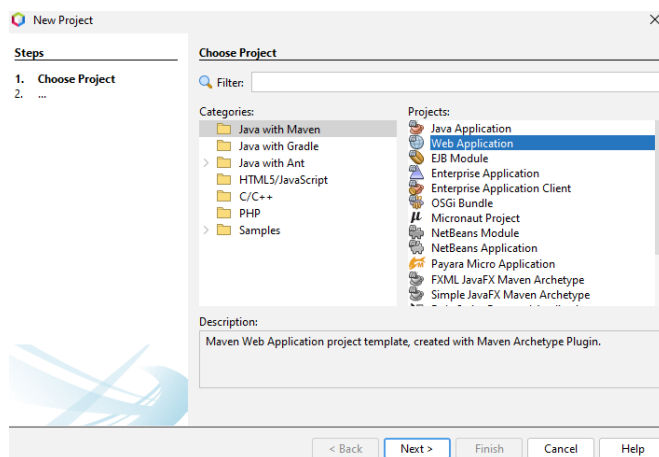
12.2 Java Web Server

Di dalam Netbeans, untuk dapat menggunakan JSP, diperlukan Java Web Server, contohnya Tomcat. Proses instalasi dapat dilakukan sama seperti pembuatan project baru yang hanya mengikuti alur pembuatan hingga selesai, jika sebelumnya pada java with maven memilih Java Application, maka untuk menggunakan JSP kita memilih pilihan Web Application.

1) Instalasi Tomcat Server

Sebelum membuat program web java menggunakan Netbeans, langkah pertama adalah instalasi Tomcat terlebih dahulu.

- a) Jalankan *menu File | Project* (Ctrl + Shift +N) untuk membuka suatu dialog New Project.



Gambar 12-1 Tampilan Dialog New Project

- b) Dalam dialog, pilih *Categories : Java with Maven* dan *Projects : Web Application*. Lalu klik tombol next.
- c) Pilih dahulu lokasi project dengan menekan tombol *browse*. Akan tampil direktori dan kita dapat memilih di direktori mana project disimpan. Setelah memilih, klik open.

- d) Kembali ke dialog *new project*, isikan *project name*. Misalnya disini *project name* : *HelloJavaWeb*. Perhatikan bahwa *project folder* secara otomatis akan diset sesuai dengan *project name*. Lalu klik tombol next.
- e) Akan muncul dialog settings, pilih Java EE Version : **Java EE 7 Web** dan Server : **Apache Tomcat or TomEE** jika sudah terinstall. Jika belum, klik tombol add dan klik Apache Tomcat or TomEE, lalu klik tombol next.
- f) Klik browse pada kolom server location dengan memilih letak directory folder Tomcat yang terdapat pada folder XAMPP, lalu klik open.
- g) Terakhir masukkan *username* : praktikan dan *password* : praktikan, lalu klik Finish.

2) Memulai membuat java web sederhana

Setelah instalasi Tomcat server di Netbeans, selanjutnya membuat program sederhana. Di bawah ini merupakan contoh program yang akan menampilkan “Hello Java Web!”.

```
<!DOCTYPE html>
<html>
  <head>
    <tittle>Start Page</tittle>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
  </head>
  <body>
    <h1>Hello Java Web!</h1>
  </body>
</html>
```

3) Meng-compile dan menjalankan java web di Netbeans

Untuk meng-*compile* dan menjalankan, klik kanan pada index.html dan pilih *run file*. Jika web java sukses, maka akan tampil tulisan *Hello Java Web!* seperti di bawah ini :



Gambar 12-2 Tampilan hasil Compile dan run java web

Modul 13 Java Servlet & JDBC Part 1

Tujuan Praktikum

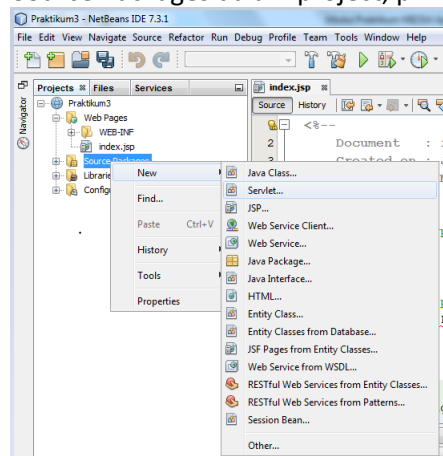
1. Mengerti konsep *Java Web* dalam Java dan dapat mengimplementasikannya dalam sebuah web sederhana.
2. Mengimplementasikan JDBC ke dalam suatu program java yang menggunakan JSP
3. Mengimplementasikan metode fetching dan create pada konsep JDBC

13.1 Java Servlet

Java Web Server memiliki Servlet engine untuk memproses konten secara dinamis. Servlet bertindak sebagai controller dalam aplikasi.

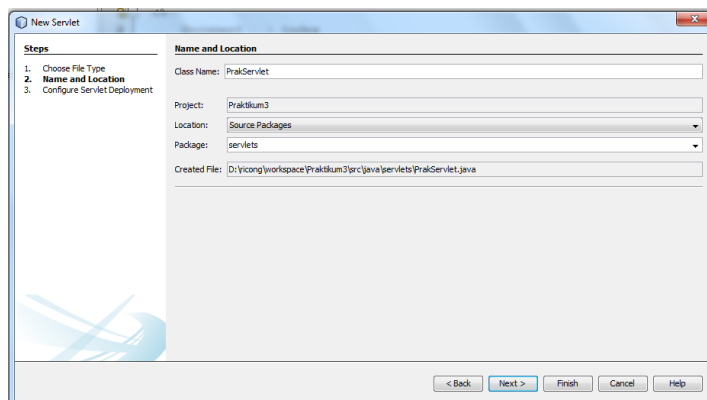
Berikut adalah contoh sederhana implementasi servlet:

- 1) Membuat servlet dalam aplikasi web
 - a. Klik kanan pada folder Source Packages dalam project, pilih Servlet



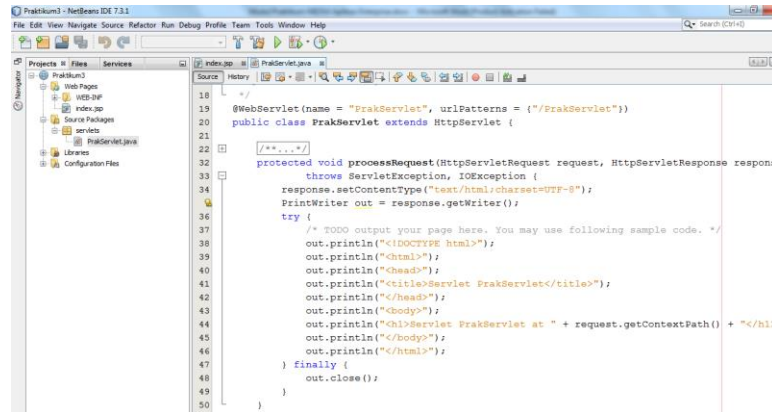
Gambar 13-1 Tampilan menu membuat Servlet

- b. Beri nama servlet 'PrakServlet' dan isi field *package* dengan 'servlets', lalu klik Finish



Gambar 13-2 Tampilan dialog New Servlet

- c. Servlet berhasil dibuat



Gambar 13-3 Tampilan utama Servlet

- 2) Buat form pada index.jsp untuk mengirim data ke PrakServlet dan menampilkan hasil kembalian dari PrakServlet

```

<form method="get" action="PrakServlet">
    NIM: <input type="text" name="nim" /> <br />
    Nama: <input type="text" name="nama" /> <br />
    <input type="submit" value="Kirim" />
</form>
<%
    out.print(request.getAttribute("nim")+"<br />");
    out.print(request.getAttribute("nama")+"<br />");
%>

```

- 3) Hapus semua kode di dalam prosedur processRequest() pada PrakServlet.java dan buat kode agar dapat menerima data dari index.jsp lalu mengirim data yang telah diubah ke index.jsp

```

String nim = request.getParameter("nim");
String nama = request.getParameter("nama");
nim = "NIM Anda adalah: "+nim;
nama = "Nama Anda adalah: "+nama;
request.setAttribute("nim", nim);
request.setAttribute("nama", nama);
request.getRequestDispatcher("index.jsp").forward( request, response);

```

13.2 Memulai JDBC

Java juga menyediakan fungsi untuk menghubungkan suatu aplikasi yang dibangun dengan menggunakan bahasa pemrograman java dengan database. Untuk menghubungkannya menggunakan JDBC. Dengan **JDBC**, programmer dapat **menggunakan SQL statement untuk membentuk, memanipulasi atau memelihara suatu data**.

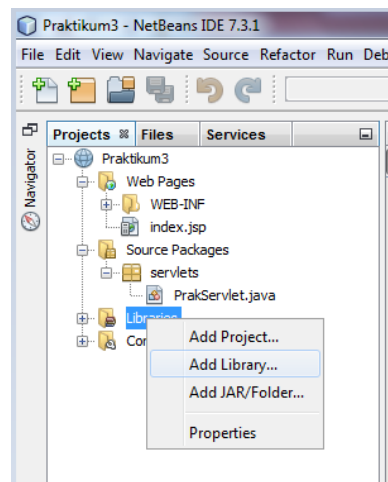
Java Database Connectivity adalah versi ODBC yang dibuat oleh Sun Microsystem. Cara kerjanya mirip tapi JDBC sepenuhnya dibangun dengan java API sehingga ia dapat dipakai cross platform sedangkan ODBC dibangun dengan bahasa C sehingga ia hanya dapat dibangun pada platform spesifik. Seperti ODBC, JDBC didasarkan pada **X/Open SQL Level Interface (CLI)**.

Dengan JDBC API, kita dapat mengakses database. Untuk itu, diperlukan *driver* yang akan dipakai untuk mengakses *database* di server dari dalam program kita (*client*). Setiap server dari vendor berbeda memiliki *driver* yang berbeda. *Driver* ini dapat kita *download* dari situs Java atau dari situs vendor yang bersangkutan.

Dengan JDBC, kita dapat melakukan mengirimkan perintah-perintah SQL ke RDBMS. Kelas-kelas serta *interface* JDBC API dapat diakses dalam paket Java, sql(core API) atau javax.sql (Standard Extension API).

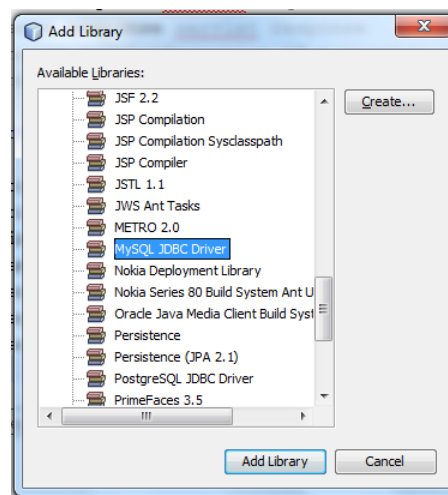
Berikut adalah contoh sederhana implementasi JDBC:

- 1) Koneksi aplikasi web dengan JDBC
 - a. Jalankan MySQL server. Pastikan database server memiliki user root dan tanpa password, dan memiliki database bernama 'test'. Dalam database 'test' buat tabel 'barang' dengan 2 kolom yaitu 'id' (int(5), primary key, auto-increment) dan 'nama' (varchar(20)).
 - b. Klik kanan pada folder Libraries di dalam project, pilih Add Library



Gambar 13-4 Tampilan Menu Libraries

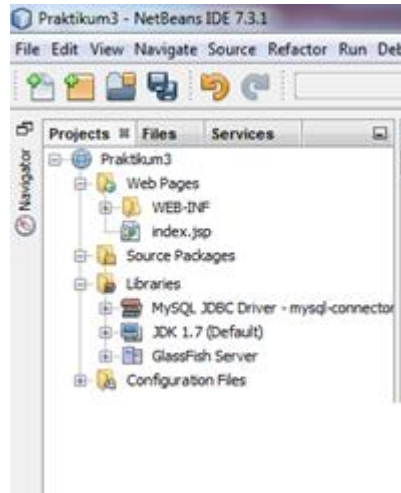
- c. Pilih MySQL JDBC Driver, klik Add Library



Gambar 13-5 Tampilan dialog Add Library

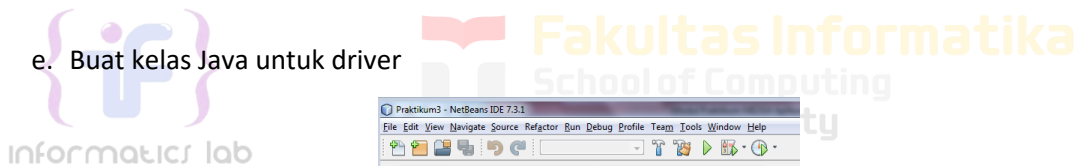
Jika MySQL JDBC Driver tidak terdapat pada Library, kita bisa menambahkan secara manual. Download di <https://dev.mysql.com/downloads/connector/j/>, pilih Platform Independent. Ekstrak file hasil download dan pindahkan ke dalam folder project. Kemudian klik kanan pada folder Libraries di dalam project, pilih Add JAR/Folder. Arahkan ke file yang sudah diekstrak tadi secara Relative Path.

d. JDBC driver untuk MySQL sudah ditambahkan



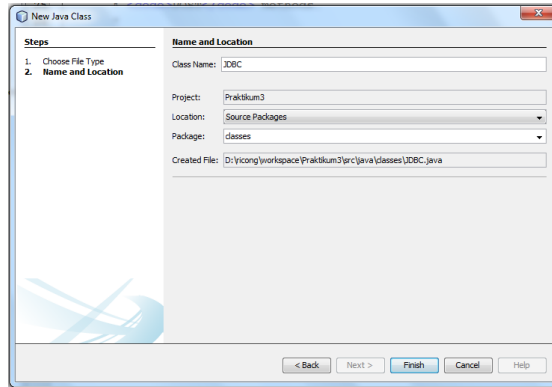
Gambar 13-6 Tampilan setelah Add Library JDBC

e. Buat kelas Java untuk driver



Gambar 13-7 Tampilan menu menambahkan Class

f. Beri nama kelas 'JDBC', isi field package dengan 'classes'



Gambar 13-8 Tampilan dialog New Java Class

- g. Tambahkan "import java.sql.*" pada kelas JDBC.java, dan deklarasikan atribut. Buat getter untuk atribut message

```
package classes;
import java.sql.*;

public class JDBC {
    private Connection con;
    private Statement stmt;
    private boolean isConnected;
    private String message;
}
```

- h. Buat prosedur di kelas JDBC.java untuk membuka koneksi dengan database dengan parameter yang digunakan dalam koneksi database

```
public void connect() {
    String dbname = "test";
    String username = "root";
    String password = "";
    try {
        Class.forName("com.mysql.jdbc.Driver");
        con = DriverManager.getConnection("jdbc:mysql://localhost:3306/"
+ dbname, username, password);
        stmt = con.createStatement();
        isConnected = true;
        message = "DB connected";
    } catch (Exception e) {
        isConnected = false;
        message = e.getMessage();
    }
}
```

- i. Buat prosedur untuk menutup koneksi database

```
private void disconnect() {
    try {
        stmt.close();
        con.close();
    } catch (Exception e) {
        message = e.getMessage();
    }
}
```

2) Melakukan query Sederhana

- a. Buat prosedur untuk menjalankan query database (DDL / DML) di dalam kelas JDBC.java

```
public void runQuery(String query) {
    try {
        connect();
        int result = stmt.executeUpdate(query);
        message = "info: " + result + " rows affected";
    } catch (Exception e) {
        message = e.getMessage();
    } finally {
        disconnect();
    }
}
```

- b. Buat package 'servlets' dan Servlet baru TestServlet di dalamnya, panggil runQuery di dalam processRequest() untuk menguji koneksi database dan melakukan query insert, output hasilnya

```
JDBC db = new JDBC();
db.runQuery("insert into barang (nama) values ('PC')");
response.setContentType("text/html;charset=UTF-8");
try (PrintWriter out = response.getWriter()) {
    out.println("<!DOCTYPE html>");
    out.println("<html>");
    out.println("<head>");
    out.println("<title>Servlet Test</title>");
    out.println("</head>");
    out.println("<body>");
    out.println(db.getMessage() + "<br />");
    out.println("</body>");
    out.println("</html>");
}
```

13.3 Fetching Data

Berikut adalah contoh penggunaan kode untuk menghapus sebuah data yang diinginkan dari sebuah database.

- 1) Buat fungsi pada kelas JDBC.java untuk mengambil data dari database

```
public ResultSet getData(String query) {
    ResultSet rs = null;
    try {
        connect();
        rs = stmt.executeQuery(query);
    } catch (Exception e) {
        message = e.getMessage();
    }
    return rs;
}
```

- 2) Buat Servlet baru, 'BarangServlet' dan letakkan di dalam package 'servlets'. Ganti kode pada prosedur processRequest() menjadi kode untuk memanggil fungsi pengambilan data dari database

```
JDBC db = new JDBC();
ResultSet rs = db.getData("select * from barang");
request.setAttribute("list", rs);
request.getRequestDispatcher("daftarbarang.jsp").forward(request,
response);
```

- 3) Buat halaman JSP dengan nama 'daftarbarang'. Tambahkan JSP directive diatasnya untuk mengimpor kelas ResultSet agar dapat digunakan pada halaman

```
<%@page import="java.sql.ResultSet"%>
```

- 4) Buat scriptlet pada halaman daftarbarang.jsp untuk menerima data dari Servlet, dan tampilkan dalam bentuk tabel

```
<table border="1">
  <tr>
    <td>ID</td>
    <td>Nama</td>
  </tr>
  <%
    ResultSet rs = (ResultSet)request.getAttribute("list");
    if (rs != null) {
      while (rs.next()) {
        out.print("<tr>");
        out.print("<td>" + rs.getInt("id") + "</td>");
        out.print("<td>" +rs.getString("nama")+ "</td>");
        out.print("</tr>");
      }
    }
  %>
</table>
```

Dengan menggunakan kode di atas, saat halaman BarangServlet diakses, program akan mengambil data dari tabel barang dalam database. Data yang diambil kemudian ditampilkan dalam bentuk tabel di halaman tersebut, menampilkan kolom ID dan Nama untuk setiap barang yang ada di database.

13.4 Insert Data

Berikut adalah contoh penggunaan kode untuk *insert* sebuah data yang diinginkan ke database.

- 1) Buka servlet 'BarangServlet', ganti dan tambahkan kode pada prosedur processRequest() menjadi kode untuk menerima parameter dari URL BarangServlet. Tambahkan aksi untuk menambah data ke database, lalu kembali ke daftar barang

```
JDBC db = new JDBC();
String menu = request.getParameter("menu");
if (request.getParameterMap().isEmpty()) { //view menu
  ResultSet rs = db.getData("select * from barang");
  request.setAttribute("list", rs);
  request.getRequestDispatcher("daftarbarang.jsp").forward(request,
  response);
} else if ("add".equals(menu)) { //add menu
  request.getRequestDispatcher("tambahbarang.jsp").forward(request,
  response);
} else if ("insert".equals(menu)) { //insert action
  String nama = request.getParameter("nama");
  db.runQuery("insert into barang (nama) values ('" + nama + "')");
  response.sendRedirect("BarangServlet");
}
```

- 2) Buat halaman JSP 'tambahbarang.jsp', dan isi dengan form penambahan barang

```
<form method="post" action="BarangServlet?menu=insert">  
  Nama Barang: <input type="text" name="nama" /><br />  
  <input type="submit" value="Tambah" />  
</form>
```

- 3) Uji dengan membuka URL .../BarangServlet?menu=add



Modul 14 JDBC part 2

Tujuan Praktikum

1. Mengimplementasikan metode update pada konsep JDBC
2. Mengimplementasikan metode delete pada konsep JDBC

14.1 Pendahuluan

Setelah mempelajari tentang cara memulai dan mengakhiri koneksi program dengan database menggunakan JDBC, memasukkan data ke database, dan juga mengambil data dari database (*Create* dan *Read*), kita akan mempelajari cara untuk mengedit dan menghapus sebuah data pada database (*Update* dan *Delete*).

14.2 Update Data

Berikut adalah contoh penggunaan kode untuk mengubah sebuah data yang diinginkan dari sebuah database.

- 1) Modifikasi `BarangServlet`, tambahkan untuk pengubahan ke database

```
...
} else if ("insert".equals(menu)) { //insert action
    String nama = request.getParameter("nama");
    db.runQuery("insert into barang (nama) values ('" + nama + "')");
    response.sendRedirect("BarangServlet");
} else if ("edit".equals(menu)) { //edit menu
    String nama = "";
    String id = request.getParameter("id");
    ResultSet rs = db.getData("select * from barang where id = '" + id +
    "'");
    request.setAttribute("list", rs);
    request.getRequestDispatcher("editbarang.jsp").forward(request,
    response);
} else if ("update".equals(menu)) { //update action
    String id = request.getParameter("id");
    String nama = request.getParameter("nama");
    db.runQuery("update barang set nama = '" + nama + "' where id = '" +
    id + "'");
    response.sendRedirect("BarangServlet");
}
```

- 2) Buat halaman JSP 'editbarang', yang mengambil data dari database sesuai ID dan di-input ke dalam field pada form

```
<%@page import="java.sql.ResultSet"%>
<%
    String nama = "";
    ResultSet rs = (ResultSet)request.getAttribute("list");
    if (rs.next()) {
        nama = rs.getString("nama");
    }
%>
<form method="post" action="BarangServlet?menu=update&id= <%=id%>">
    Nama Barang: <input type="text" name="nama" value="<%=nama%>" /> <br
/>
    <input type="submit" value="Edit" />
</form>
```

- 3) Modifikasi daftarbarang.jsp, tambahkan satu kolom Aksi untuk pengaksesan ke halaman editbarang.jsp dan untuk menghapus data berdasarkan ID data pada scriptlet

```
...  
<td>Nama</td>  
<td>Aksi</td>  
...  
out.print("<td>" + "<a href='BarangServlet?m=del&id=" +  
    rs.getInt("id") + "'>Hapus</a>" +  
    " &nbsp; <a href='BarangServlet?menu=edit&id=" +  
    rs.getInt("id") + "'>Edit</a>" +  
    "</td>");  
out.print("</tr>");  
...
```

Dengan menggunakan kode di atas, saat pengguna menekan tombol "Edit", program akan mengambil data berdasarkan parameter 'id' dari barang yang dipilih, kemudian mengarahkan pengguna ke halaman form edit. Setelah pengguna menyimpan data, program akan memperbarui entri barang di tabel sesuai dengan perubahan yang diinginkan.

14.3 Delete Data

Berikut adalah contoh penggunaan kode untuk menghapus sebuah data yang diinginkan dari sebuah database.

- 1) Modifikasi BarangServlet, tambahkan untuk penghapusan ke database

```
...  
} else if ("update".equals(menu)) { //update action  
    String id = request.getParameter("id");  
    String nama = request.getParameter("nama");  
    db.runQuery("update barang set nama = '" + nama + "' where id = '" +  
id + "'");  
    response.sendRedirect("BarangServlet");  
} else if ("del".equals(menu)) { //del action  
    String id = request.getParameter("id");  
    db.runQuery("delete from barang where id = '" + id + "'");  
    response.sendRedirect("BarangServlet");  
}  
}
```

- 2) Saat tombol "Delete" ditekan, program akan menghapus data dari table berdasarkan parameter 'id' yang diperoleh. Hal ini memastikan bahwa barang yang dihapus sesuai dengan barang yang dipilih oleh pengguna

Modul 15 JDBC Part 3 & Session

Tujuan Praktikum

1. Mengimplementasikan session pada Java Web

15.1 Pendahuluan

Setelah mempelajari tentang cara memulai dan mengakhiri koneksi program dengan database menggunakan JDBC dan manajemen data dari database (*Create, Read, Update, & Delete*), kita akan mempelajari cara untuk menggunakan session.

15.2 Session

Session adalah mekanisme yang digunakan untuk menyimpan informasi tentang pengguna antara permintaan HTTP yang berbeda. Karena HTTP adalah protokol yang stateless (tidak menyimpan status antara permintaan), session memungkinkan aplikasi web untuk "mengingat" pengguna saat mereka berpindah dari satu halaman ke halaman lain dalam situs yang sama. Dalam JSP, session dikelola melalui objek `HttpSession`, yang secara otomatis dibuat untuk setiap pengguna ketika mereka mengakses aplikasi untuk pertama kali. Informasi yang disimpan dalam session bisa berupa data user seperti nama pengguna, preferensi, atau bahkan objek kompleks seperti keranjang belanja. Data ini akan tetap ada selama session aktif, biasanya sampai pengguna menutup browser atau session berakhir karena batas waktu.

Berikut adalah contoh sederhana implementasi session untuk autentikasi:

- 1) Buat halaman JSP 'form_login'

```
<form method="post" action="LoginServlet">
  Username: <input type="text" name="user" /> <br />
  Password: <input type="password" name="pass" /> <br />
  <input type="submit" value="Login" />
</form>
```

- 2) Buat servlet 'LoginServlet', hapus prosedur `processRequest()` dan ganti kode dalam prosedur `doGet()` dan `doPost()` dengan kode untuk membuat atau menghapus session sesuai username & password yang dikirim dari halaman `form_login.jsp`

```
protected void doGet(HttpServletRequest request, HttpServletResponse
response)
    throws ServletException, IOException {
    if (request.getParameter("logout") != null) {
        request.getSession().invalidate();
    }
    request.getRequestDispatcher("form_login.jsp").forward(request,
response);
}

protected void doPost(HttpServletRequest request, HttpServletResponse
response)
    throws ServletException, IOException {
    String username = request.getParameter("user");
    String password = request.getParameter("pass");
    if ("admin".equals(username) && "admin123".equals(password)) {
        request.getSession().setAttribute("username", username);
        response.sendRedirect("BarangServlet");
    } else {
```

```
request.getRequestDispatcher("form_login.jsp").forward(request,  
response);  
}  
}
```

Dengan menggunakan kode di atas, memungkinkan pengguna untuk membuat atau menghancurkan session. Jika pengguna login di URL .../LoginServlet dengan username 'admin' dan password 'admin123', session baru akan dibuat. Jika pengguna mengakses URL .../LoginServlet?logout, session yang ada akan dihapus.



Modul 16 Responsi & Progress Tugas Besar

Tujuan Praktikum
1. Mahasiswa mempresentasikan progress tugas besar Java Web yang terintegrasi dengan database dan pengelolaan konten yang dinamis.

16.1 Responsi & Progress Tugas Besar

Pada Modul 16 ini, mahasiswa diharapkan sudah memahami konsep Java Web dan dapat merancang serta mengembangkan aplikasi web untuk tugas besar mereka. Mahasiswa juga diharapkan mampu mempresentasikan hasil perkembangan aplikasi yang telah dibuat kepada asisten praktikum, serta menjelaskan setiap langkah yang diambil dalam pengembangan JSP, termasuk integrasi dengan database dan pengelolaan konten dinamis.



Daftar Pustaka

- [1] Anonim. 2012. *Modul Praktikum Pemrograman Berorientasi Objek*. Fakultas Informatika. Institut Teknologi Telkom, Bandung.
- [2] Barclay, K. & J.Savage.2004.*Object-Oriented Design with UML and Java*. Massachusetts:Elsevier Butterworth-Heinemann.
- [3] Bruegge , Bernd & Allen H. Dutoit .2010. *Object-Oriented Software Engineering Using UML, Patterns, and Java™ Third Edition*.Pennsylvania: Prentice Hall.
- [4] Deitel, H.M & Deitel,P.J. 2004. *Java How to Program Sixth Edition*. New Jersey: Prentice Hall.
- [5] Downey, Tim. 2012. *Guide to Web Development with Java*. Springer
- [6] Sierra, Kathy & Bates, Bert. 2005. *Head First Java, 2nd Edition*. Sebastopol: O'Reilly Media.
- [7] Sierra, Kathy & Bates, Bert. 2008. *SCJP Sun Certified Programmer for Java™ 6 Study Guide*. New York: McGraw-Hill
- [8] Edward Hill, Jr. 2005. *Learning to Program Java*. New York: iUniverse.
- [9] Java Tutorial. <https://beginnersbook.com/java-tutorial-for-beginners-with-examples/>. 28 Desember 2018.
- [10] Java Tutorial. <https://tutorialspoint.com/java/>. 28 Desember 2018.
- [11] Java Tutorial. <https://w3schools.com/java>. 28 Desember 2018.
- [12] Java SE Documentations. <http://docs.oracle.com/javase>. 28 Desember 2018.
- [13] Zambon, Giulio. 2012. *Beginning JSP, JSF, and Tomcat*. Apress



Kontak Kami :



@fiflab



Praktikum IF LAB



informaticslab@telkomuniversity.ac.id



informatics.labs.telkomuniversity.ac.id